```
from google.colab import files
uploaded = files.upload()
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving brazilian_ecommerce_olist.csv to brazilian_ecommerce_olist (1).csv
```

```
# ==========================
# EDA - Brazilian E-commerce Olist Dataset
# ==========================

# 1. Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# 2. Load Dataset
df = pd.read_csv("/content/brazilian_ecommerce_olist.csv")

# 3. Basic Information
print("First 5 Rows")
print(df.head())

print("\nLast 5 Rows")
print(df.tail())

print("\nShape of Dataset")
print(df.shape)

print("\nColumn Names")
print(df.columns)

print("\nDataset Information")
print(df.info())

print("\nData Types")
print(df.dtypes)

# 4. Missing Values
print("\nMissing Values")
print(df.isnull().sum())

# 5. Duplicate Values
print("\nDuplicate Rows")
print(df.duplicated().sum())

# Remove duplicates if needed
df.drop_duplicates(inplace=True)

# 6. Statistical Summary
print("\nStatistical Summary")
print(df.describe())

print("\nStatistical Summary (Including Categorical)")
```

```
print(df.describe(include='all'))

# 7. Unique Values
print("\nUnique Values in Each Column")
print(df.nunique())

# 8. Correlation Matrix
numeric_df = df.select_dtypes(include=np.number)

plt.figure(figsize=(8,5))
sns.heatmap(numeric_df.corr(), annot=True, cmap="coolwarm")
plt.title("Correlation Matrix")
plt.show()

# 9. Distribution of Numerical Columns
numeric_df.hist(figsize=(10,6), bins=20)
plt.tight_layout()
plt.show()

# 10. Boxplots for Numerical Columns
for col in numeric_df.columns:
    plt.figure(figsize=(5,3))
    sns.boxplot(x=df[col])
    plt.title(f"Boxplot of {col}")
    plt.show()

# 11. Countplots for Categorical Columns
categorical_cols = df.select_dtypes(include="object").columns

for col in categorical_cols:
    plt.figure(figsize=(8,4))
    sns.countplot(y=df[col], order=df[col].value_counts().index)
    plt.title(f"Countplot of {col}")
    plt.tight_layout()
    plt.show()

# 12. Review Score Distribution
plt.figure(figsize=(6,4))
sns.countplot(x=df["review_score"])
plt.title("Review Score Distribution")
plt.show()

# 13. Payment Type Distribution
plt.figure(figsize=(6,4))
sns.countplot(x=df["payment_type"])
plt.title("Payment Type Distribution")
plt.xticks(rotation=45)
plt.show()

# 14. Customer State Distribution (Top 10)
plt.figure(figsize=(8,5))
top_states = df["customer_state"].value_counts().head(10)
sns.barplot(x=top_states.index, y=top_states.values)
plt.title("Top 10 Customer States")
plt.xlabel("State")
plt.ylabel("Count")
plt.show()

# 15. Sales Channel Distribution
```

```
# 15. Sales Channel Distribution
plt.figure(figsize=(6,4))
sns.countplot(x=df["sales_channel"])
plt.title("Sales Channel Distribution")
plt.xticks(rotation=45)
plt.show()

# 16. Price Distribution
plt.figure(figsize=(7,4))
sns.histplot(df["price"], bins=30, kde=True)
plt.title("Price Distribution")
plt.show()

# 17. Freight Value Distribution
plt.figure(figsize=(7,4))
sns.histplot(df["freight_value"], bins=30, kde=True)
plt.title("Freight Value Distribution")
plt.show()

# 18. Scatter Plot (Price vs Freight Value)
plt.figure(figsize=(6,4))
sns.scatterplot(x="price", y="freight_value", data=df)
plt.title("Price vs Freight Value")
plt.show()

# 19. Pairplot of Numerical Features
sns.pairplot(numeric_df)
plt.show()

# 20. Convert Timestamp and Extract Date Features
df["order_purchase_timestamp"] = pd.to_datetime(df["order_purchase_timestamp"])

df["Year"] = df["order_purchase_timestamp"].dt.year
df["Month"] = df["order_purchase_timestamp"].dt.month
df["Day"] = df["order_purchase_timestamp"].dt.day
df["Hour"] = df["order_purchase_timestamp"].dt.hour

# Monthly Orders
plt.figure(figsize=(8,4))
sns.countplot(x="Month", data=df)
plt.title("Monthly Orders")
plt.show()

# Hourly Orders
plt.figure(figsize=(8,4))
sns.countplot(x="Hour", data=df)
plt.title("Orders by Hour")
plt.show()

print("\nEDA Completed Successfully!")
```

```
First 5 Rows
         order_id     product_id order_purchase_timestamp   price  \
0  ORD-2025-10002  PROD_SPRT_002      2025-01-01 18:05:00  249.28
1  ORD-2025-10003   PROD_APP_003      2025-01-01 11:24:00   53.91
2  ORD-2025-10004  PROD_HOME_001      2025-01-01 13:44:00   83.86
3  ORD-2025-10005   PROD_APP_001      2025-01-01 20:47:00   59.47
4  ORD-2025-10006  PROD_SPRT_001      2025-01-01 17:47:00   34.73

   freight_value customer_state    payment_type        sales_channel  \
0           5.66             WA  digital_wallet           Mobile App
1          14.79             NY     credit_card   Amazon Marketplace
2           5.96             IL  digital_wallet  Direct Online Store
3           6.81             NC       gift_card   Amazon Marketplace
4           6.43             IL     credit_card           Mobile App

   review_score
0             5
1             5
2             3
3             5
4             4

Last 5 Rows
            order_id     product_id order_purchase_timestamp   price  \
4382  ORD-2025-14384   PROD_APP_003      2025-12-31 21:50:00   53.07
4383  ORD-2025-14385  PROD_BEAU_002      2025-12-31 14:25:00   54.43
4384  ORD-2025-14386  PROD_HOME_002      2025-12-31 14:02:00  162.67
4385  ORD-2025-14387  PROD_HOME_003      2025-12-31 14:32:00  118.90
4386  ORD-2025-14388   PROD_APP_002      2025-12-31 12:55:00   38.12

      freight_value customer_state    payment_type        sales_channel  \
4382           8.67             FL       gift_card   Amazon Marketplace
4383          10.06             CA       gift_card  Direct Online Store
4384           8.82             NC  digital_wallet  Direct Online Store
4385          14.49             NC  digital_wallet           Mobile App
4386           7.55             NY  digital_wallet  Direct Online Store

      review_score
4382             5
4383             5
4384             5
4385             3
4386             5

Shape of Dataset
(4387, 9)

Column Names
Index(['order_id', 'product_id', 'order_purchase_timestamp', 'price',
       'freight_value', 'customer_state', 'payment_type', 'sales_channel',
       'review_score'],
      dtype='object')

Dataset Information
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4387 entries, 0 to 4386
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   order_id                  4387 non-null   object
 1   product_id                4387 non-null   object
 2   order_purchase_timestamp  4387 non-null   object
 3   price                     4387 non-null   float64
```

```
 4   freight_value             4387 non-null   float64
 5   customer_state            4387 non-null   object
 6   payment_type              4387 non-null   object
 7   sales_channel             4387 non-null   object
 8   review_score              4387 non-null   int64
dtypes: float64(2), int64(1), object(6)
memory usage: 308.6+ KB
None

Data Types
order_id                     object
product_id                   object
order_purchase_timestamp     object
price                       float64
freight_value               float64
customer_state               object
payment_type                 object
sales_channel                object
review_score                  int64
dtype: object

Missing Values
order_id                    0
product_id                  0
order_purchase_timestamp    0
price                       0
freight_value               0
customer_state              0
payment_type                0
sales_channel               0
review_score                0
dtype: int64

Duplicate Rows
0

Statistical Summary
             price  freight_value  review_score
count  4387.000000    4387.000000   4387.000000
mean     95.510317       9.991842      4.339184
std      77.685836       2.900618      0.996722
min      20.010000       5.000000      1.000000
25%      38.920000       7.465000      4.000000
50%      59.550000       9.970000      5.000000
75%     126.745000      12.500000      5.000000
max     349.420000      15.000000      5.000000

Statistical Summary (Including Categorical)
              order_id    product_id order_purchase_timestamp        price  \
count             4387          4387                     4387  4387.000000
unique            4387            20                     4333          NaN
top     ORD-2025-14388  PROD_APP_003      2025-12-23 15:48:00          NaN
freq                 1           249                        2          NaN
mean               NaN           NaN                      NaN    95.510317
std                NaN           NaN                      NaN    77.685836
min                NaN           NaN                      NaN    20.010000
25%                NaN           NaN                      NaN    38.920000
50%                NaN           NaN                      NaN    59.550000
75%                NaN           NaN                      NaN   126.745000
max                NaN           NaN                      NaN   349.420000

        freight_value customer_state payment_type      sales_channel  \
count     4387.000000           4387         4387               4387
unique            NaN              8            4                  4
```

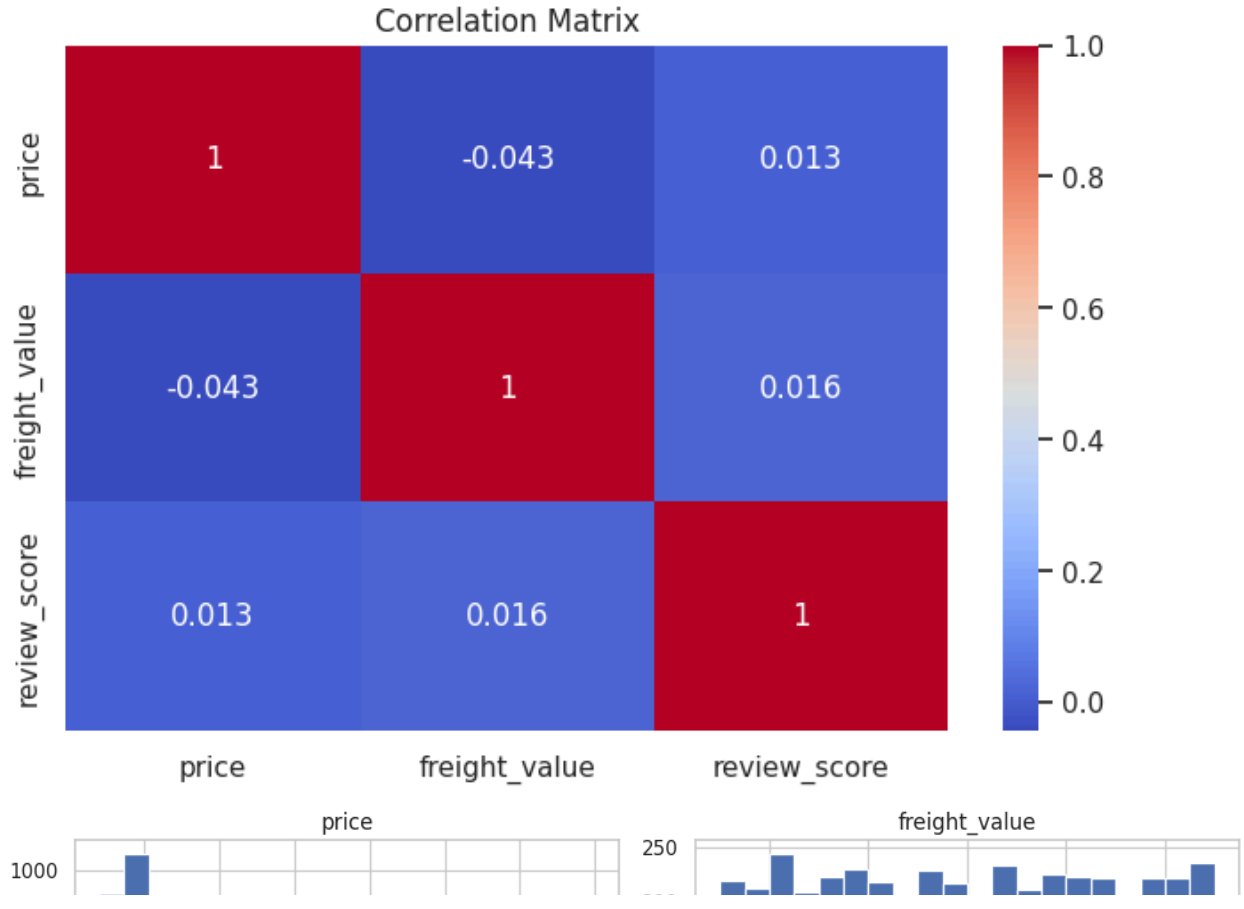
	price	customer_state	payment_type	sales_channel
top	NaN	CA	credit_card	Affiliate Channel
freq	NaN	571	1132	1151
mean	9.991842	NaN	NaN	NaN
std	2.900618	NaN	NaN	NaN
min	5.000000	NaN	NaN	NaN
25%	7.465000	NaN	NaN	NaN
50%	9.970000	NaN	NaN	NaN
75%	12.500000	NaN	NaN	NaN
max	15.000000	NaN	NaN	NaN

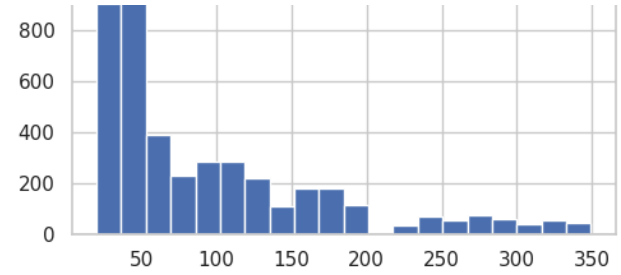
	review_score
count	4387.000000
unique	NaN
top	NaN
freq	NaN
mean	4.339184
std	0.996722
min	1.000000
25%	4.000000
50%	5.000000
75%	5.000000
max	5.000000

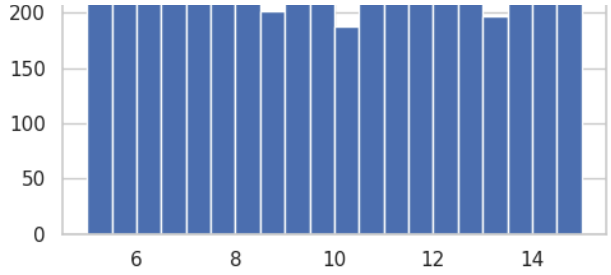
Unique Values in Each Column

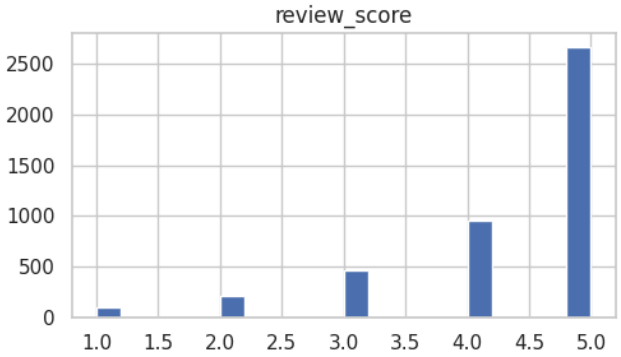
order_id	4387
product_id	20
order_purchase_timestamp	4333
price	3692
freight_value	990
customer_state	8
payment_type	4
sales_channel	4
review_score	5

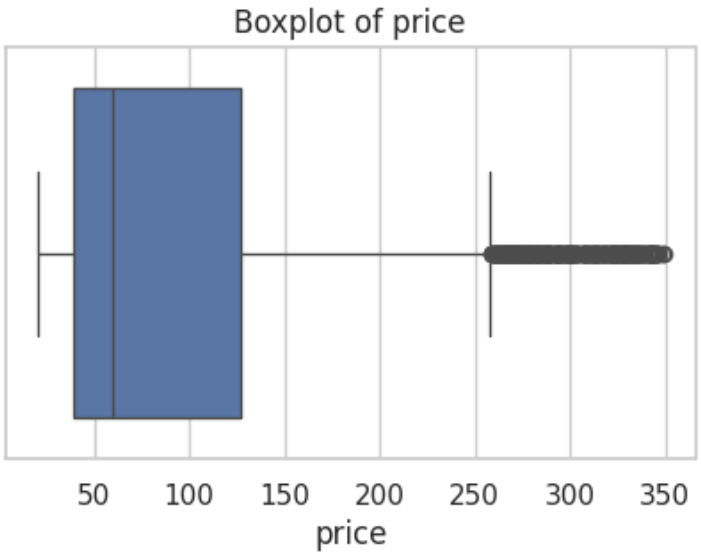
dtype: int64

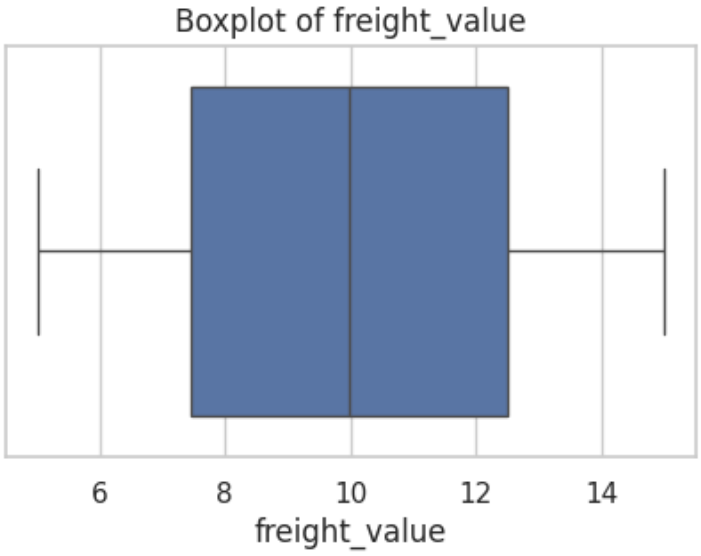



800
600
400
200
0
50
100
150
200
250
300
350

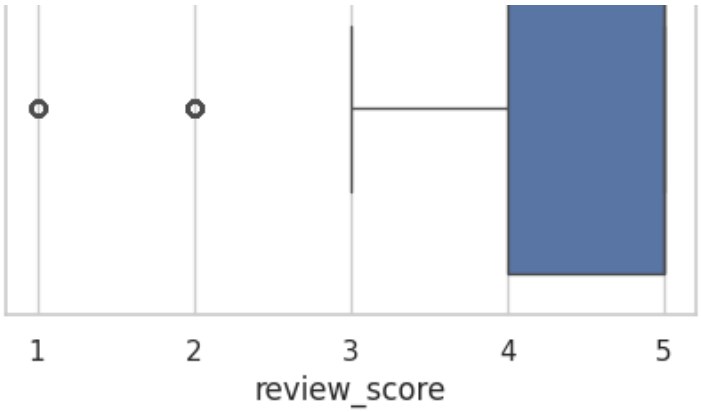

200
150
100
50
0
6
8
10
12
14

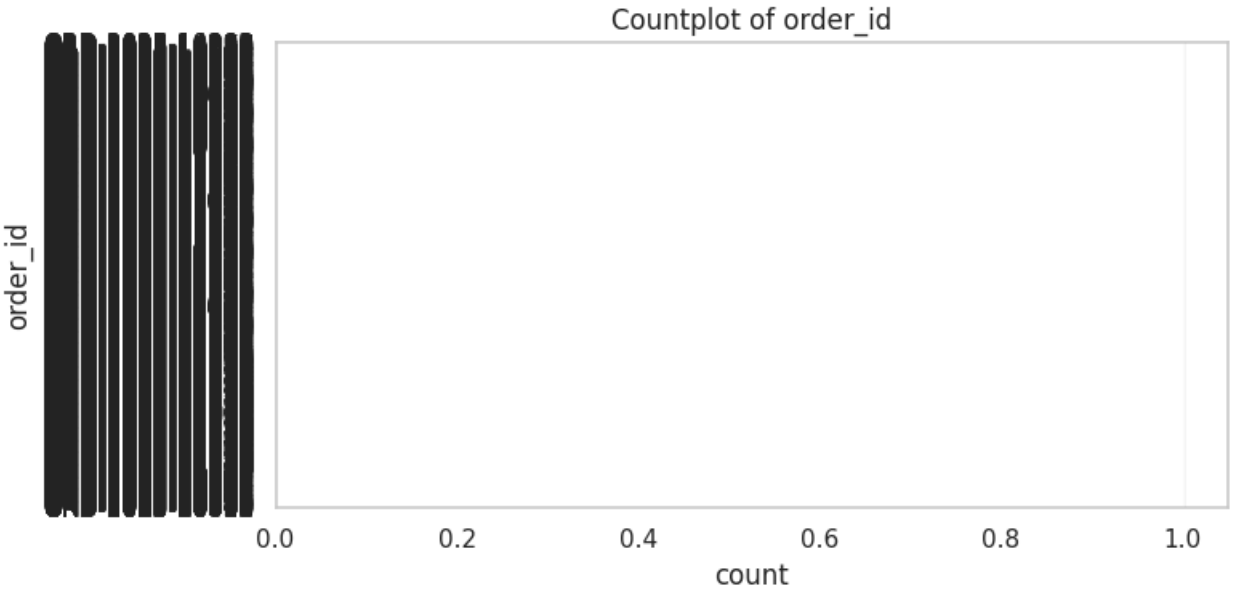

review_score
2500
2000
1500
1000
500
0
1.0
1.5
2.0
2.5
3.0
3.5
4.0
4.5
5.0

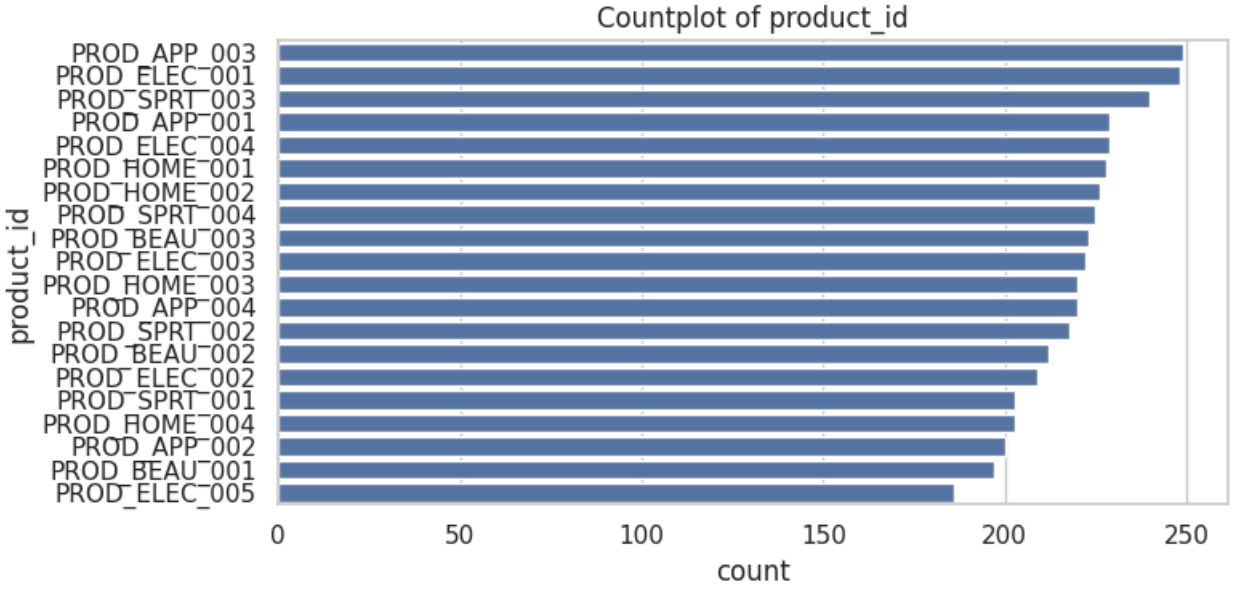

Boxplot of price
50
100
150
200
250
300
350
price

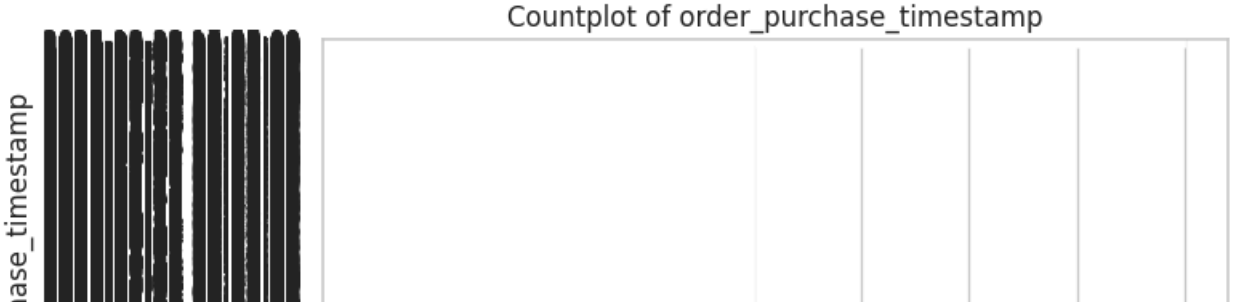

Boxplot of freight_value
6
8
10
12
14
freight_value

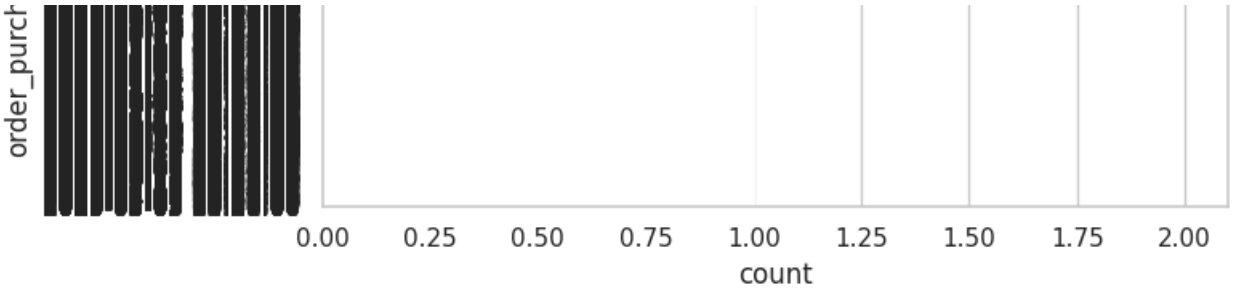

Boxplot of review_score

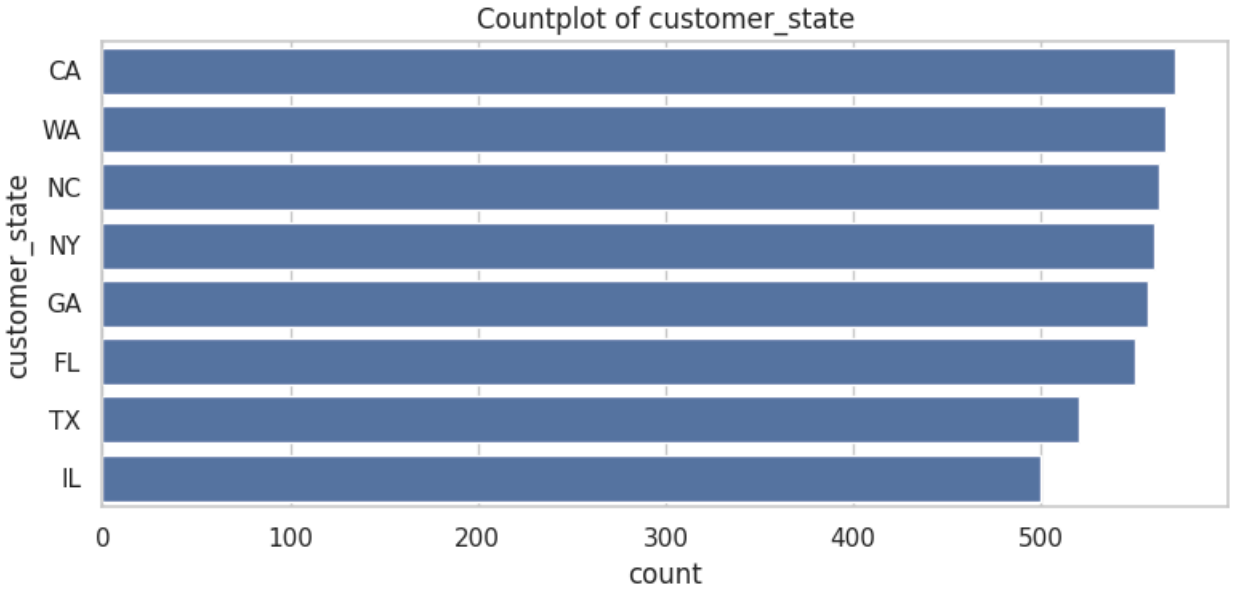

1
2
3
4
5
review_score

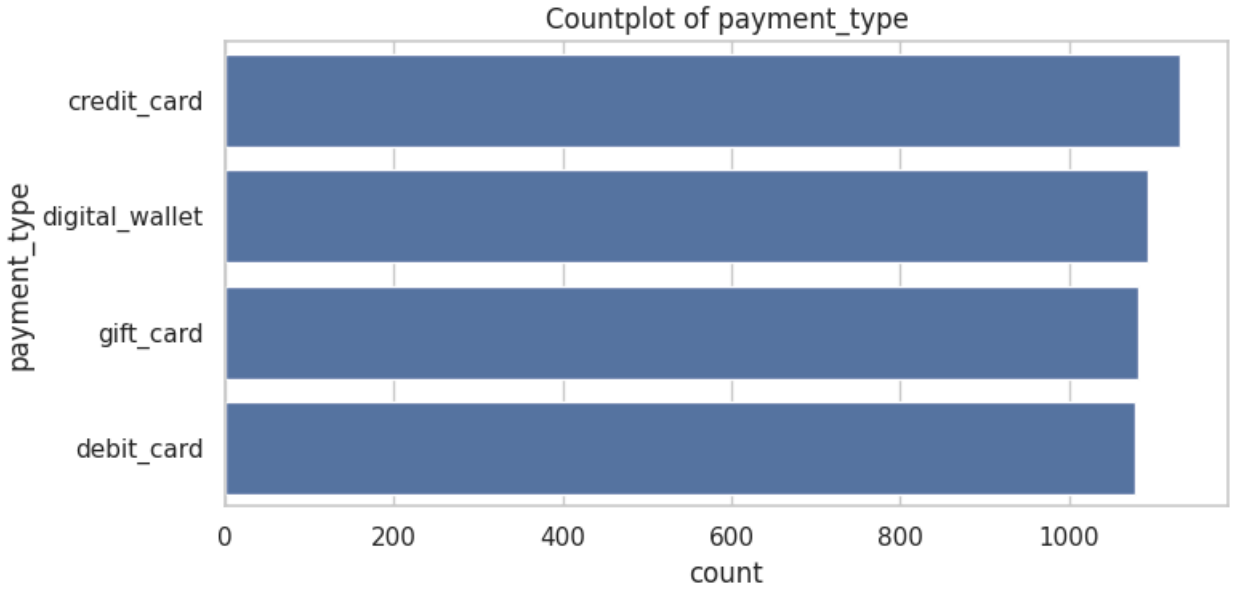

Countplot of order_id
order_id
0.0
0.2
0.4
0.6
0.8
1.0
count

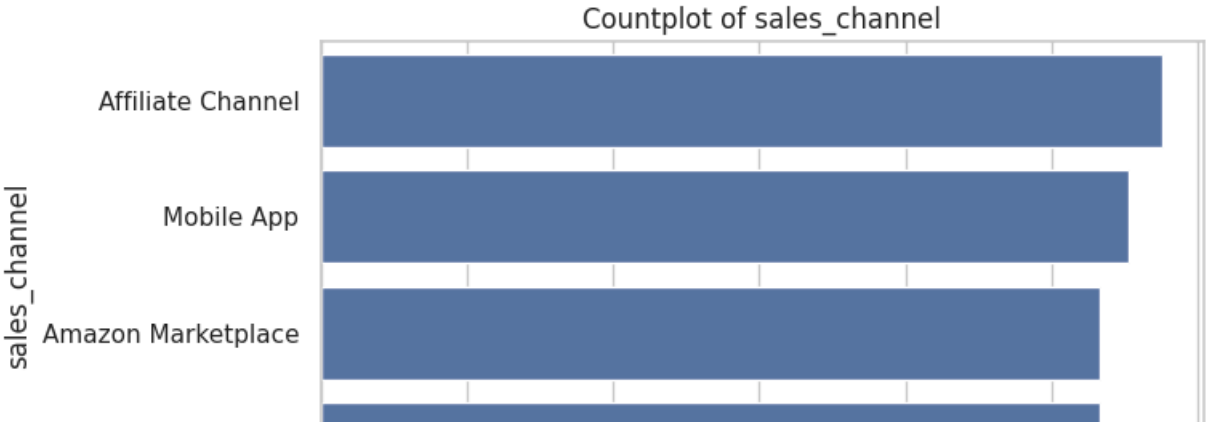

Countplot of product_id
product_id
PROD_APP_003
PROD_ELEC_001
PROD_SPRT_003
PROD_APP_001
PROD_ELEC_004
PROD_HOME_001
PROD_HOME_002
PROD_SPRT_004
PROD_BEAU_003
PROD_ELEC_003
PROD_HOME_003
PROD_APP_004
PROD_SPRT_002
PROD_BEAU_002
PROD_ELEC_002
PROD_SPRT_001
PROD_HOME_004
PROD_APP_002
PROD_BEAU_001
PROD_ELEC_005
0
50
100
150
200
250
count

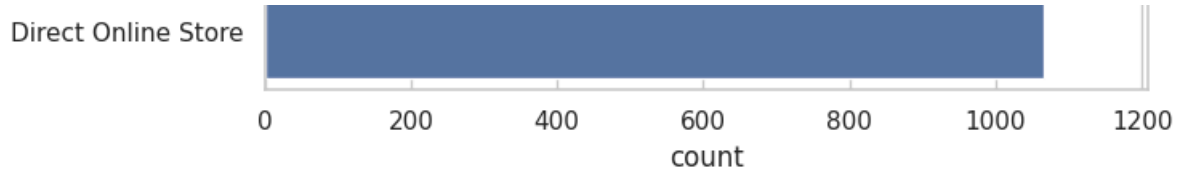

Countplot of order_purchase_timestamp

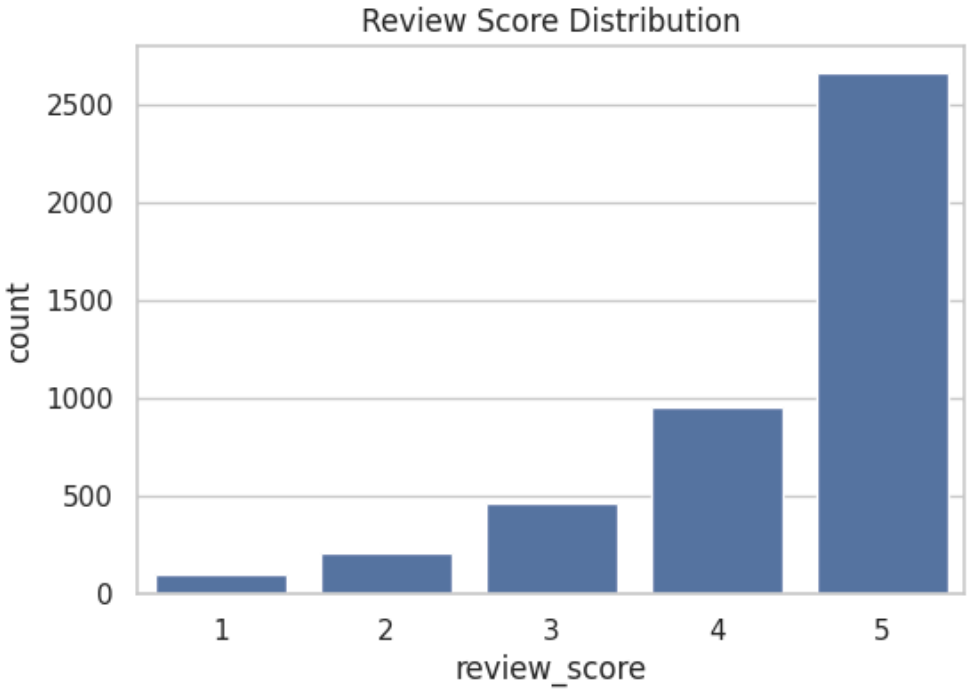

order_purch
0.00
0.25
0.50
0.75
1.00
1.25
1.50
1.75
2.00
count

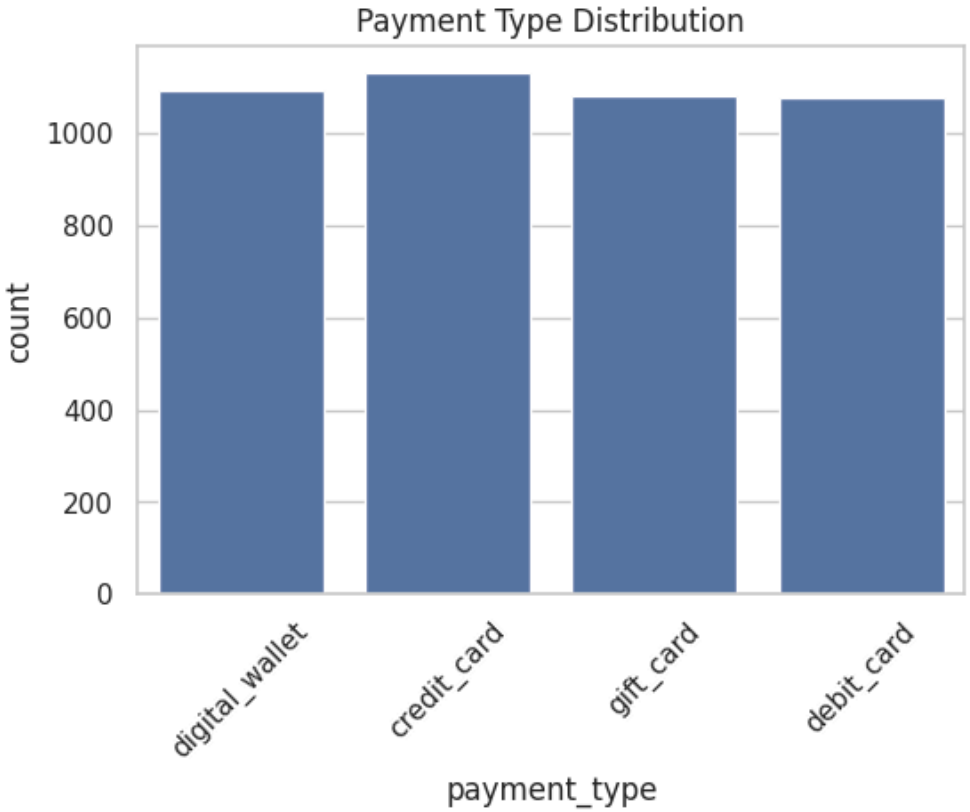

Countplot of customer_state
customer_state
CA
WA
NC
NY
GA
FL
TX
IL
0
100
200
300
400
500
count


Countplot of payment_type
payment_type
credit_card
digital_wallet
gift_card
debit_card
0
200
400
600
800
1000
count

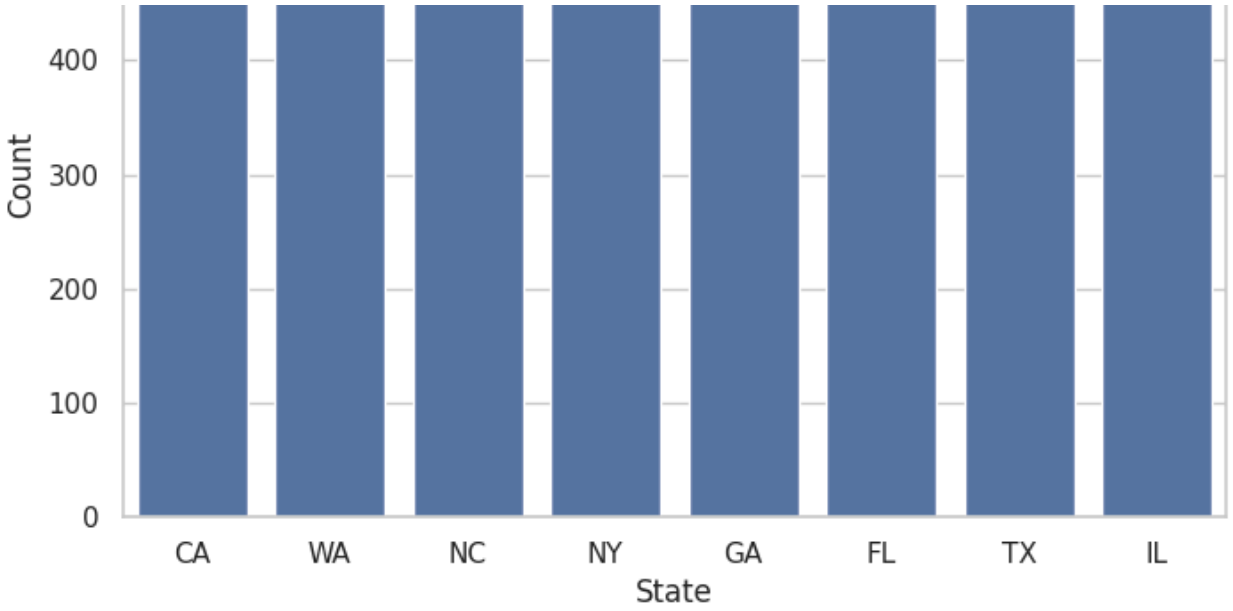

Countplot of sales_channel
sales_channel
Affiliate Channel
Mobile App
Amazon Marketplace

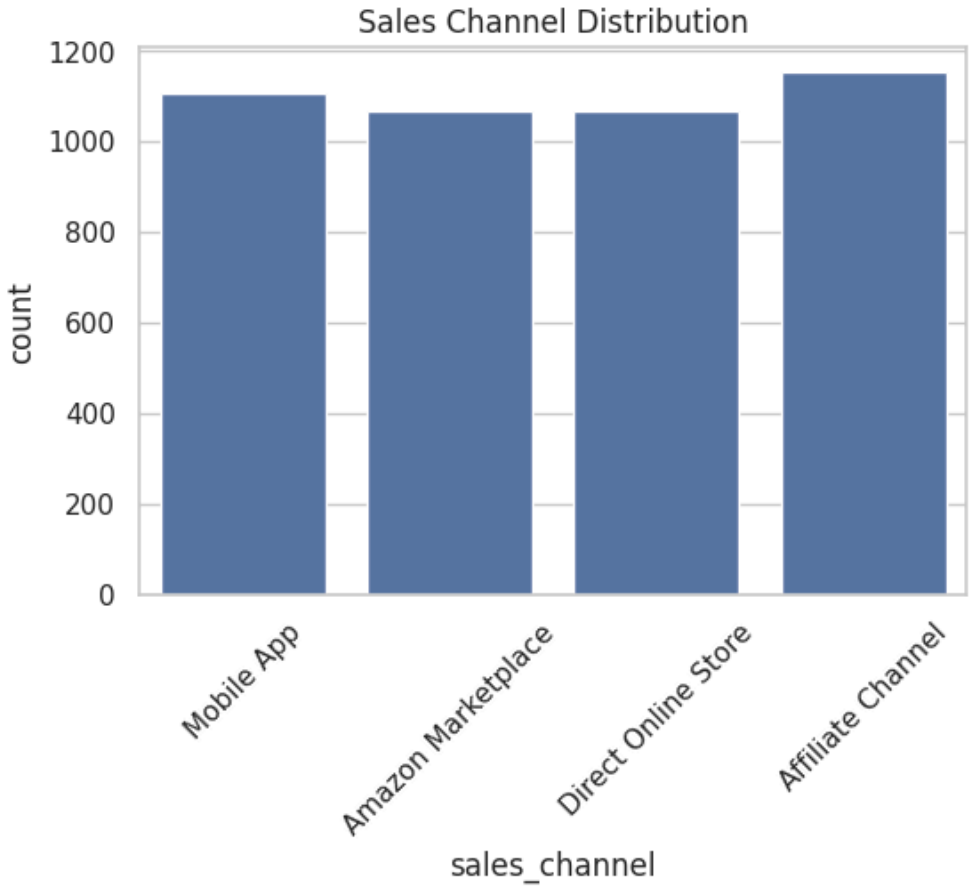

Direct Online Store
0
200
400
600
800
1000
1200
count


Review Score Distribution
count
2500
2000
1500
1000
500
0
1
2
3
4
5
review_score

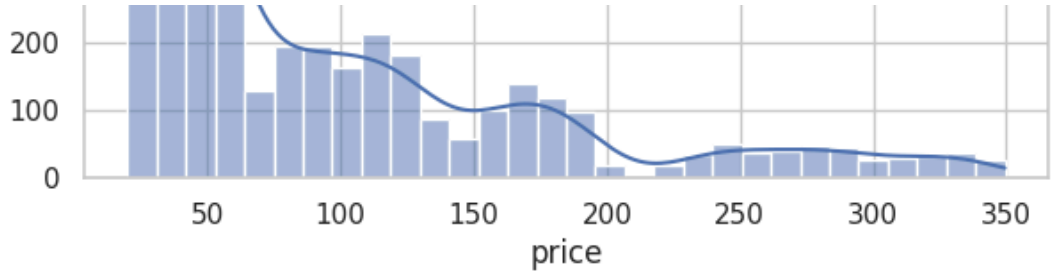

Payment Type Distribution
count
1000
800
600
400
200
0
digital_wallet
credit_card
gift_card
debit_card
payment_type

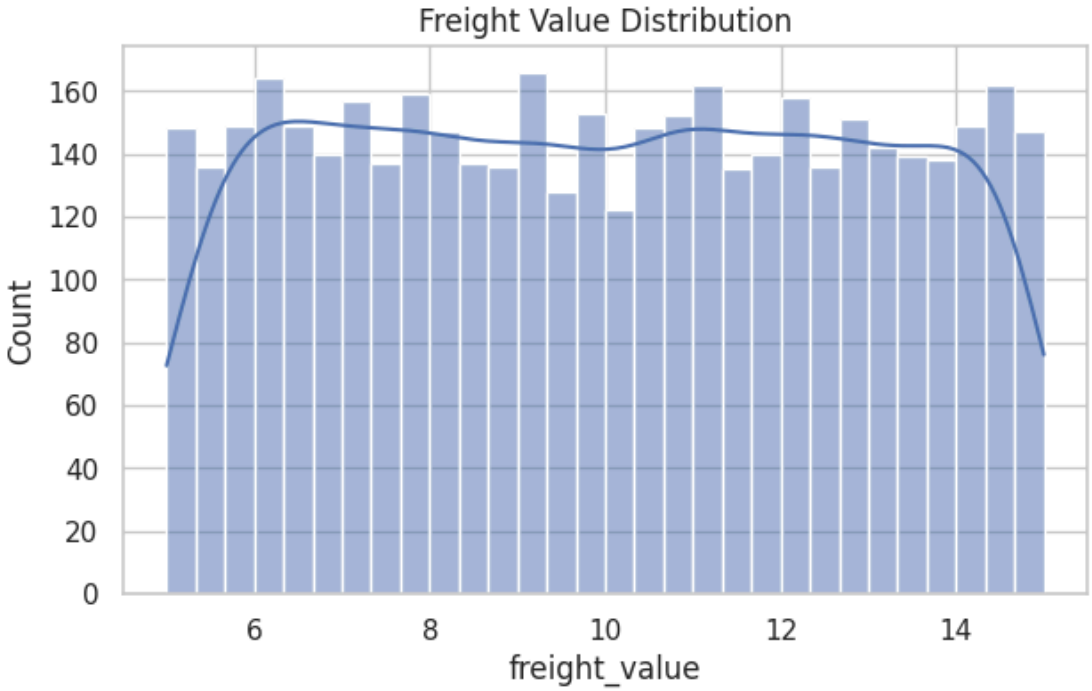

Top 10 Customer States
500


Count
400
300
200
100
0
CA
WA
NC
NY
GA
FL
TX
IL
State

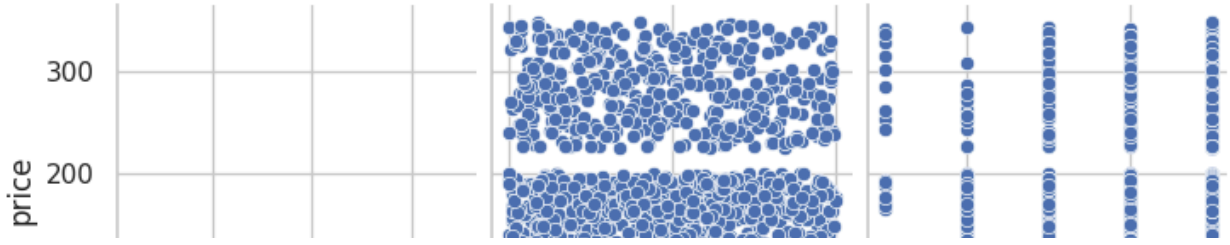

Sales Channel Distribution
count
1200
1000
800
600
400
200
0
Mobile App
Amazon Marketplace
Direct Online Store
Affiliate Channel
sales_channel

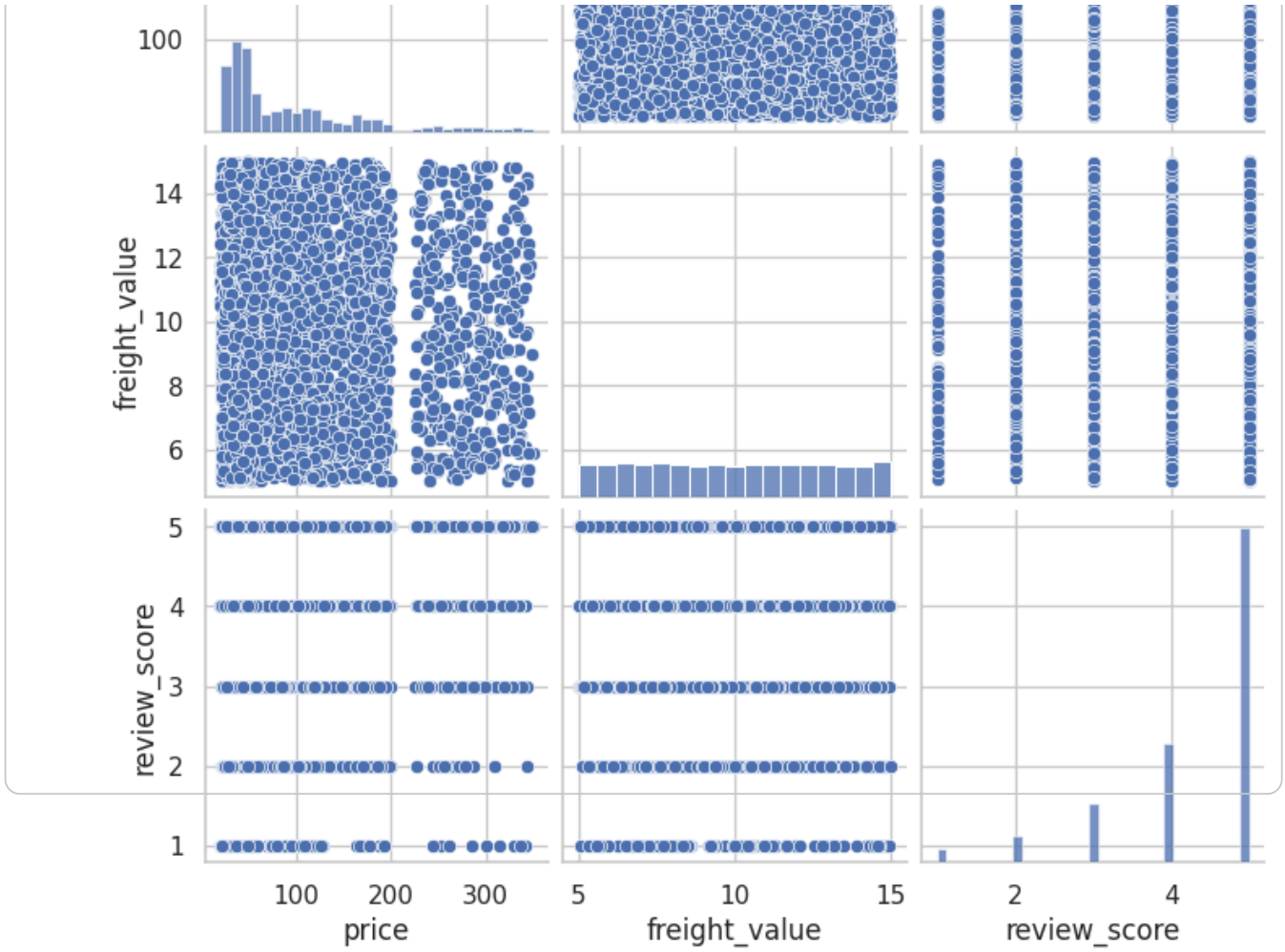

Price Distribution
Count
800
700
600
500
400
300


200
100
0
50
100
150
200
250
300
350
price


Freight Value Distribution
160
140
120
100
80
60
40
20
0
Count
6
8
10
12
14
freight_value


Price vs Freight Value
14
12
10
8
6
freight_value
50
100
150
200
250
300
350
price


300
200
price

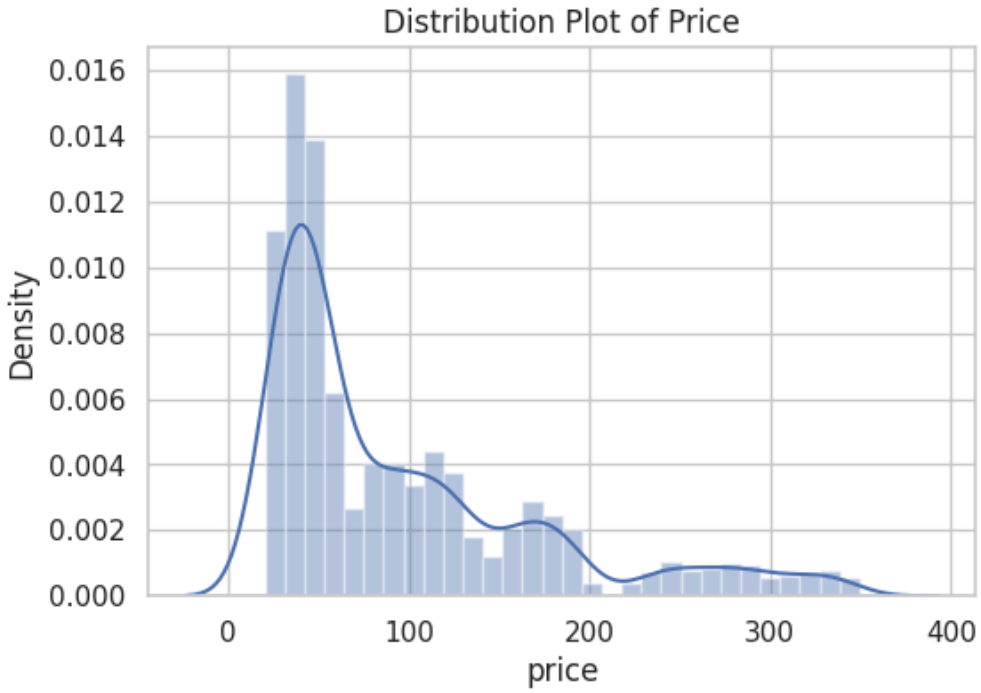

freight_value
review_score
price
freight_value
review_score


Monthly Orders
count
Month


Orders by Hour

```
import matplotlib.pyplot as plt
import seaborn as sns

# Distplot for Price
plt.figure(figsize=(6,4))
sns.distplot(df["price"], bins=30, kde=True)
plt.title("Distribution Plot of Price")
plt.show()

# Distplot for Freight Value
plt.figure(figsize=(6,4))
sns.distplot(df["freight_value"], bins=30, kde=True)
plt.title("Distribution Plot of Freight Value")
plt.show()

# Distplot for Review Score
plt.figure(figsize=(6,4))
sns.distplot(df["review_score"], bins=5, kde=True)
plt.title("Distribution Plot of Review Score")
plt.show()
```

```
/tmp/ipykernel_1876/1277950186.py:6: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with
similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see
https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751

  sns.distplot(df["price"], bins=30, kde=True)
```




```
/tmp/ipykernel_1876/1277950186.py:12: UserWarning:

`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with
```

```
from google.colab import files
uploaded = files.upload()
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
  sns.distplot(df["freight_value"], bins=30, kde=True)
Saving amazon_product_pricing.csv to amazon_product_pricing.csv
```

Distribution Plot of Freight Value

```
# ============================================================
# EDA - Amazon Product Pricing Dataset
# ============================================================

# 1. Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Set visualization style
sns.set_theme(style="whitegrid")

# ------------------------------------------------------------
```

```
# ============================================================
# 2. Load Dataset
# ============================================================

import pandas as pd

filename = next(iter(uploaded))
df = pd.read_csv(filename)

print("Dataset loaded successfully!")
print("Shape:", df.shape)
display(df.head())
# ============================================================
# 3. Basic Information
# ============================================================

print("=" * 60)
print("FIRST 5 ROWS")
print("=" * 60)
print(df.head())

print("\n" + "=" * 60)
print("LAST 5 ROWS")
print("=" * 60)
print(df.tail())

print("\n" + "=" * 60)
print("SHAPE OF DATASET")
print("=" * 60)
print("Rows:", df.shape[0])
print("Columns:", df.shape[1])

print("\n" + "=" * 60)
print("COLUMN NAMES")
print("=" * 60)
print(df.columns.tolist())

print("\n" + "=" * 60)
print("DATASET INFORMATION")
print("=" * 60)
df.info()

print("\n" + "=" * 60)
print("DATA TYPES")
print("=" * 60)
print(df.dtypes)

# ============================================================
# 4. Missing Values
# ============================================================

print("\n" + "=" * 60)
print("MISSING VALUES")
print("=" * 60)

missing_values = df.isnull().sum()
print(missing_values)

print("\nTotal Missing Values:", df.isnull().sum().sum())
```

```
print("\nTotal Missing Values:", df.isnull().sum().sum())

# Missing-value percentage
missing_percentage = (df.isnull().sum() / len(df)) * 100
print("\nMissing Value Percentage:")
print(missing_percentage)

# ============================================================
# 5. Duplicate Values
# ============================================================

print("\n" + "=" * 60)
print("DUPLICATE ROWS")
print("=" * 60)

duplicates = df.duplicated().sum()
print("Number of duplicate rows:", duplicates)

# Remove duplicates
if duplicates > 0:
    df.drop_duplicates(inplace=True)
    print("Duplicates removed.")

# ============================================================
# 6. Statistical Summary
# ============================================================

print("\n" + "=" * 60)
print("STATISTICAL SUMMARY")
print("=" * 60)

print(df.describe())

print("\n" + "=" * 60)
print("STATISTICAL SUMMARY INCLUDING CATEGORICAL DATA")
print("=" * 60)

print(df.describe(include="all"))

# ============================================================
# 7. Unique Values
# ============================================================

print("\n" + "=" * 60)
print("UNIQUE VALUES")
print("=" * 60)

print(df.nunique())

# ============================================================
# 8. Numerical and Categorical Columns
# ============================================================

numeric_df = df.select_dtypes(include=np.number)
categorical_cols = df.select_dtypes(include="object").columns

print("\nNumerical Columns:")
print(numeric_df.columns.tolist())
```

```
print("\nCategorical Columns:")
print(categorical_cols.tolist())

# ============================================================
# 9. Correlation Matrix
# ============================================================

plt.figure(figsize=(10, 7))

correlation = numeric_df.corr()

sns.heatmap(
    correlation,
    annot=True,
    cmap="coolwarm",
    fmt=".2f",
    linewidths=0.5
)

plt.title("Correlation Matrix - Amazon Product Pricing")
plt.tight_layout()
plt.show()

# ============================================================
# 10. Distribution of Numerical Variables
# ============================================================

numeric_df.hist(
    figsize=(14, 10),
    bins=15,
    edgecolor="black"
)

plt.suptitle(
    "Distribution of Numerical Variables",
    fontsize=16
)

plt.tight_layout()
plt.show()

# ============================================================
# 11. Boxplots for Numerical Columns
# ============================================================

for col in numeric_df.columns:

    plt.figure(figsize=(7, 4))

    sns.boxplot(x=df[col])

    plt.title(f"Boxplot of {col}")
    plt.xlabel(col)

    plt.tight_layout()
    plt.show()

# ============================================================
```

```
# 12. Categorical Variable Analysis
# ============================================================

for col in categorical_cols:

    plt.figure(figsize=(8, 5))

    order = df[col].value_counts().index

    sns.countplot(
        y=df[col],
        order=order
    )

    plt.title(f"Distribution of {col}")
    plt.xlabel("Count")
    plt.ylabel(col)

    plt.tight_layout()
    plt.show()

# ============================================================
# 13. Category Distribution
# ============================================================

plt.figure(figsize=(7, 5))

category_counts = df["category"].value_counts()

sns.barplot(
    x=category_counts.index,
    y=category_counts.values
)

plt.title("Product Category Distribution")
plt.xlabel("Category")
plt.ylabel("Number of Products")

plt.xticks(rotation=30)

plt.tight_layout()
plt.show()

# ============================================================
# 14. Sub-Category Distribution
# ============================================================

plt.figure(figsize=(9, 5))

subcategory_counts = df["sub_category"].value_counts()

sns.barplot(
    x=subcategory_counts.index,
    y=subcategory_counts.values
)

plt.title("Product Sub-Category Distribution")
plt.xlabel("Sub-Category")
```

```
plt.ylabel("Number of Products")

plt.xticks(rotation=45)

plt.tight_layout()
plt.show()

# ============================================================
# 15. Actual Price vs Discounted Price
# ============================================================

plt.figure(figsize=(8, 5))

sns.scatterplot(
    data=df,
    x="actual_price_msrp",
    y="discounted_price",
    s=100
)

plt.title("Actual Price vs Discounted Price")
plt.xlabel("Actual Price / MSRP")
plt.ylabel("Discounted Price")

plt.tight_layout()
plt.show()

# ============================================================
# 16. Cost Price vs Discounted Price
# ============================================================

plt.figure(figsize=(8, 5))

sns.scatterplot(
    data=df,
    x="cost_price",
    y="discounted_price",
    s=100
)

plt.title("Cost Price vs Discounted Price")
plt.xlabel("Cost Price")
plt.ylabel("Discounted Price")

plt.tight_layout()
plt.show()

# ============================================================
# 17. Price Distribution
# ============================================================

plt.figure(figsize=(8, 5))

sns.histplot(
    df["actual_price_msrp"],
    bins=15,
    kde=True
)
```

```
plt.title("Distribution of Actual Price / MSRP")
plt.xlabel("Actual Price")
plt.ylabel("Frequency")

plt.tight_layout()
plt.show()

# ============================================================
# 18. Discounted Price Distribution
# ============================================================

plt.figure(figsize=(8, 5))

sns.histplot(
    df["discounted_price"],
    bins=15,
    kde=True
)

plt.title("Distribution of Discounted Price")
plt.xlabel("Discounted Price")
plt.ylabel("Frequency")

plt.tight_layout()
plt.show()

# ============================================================
# 19. Discount Percentage Distribution
# ============================================================

plt.figure(figsize=(8, 5))

sns.histplot(
    df["discount_percentage"],
    bins=10,
    kde=True
)

plt.title("Distribution of Discount Percentage")
plt.xlabel("Discount Percentage")
plt.ylabel("Frequency")

plt.tight_layout()
plt.show()

# ============================================================
# 20. Rating Distribution
# ============================================================

plt.figure(figsize=(7, 5))

sns.histplot(
    df["rating"],
    bins=10,
    kde=True
)
```

```
plt.title("Product Rating Distribution")
plt.xlabel("Rating")
plt.ylabel("Frequency")

plt.tight_layout()
plt.show()

# ============================================================
# 21. Rating Count Distribution
# ============================================================

plt.figure(figsize=(8, 5))

sns.histplot(
    df["rating_count"],
    bins=15,
    kde=True
)

plt.title("Distribution of Rating Count")
plt.xlabel("Rating Count")
plt.ylabel("Frequency")

plt.tight_layout()
plt.show()

# ============================================================
# 22. Stock Availability
# ============================================================

plt.figure(figsize=(8, 5))

sns.histplot(
    df["stock_available"],
    bins=15,
    kde=True
)

plt.title("Distribution of Stock Availability")
plt.xlabel("Stock Available")
plt.ylabel("Frequency")

plt.tight_layout()
plt.show()

# ============================================================
# 23. Discount Percentage by Category
# ============================================================

plt.figure(figsize=(8, 5))

sns.boxplot(
    data=df,
    x="category",
    y="discount_percentage"
)

plt.title("Discount Percentage by Category")
```

```
plt.xlabel("Category")
plt.ylabel("Discount Percentage")

plt.xticks(rotation=30)

plt.tight_layout()
plt.show()

# ============================================================
# 24. Price by Category
# ============================================================

plt.figure(figsize=(8, 5))

sns.boxplot(
    data=df,
    x="category",
    y="discounted_price"
)

plt.title("Discounted Price by Category")
plt.xlabel("Category")
plt.ylabel("Discounted Price")

plt.xticks(rotation=30)

plt.tight_layout()
plt.show()

# ============================================================
# 25. Rating vs Rating Count
# ============================================================

plt.figure(figsize=(8, 5))

sns.scatterplot(
    data=df,
    x="rating",
    y="rating_count",
    s=100
)

plt.title("Rating vs Rating Count")
plt.xlabel("Rating")
plt.ylabel("Rating Count")

plt.tight_layout()
plt.show()

# ============================================================
# 26. Discount vs Rating
# ============================================================

plt.figure(figsize=(8, 5))

sns.scatterplot(
    data=df,
    x="discount_percentage",
```

```
    y="rating",
    s=100
)

plt.title("Discount Percentage vs Rating")
plt.xlabel("Discount Percentage")
plt.ylabel("Rating")

plt.tight_layout()
plt.show()

# ============================================================
# 27. Stock vs Discounted Price
# ============================================================

plt.figure(figsize=(8, 5))

sns.scatterplot(
    data=df,
    x="discounted_price",
    y="stock_available",
    s=100
)

plt.title("Discounted Price vs Stock Availability")
plt.xlabel("Discounted Price")
plt.ylabel("Stock Available")

plt.tight_layout()
plt.show()

# ============================================================
# 28. Pairplot
# ============================================================

sns.pairplot(
    df[
        [
            "actual_price_msrp",
            "discounted_price",
            "discount_percentage",
            "cost_price",
            "rating",
            "rating_count",
            "stock_available"
        ]
    ]
)

plt.show()

# ============================================================
# 29. Calculate Profit Margin
# ============================================================

df["profit"] = df["discounted_price"] - df["cost_price"]

df["profit_margin_percentage"] = (
    df["profit"] / df["discounted_price"]
```

```
    df["profit"] / df["discounted_price"]
) * 100

print("\n" + "=" * 60)
print("PROFIT ANALYSIS")
print("=" * 60)

print(df[
    [
        "product_name",
        "discounted_price",
        "cost_price",
        "profit",
        "profit_margin_percentage"
    ]
])

# ============================================================
# 30. Top Products by Profit
# ============================================================

top_profit = df.sort_values(
    "profit",
    ascending=False
).head(10)

plt.figure(figsize=(10, 6))

sns.barplot(
    data=top_profit,
    x="profit",
    y="product_name"
)

plt.title("Top Products by Profit")
plt.xlabel("Profit")
plt.ylabel("Product")

plt.tight_layout()
plt.show()

# ============================================================
# 31. Top Products by Discount
# ============================================================

top_discount = df.sort_values(
    "discount_percentage",
    ascending=False
).head(10)

plt.figure(figsize=(10, 6))

sns.barplot(
    data=top_discount,
    x="discount_percentage",
    y="product_name"
)

plt.title("Top Products by Discount Percentage")
```

```
plt.title("Top Products by Discount Percentage")
plt.xlabel("Discount Percentage")
plt.ylabel("Product")

plt.tight_layout()
plt.show()

# ============================================================
# 32. Top Products by Rating
# ============================================================

top_rating = df.sort_values(
    "rating",
    ascending=False
).head(10)

plt.figure(figsize=(10, 6))

sns.barplot(
    data=top_rating,
    x="rating",
    y="product_name"
)

plt.title("Top Products by Rating")
plt.xlabel("Rating")
plt.ylabel("Product")

plt.tight_layout()
plt.show()

# ============================================================
# 33. EDA Summary
# ============================================================

print("\n" + "=" * 60)
print("EDA SUMMARY")
print("=" * 60)

print("Dataset Shape:", df.shape)
print("Total Products:", len(df))
print("Total Categories:", df["category"].nunique())
print("Total Sub-Categories:", df["sub_category"].nunique())

print(
    "Average Actual Price:",
    round(df["actual_price_msrp"].mean(), 2)
)

print(
    "Average Discounted Price:",
    round(df["discounted_price"].mean(), 2)
)

print(
    "Average Discount:",
    round(df["discount_percentage"].mean(), 2), "%"
)
```

```
print(
    "Average Rating:",
    round(df["rating"].mean(), 2)
)

print(
    "Average Stock:",
    round(df["stock_available"].mean(), 2)
)

print(
    "Average Profit:",
    round(df["profit"].mean(), 2)
)

print("\nEDA Completed Successfully!")
```

```
Dataset loaded successfully!
Shape: (20, 11)
```

	product_id	product_name	category	sub_category	actual_price_msrp	discounted_pri
0	PROD_ELEC_001	Aura Pro Wireless Noise-Cancelling Headphones	Electronics	Audio	199.99	157.
1	PROD_ELEC_002	UltraView 27-inch 4K UHD Gaming Monitor	Electronics	Monitors	349.99	330.
2	PROD_ELEC_003	Vortex Pulse Smartwatch with Heart Rate & GPS	Electronics	Wearables	149.99	132.
3	PROD_ELEC_004	HyperCharge 65W GaN Fast Charger Multi-Port	Electronics	Accessories	39.99	35.
4	PROD_ELEC_005	NovaCast 4K Streaming Media Player HDR	Electronics	Home Entertainment	49.99	38.

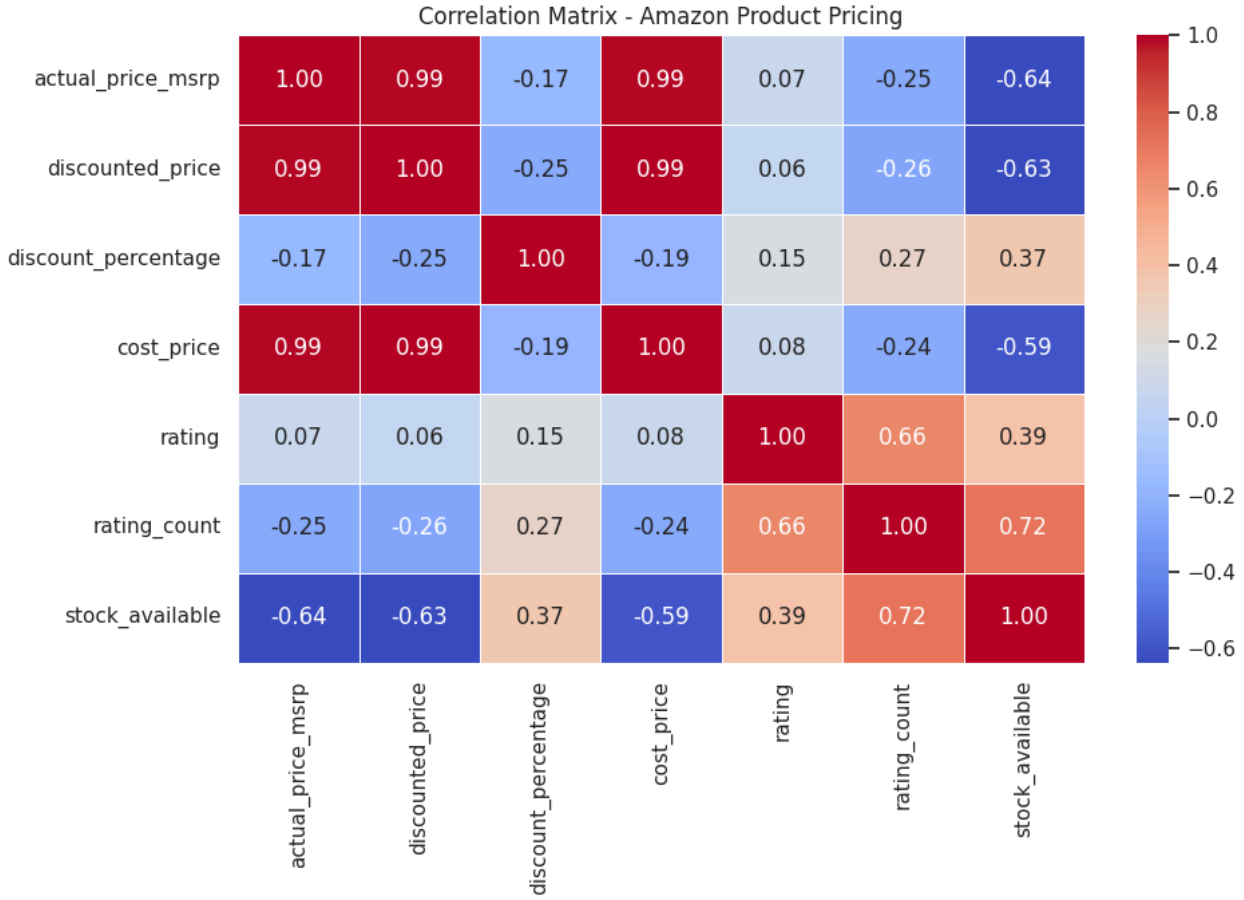
```
============================================================
FIRST 5 ROWS
============================================================
      product_id                                       product_name     category  \
0  PROD_ELEC_001  Aura Pro Wireless Noise-Cancelling Headphones  Electronics
1  PROD_ELEC_002        UltraView 27-inch 4K UHD Gaming Monitor  Electronics
2  PROD_ELEC_003  Vortex Pulse Smartwatch with Heart Rate & GPS  Electronics
3  PROD_ELEC_004    HyperCharge 65W GaN Fast Charger Multi-Port  Electronics
4  PROD_ELEC_005         NovaCast 4K Streaming Media Player HDR  Electronics

         sub_category  actual_price_msrp  discounted_price  \
0               Audio             199.99            157.99
1            Monitors             349.99            330.39
2           Wearables             149.99            132.14
3         Accessories              39.99             35.75
4  Home Entertainment              49.99             38.29

   discount_percentage  cost_price  rating  rating_count  stock_available
0                 21.0       110.0     4.6          1240              450
1                  5.6       210.0     4.7           890              280
2                 11.9        75.0     4.4          2150              600
3                 10.6        14.0     4.8          4320             1200
4                 23.4        22.0     4.5          3100              850

============================================================
LAST 5 ROWS
============================================================
       product_id                                        product_name  \
15  PROD_BEAU_003      VitalGlow Deep Moisturizing Hydration Complex
16  PROD_SPRT_001    TrailBlazer Lightweight Aluminum Trekking Poles
17  PROD_SPRT_002      FlexCore Adjustable Dumbbell Set (5-52.5 lbs)
18  PROD_SPRT_003  HydroShield 32oz Insulated Stainless Steel Bottle
19  PROD_SPRT_004           AeroGlide High-Speed Folding Kick Scooter
```

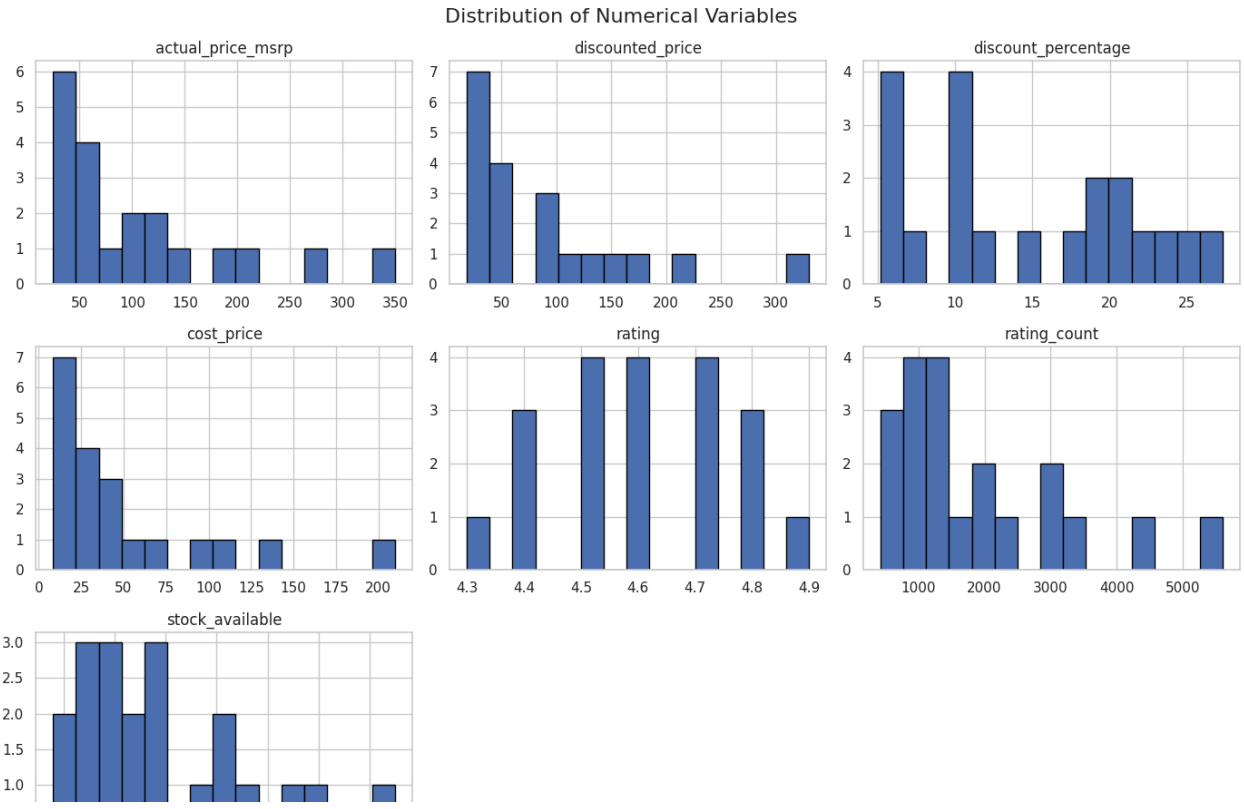
```
             category   sub_category  actual_price_msrp  discounted_price  \
15    Health & Beauty       Skincare              34.99             28.48
16  Sports & Outdoors   Outdoor Gear              39.99             35.79
17  Sports & Outdoors        Fitness             279.99            224.83
18  Sports & Outdoors      Hydration              24.99             18.69
19  Sports & Outdoors  Action Sports              89.99             85.31

    discount_percentage  cost_price  rating  rating_count  stock_available
15                 18.6        11.0     4.5          1210              870
16                 10.5        15.0     4.7           780              420
17                 19.7       140.0     4.8          2340              160
18                 25.2         9.0     4.9          5600             1500
19                  5.2        42.0     4.4           430              290

============================================================
SHAPE OF DATASET
============================================================
Rows: 20
Columns: 11

============================================================
COLUMN NAMES
============================================================
['product_id', 'product_name', 'category', 'sub_category', 'actual_price_msrp', 'discounted_

============================================================
DATASET INFORMATION
============================================================
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20 entries, 0 to 19
Data columns (total 11 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   product_id           20 non-null     object
 1   product_name         20 non-null     object
 2   category             20 non-null     object
 3   sub_category         20 non-null     object
 4   actual_price_msrp    20 non-null     float64
 5   discounted_price     20 non-null     float64
 6   discount_percentage  20 non-null     float64
 7   cost_price           20 non-null     float64
 8   rating               20 non-null     float64
 9   rating_count         20 non-null     int64
 10  stock_available      20 non-null     int64
dtypes: float64(5), int64(2), object(4)
memory usage: 1.8+ KB

============================================================
DATA TYPES
============================================================
product_id              object
product_name            object
category                object
sub_category            object
actual_price_msrp      float64
discounted_price       float64
discount_percentage    float64
cost_price             float64
rating                 float64
rating_count             int64
stock_available          int64
dtype: object

============================================================
```

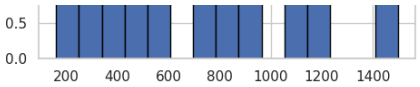
```
MISSING VALUES
============================================================
product_id             0
product_name           0
category               0
sub_category           0
actual_price_msrp      0
discounted_price       0
discount_percentage    0
cost_price             0
rating                 0
rating_count           0
stock_available        0
dtype: int64

Total Missing Values: 0

Missing Value Percentage:
product_id             0.0
product_name           0.0
category               0.0
sub_category           0.0
actual_price_msrp      0.0
discounted_price       0.0
discount_percentage    0.0
cost_price             0.0
rating                 0.0
rating_count           0.0
stock_available        0.0
dtype: float64

============================================================
DUPLICATE ROWS
============================================================
Number of duplicate rows: 0

============================================================
STATISTICAL SUMMARY
============================================================
       actual_price_msrp  discounted_price  discount_percentage  cost_price  \
count          20.000000         20.000000            20.000000     20.0000
mean          105.490000         91.014500            14.715000     50.3750
std            89.087479         79.578479             7.258626     52.5249
min            24.990000         18.690000             5.200000      8.5000
25%            43.740000         35.780000             9.300000     15.7500
50%            64.990000         52.845000            13.700000     26.0000
75%           134.990000        113.367500            21.050000     60.0000
max           349.990000        330.390000            27.300000    210.0000

          rating  rating_count  stock_available
count  20.000000     20.000000        20.000000
mean    4.600000   1871.500000       614.000000
std     0.162221   1381.179035       360.268905
min     4.300000    430.000000       160.000000
25%     4.500000    890.000000       370.000000
50%     4.600000   1335.000000       500.000000
75%     4.700000   2477.500000       855.000000
max     4.900000   5600.000000      1500.000000

============================================================
STATISTICAL SUMMARY INCLUDING CATEGORICAL DATA
============================================================
          product_id                                product_name  \
count             20                                          20
```

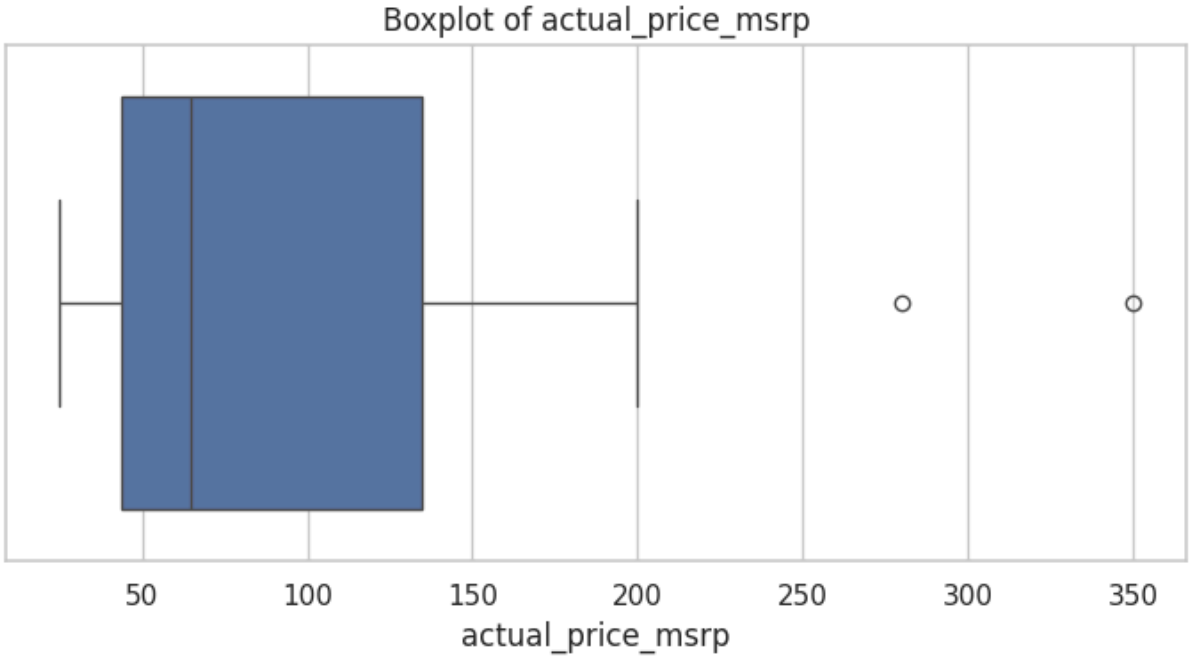
```
count              20                                                  20
unique             20                                                  20
top     PROD_ELEC_001  Aura Pro Wireless Noise-Cancelling Headphones
freq                1                                                   1
mean              NaN                                                 NaN
std               NaN                                                 NaN
min               NaN                                                 NaN
25%               NaN                                                 NaN
50%               NaN                                                 NaN
75%               NaN                                                 NaN
max               NaN                                                 NaN

           category sub_category  actual_price_msrp  discounted_price  \
count            20           20          20.000000         20.000000
unique            5           18                 NaN               NaN
top     Electronics     Skincare                NaN               NaN
freq              5            2                NaN               NaN
mean            NaN          NaN         105.490000         91.014500
std             NaN          NaN          89.087479         79.578479
min             NaN          NaN          24.990000         18.690000
25%             NaN          NaN          43.740000         35.780000
50%             NaN          NaN          64.990000         52.845000
75%             NaN          NaN         134.990000        113.367500
max             NaN          NaN         349.990000        330.390000

        discount_percentage  cost_price     rating  rating_count  \
count              20.000000     20.0000  20.000000     20.000000
unique                   NaN         NaN        NaN           NaN
top                      NaN         NaN        NaN           NaN
freq                     NaN         NaN        NaN           NaN
mean               14.715000     50.3750   4.600000   1871.500000
std                 7.258626     52.5249   0.162221   1381.179035
min                 5.200000      8.5000   4.300000    430.000000
25%                 9.300000     15.7500   4.500000    890.000000
50%                13.700000     26.0000   4.600000   1335.000000
75%                21.050000     60.0000   4.700000   2477.500000
max                27.300000    210.0000   4.900000   5600.000000

        stock_available
count         20.000000
unique              NaN
top                 NaN
freq                NaN
mean         614.000000
std          360.268905
min          160.000000
25%          370.000000
50%          500.000000
75%          855.000000
max         1500.000000

============================================================
UNIQUE VALUES
============================================================
product_id            20
product_name          20
category               5
sub_category          18
actual_price_msrp     18
discounted_price      20
discount_percentage   18
cost_price            19
rating                 7
rating_count          19
```

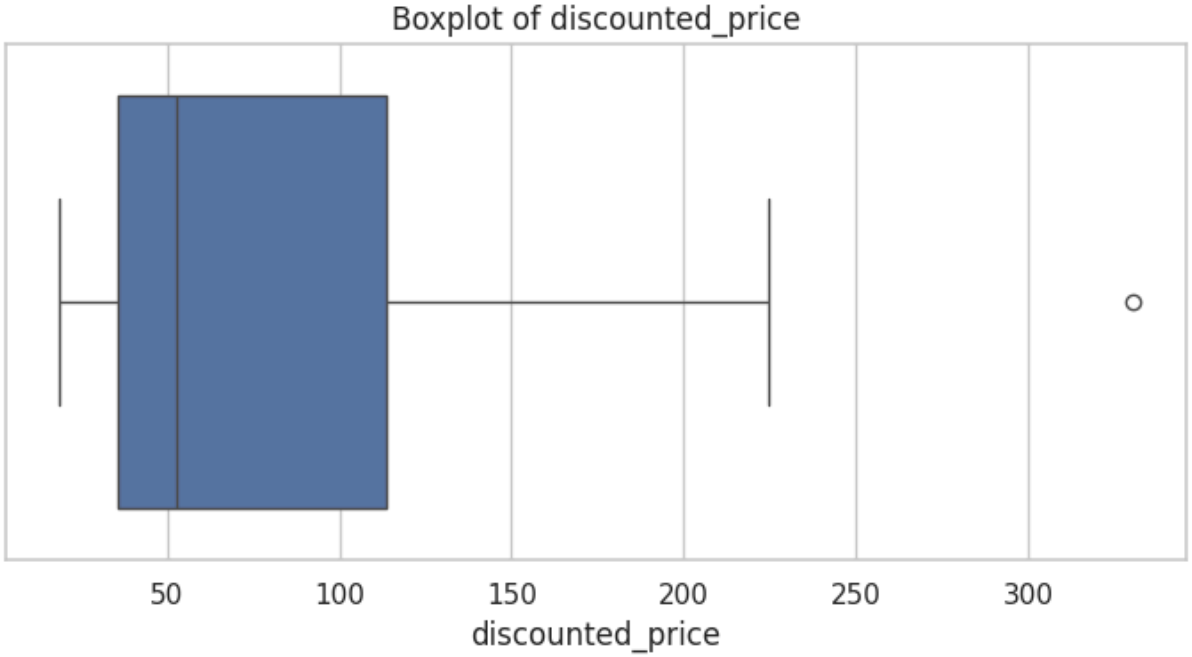
```
stock_available        20
dtype: int64

Numerical Columns:
['actual_price_msrp', 'discounted_price', 'discount_percentage', 'cost_price', 'rating', 'ra

Categorical Columns:
['product_id', 'product_name', 'category', 'sub_category']
```

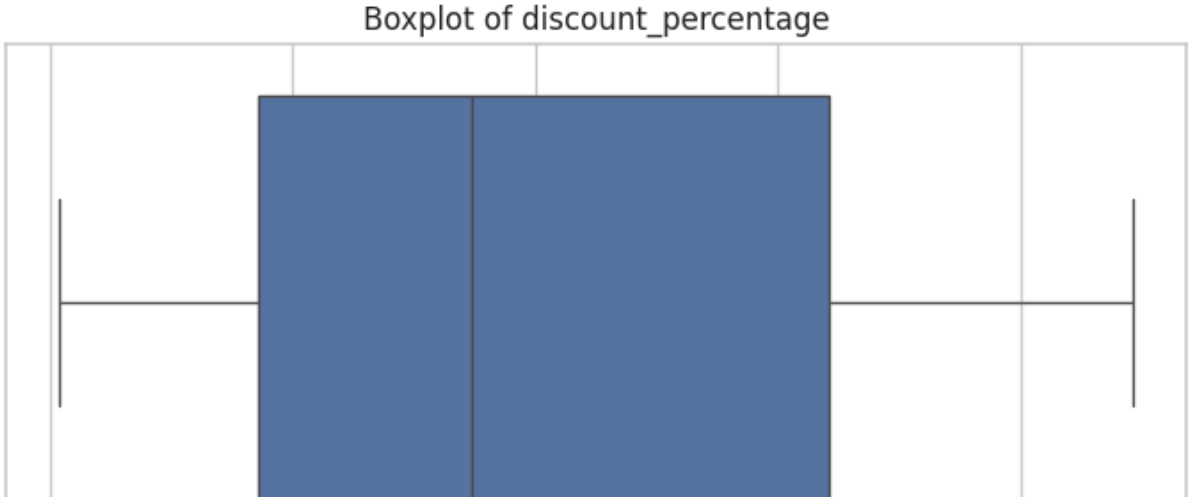


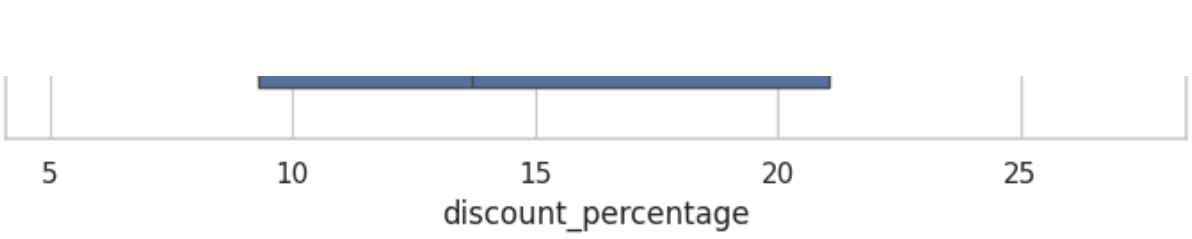


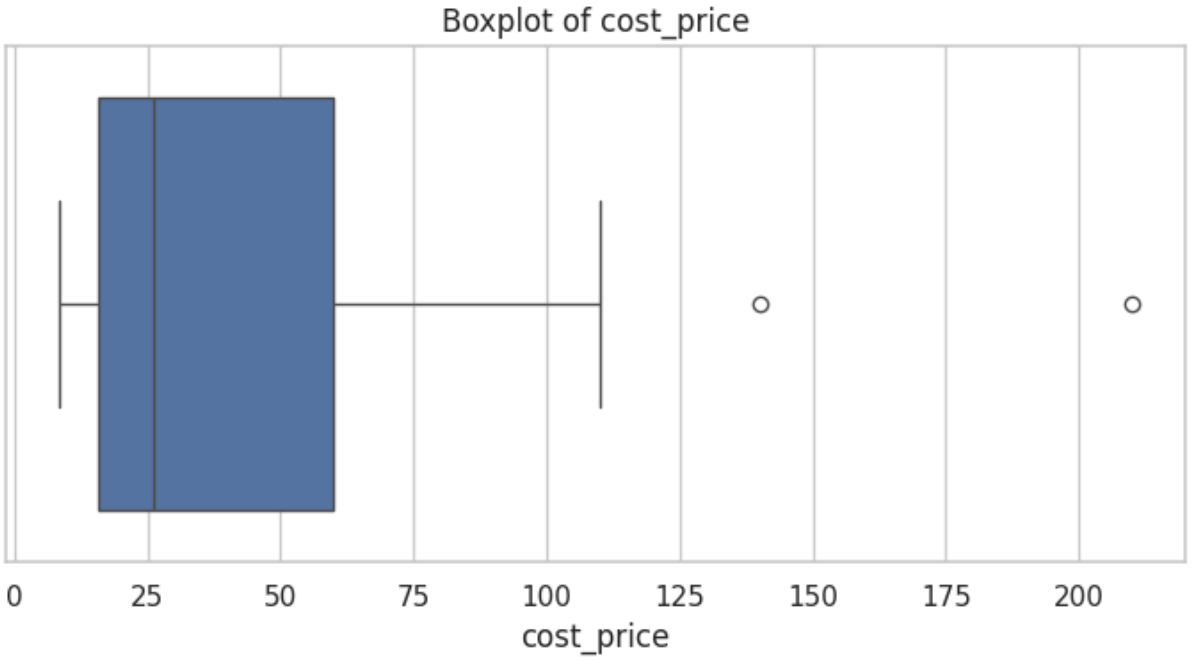

0.5
0.0
200
400
600
800
1000
1200
1400

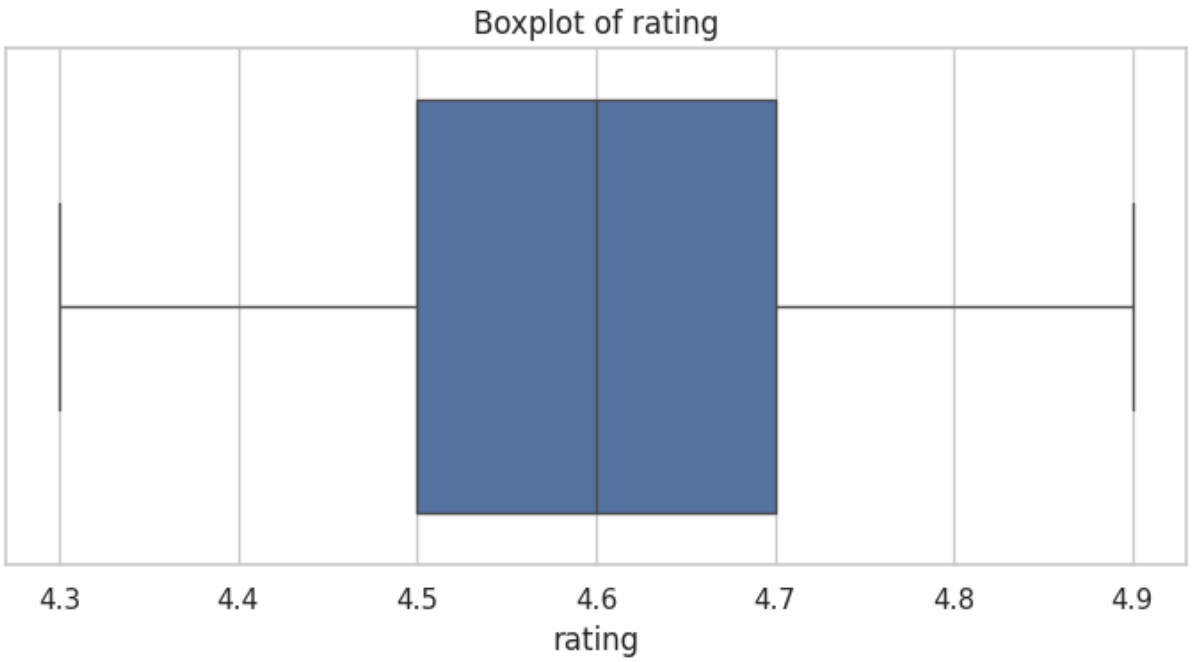

Boxplot of actual_price_msrp
50
100
150
200
250
300
350
actual_price_msrp

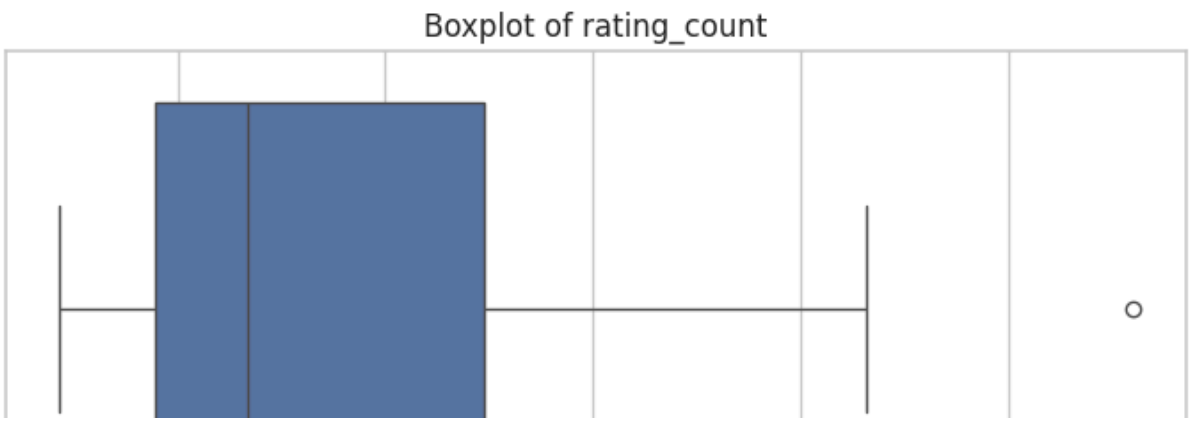

Boxplot of discounted_price
50
100
150
200
250
300
discounted_price

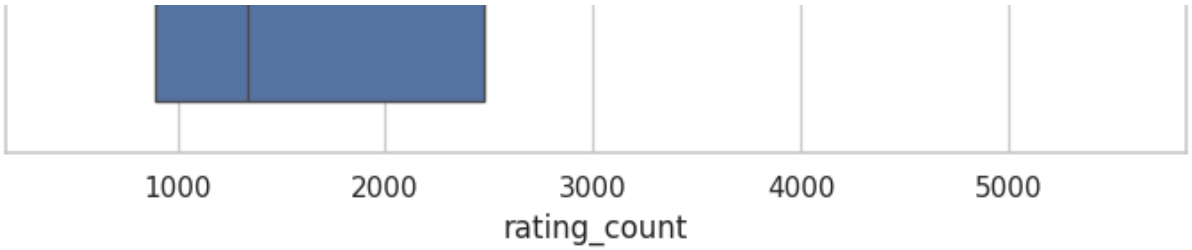

Boxplot of discount_percentage

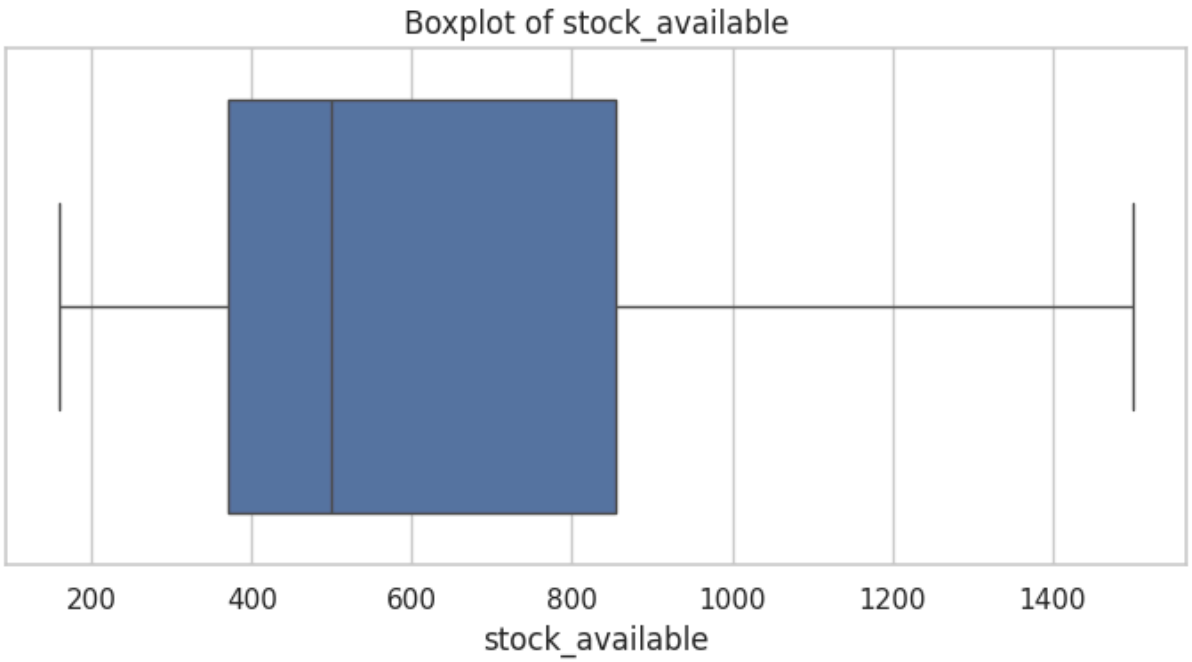

5
10
15
20
25
discount_percentage

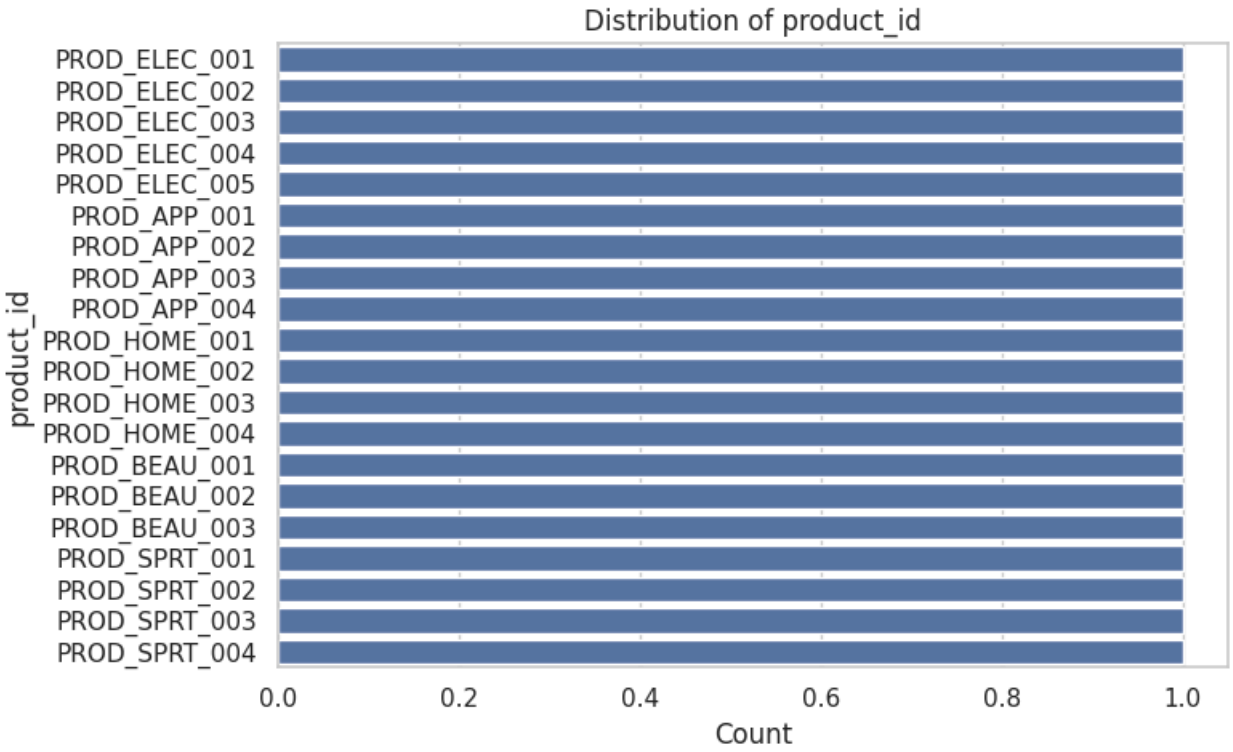

Boxplot of cost_price
0
25
50
75
100
125
150
175
200
cost_price

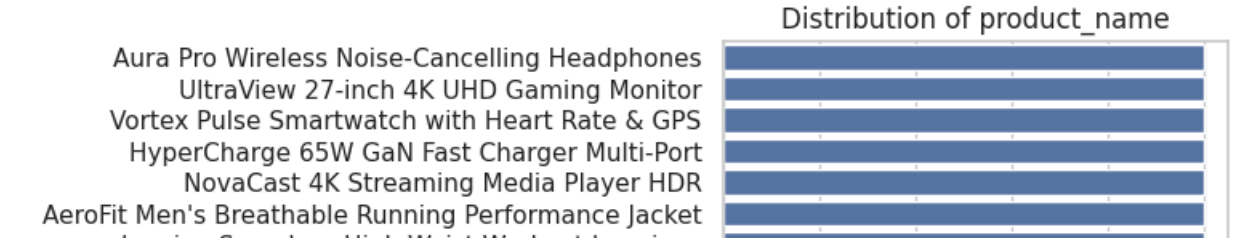

Boxplot of rating
4.3
4.4
4.5
4.6
4.7
4.8
4.9
rating

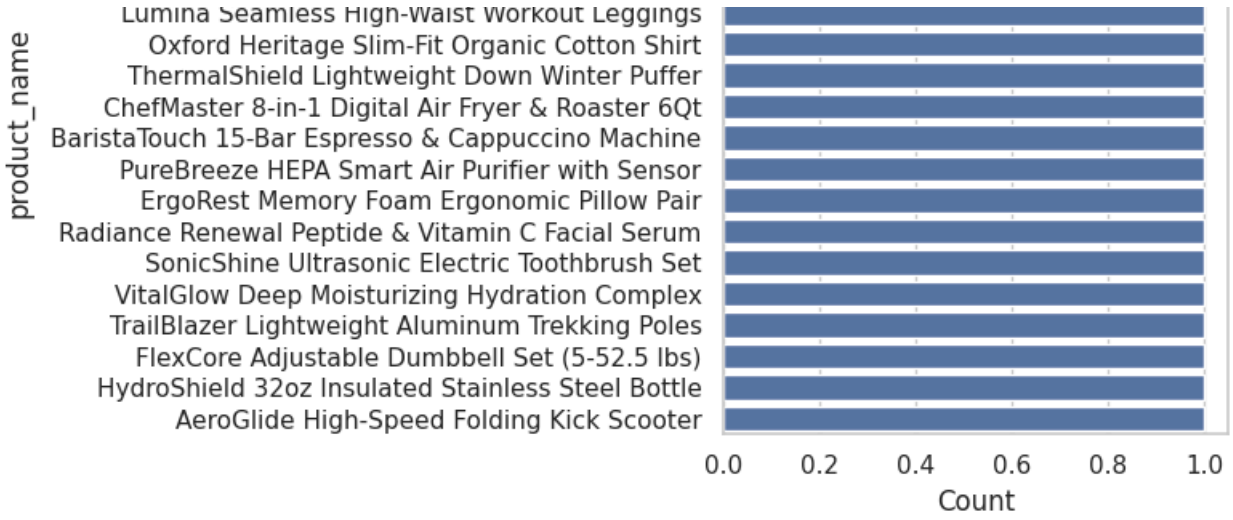

Boxplot of rating_count

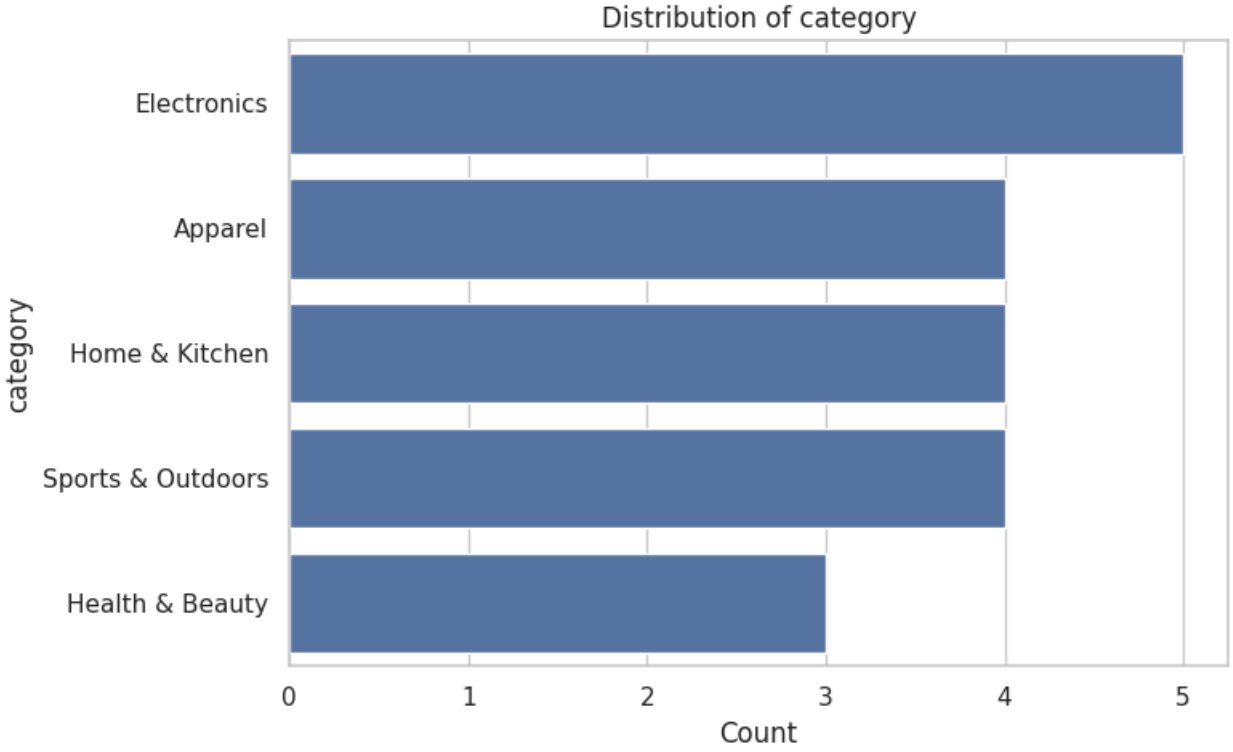

1000
2000
3000
4000
5000
rating_count

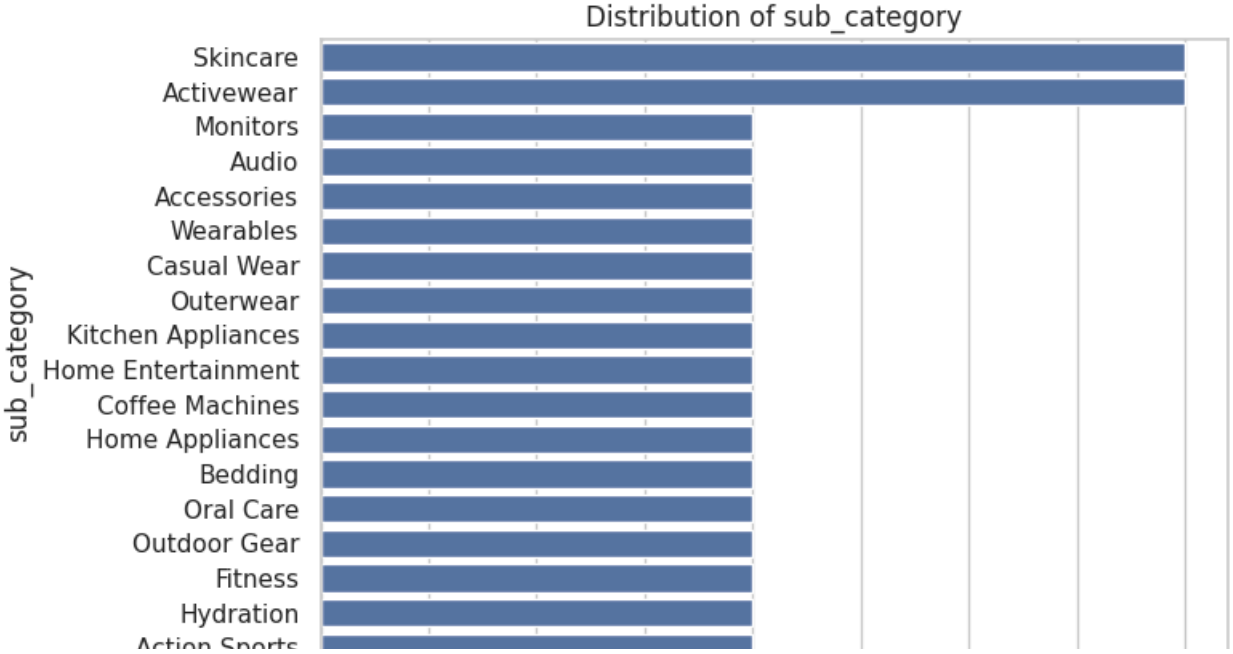

Boxplot of stock_available
200
400
600
800
1000
1200
1400
stock_available

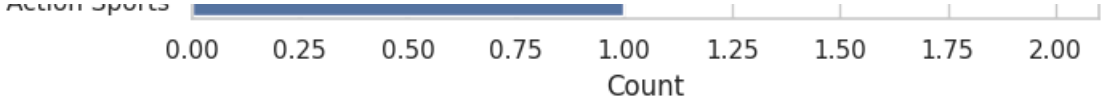

Distribution of product_id
PROD_ELEC_001
PROD_ELEC_002
PROD_ELEC_003
PROD_ELEC_004
PROD_ELEC_005
PROD_APP_001
PROD_APP_002
PROD_APP_003
PROD_APP_004
PROD_HOME_001
PROD_HOME_002
PROD_HOME_003
PROD_HOME_004
PROD_BEAU_001
PROD_BEAU_002
PROD_BEAU_003
PROD_SPRT_001
PROD_SPRT_002
PROD_SPRT_003
PROD_SPRT_004
product_id
0.0
0.2
0.4
0.6
0.8
1.0
Count

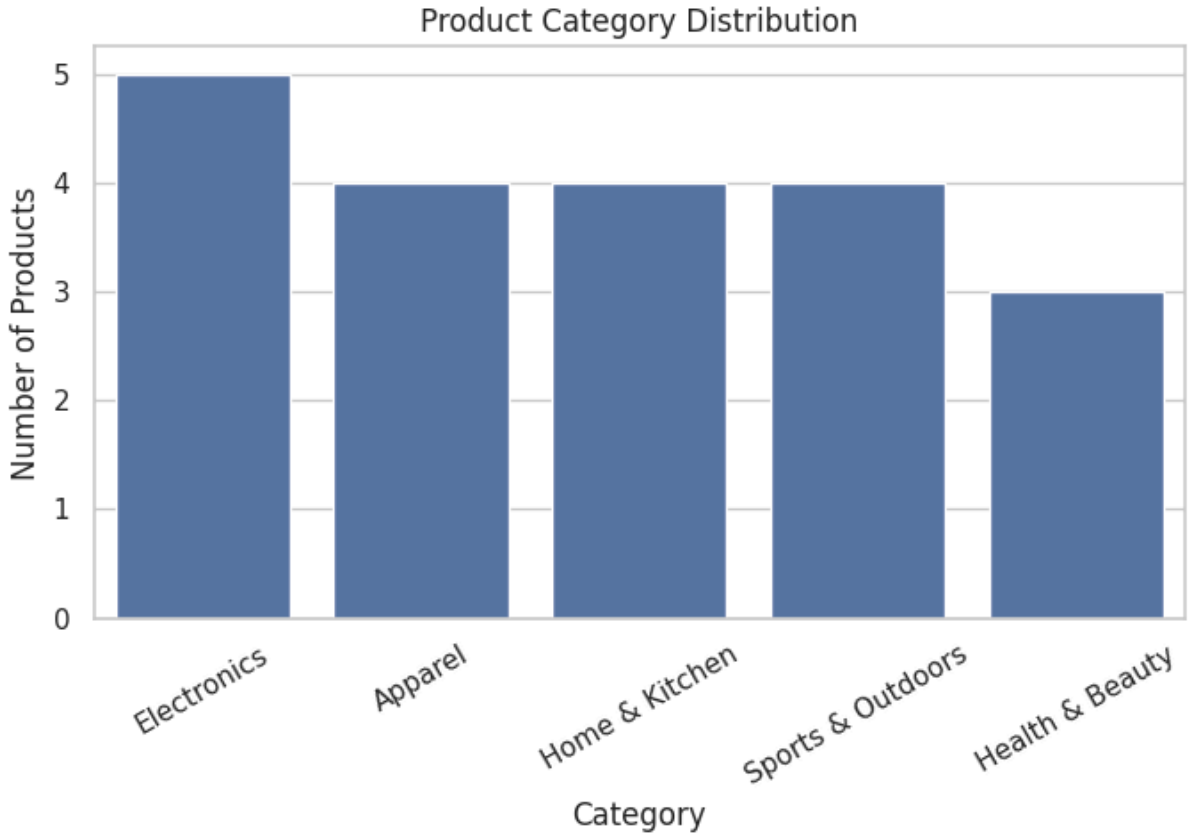

Distribution of product_name
Aura Pro Wireless Noise-Cancelling Headphones
UltraView 27-inch 4K UHD Gaming Monitor
Vortex Pulse Smartwatch with Heart Rate & GPS
HyperCharge 65W GaN Fast Charger Multi-Port
NovaCast 4K Streaming Media Player HDR
AeroFit Men's Breathable Running Performance Jacket

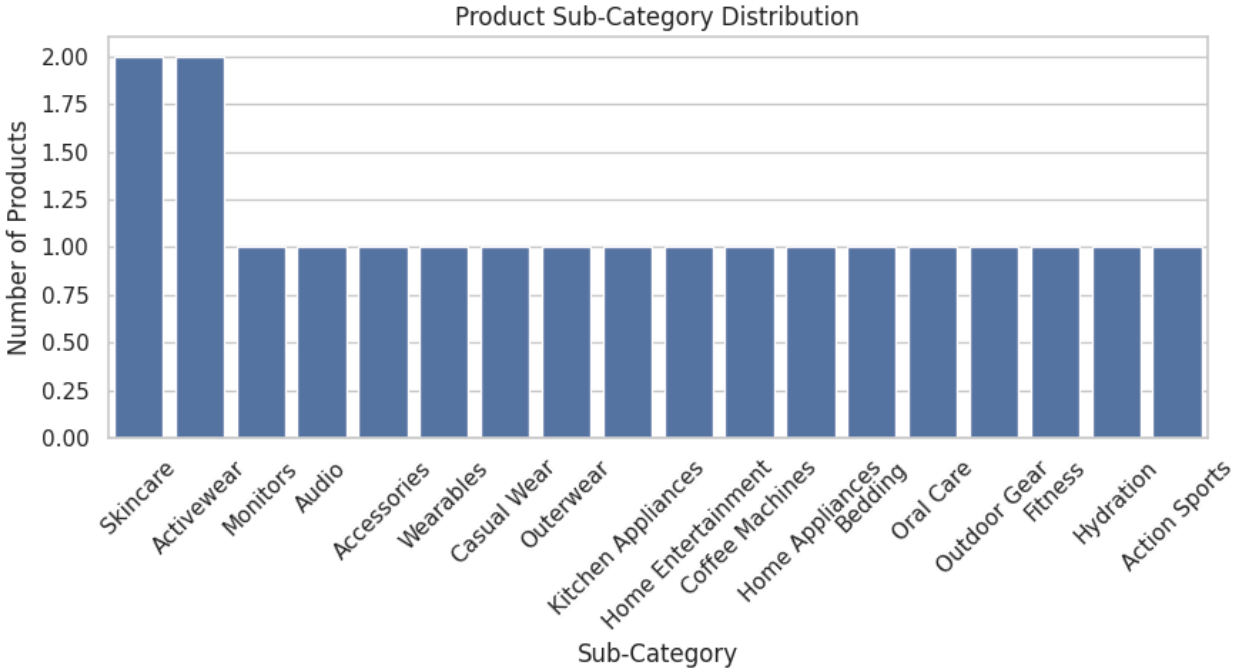

product_name
Lumina Seamless High-Waist Workout Leggings
Oxford Heritage Slim-Fit Organic Cotton Shirt
ThermalShield Lightweight Down Winter Puffer
ChefMaster 8-in-1 Digital Air Fryer & Roaster 6Qt
BaristaTouch 15-Bar Espresso & Cappuccino Machine
PureBreeze HEPA Smart Air Purifier with Sensor
ErgoRest Memory Foam Ergonomic Pillow Pair
Radiance Renewal Peptide & Vitamin C Facial Serum
SonicShine Ultrasonic Electric Toothbrush Set
VitalGlow Deep Moisturizing Hydration Complex
TrailBlazer Lightweight Aluminum Trekking Poles
FlexCore Adjustable Dumbbell Set (5-52.5 lbs)
HydroShield 32oz Insulated Stainless Steel Bottle
AeroGlide High-Speed Folding Kick Scooter
0.0
0.2
0.4
0.6
0.8
1.0
Count


Distribution of category
category
Electronics
Apparel
Home & Kitchen
Sports & Outdoors
Health & Beauty
0
1
2
3
4
5
Count

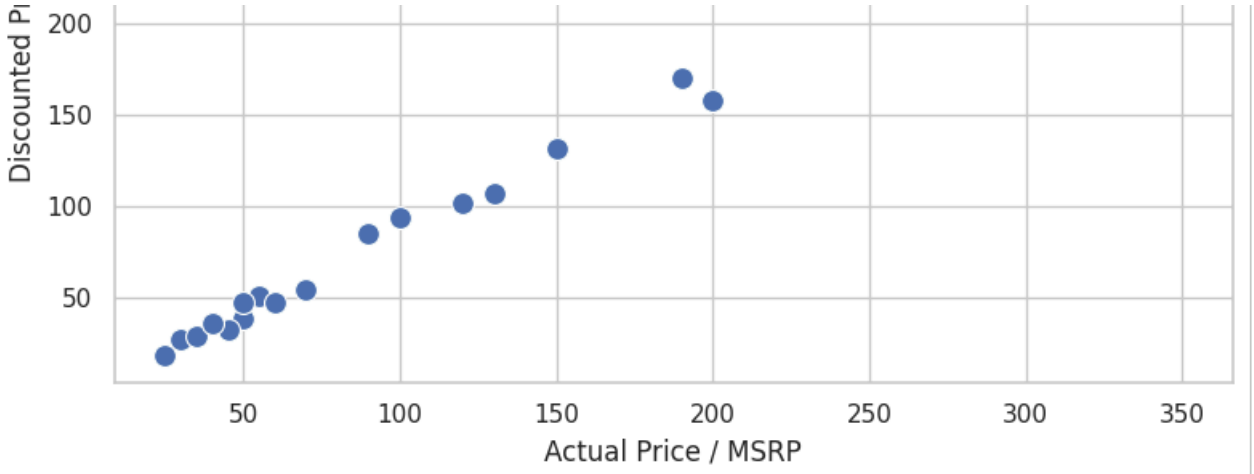

Distribution of sub_category
sub_category
Skincare
Activewear
Monitors
Audio
Accessories
Wearables
Casual Wear
Outerwear
Kitchen Appliances
Home Entertainment
Coffee Machines
Home Appliances
Bedding
Oral Care
Outdoor Gear
Fitness
Hydration

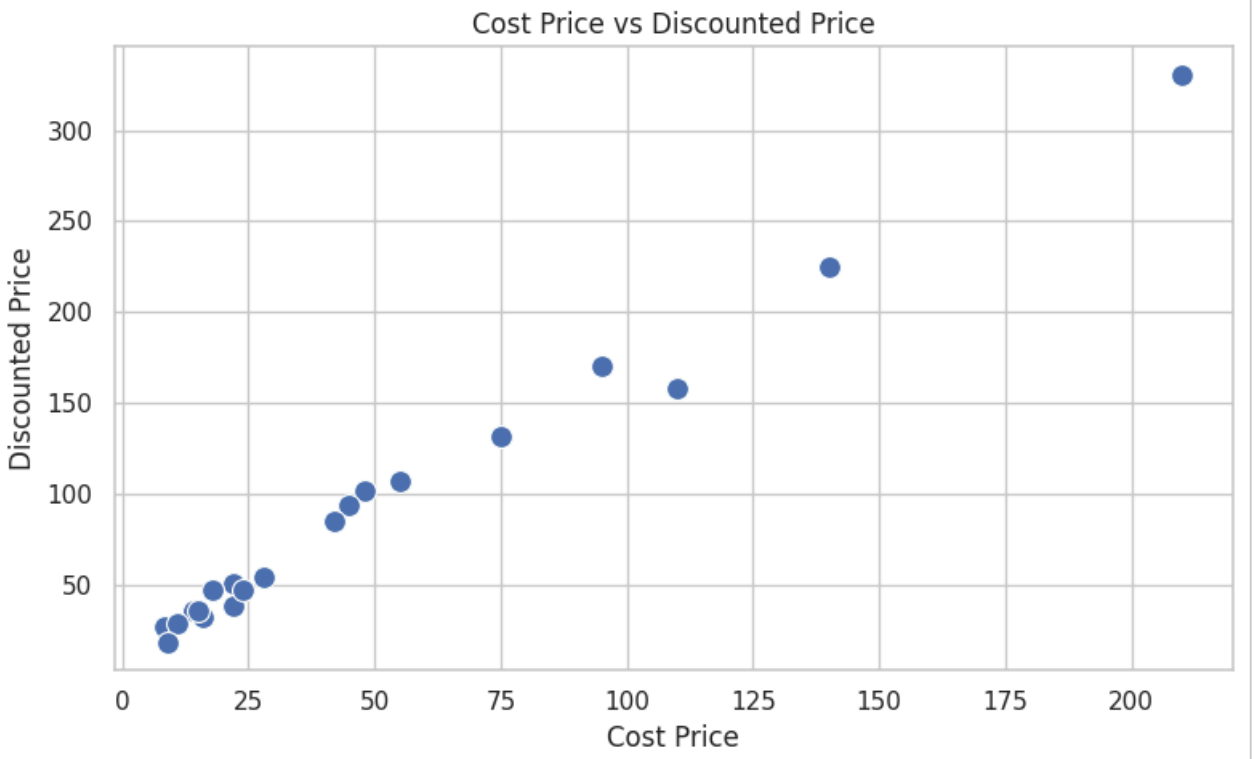

0.00
0.25
0.50
0.75
1.00
1.25
1.50
1.75
2.00
Count

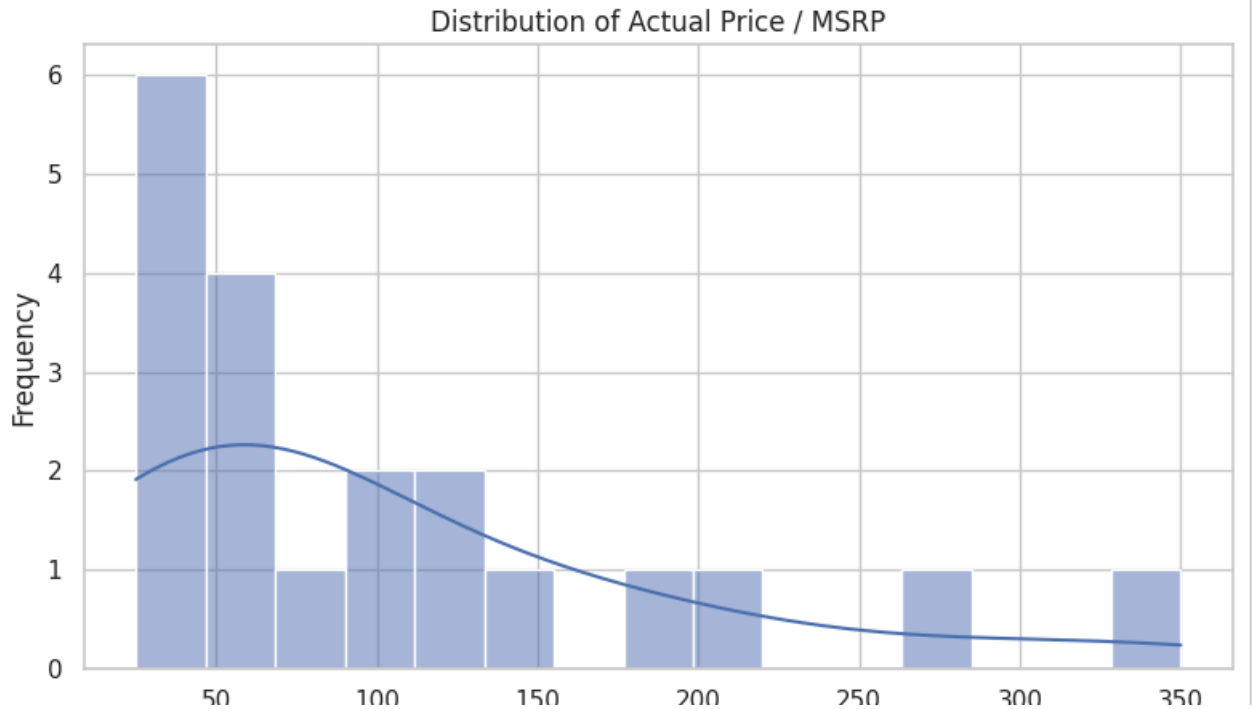

Product Category Distribution
Number of Products
0
1
2
3
4
5
Electronics
Apparel
Home & Kitchen
Sports & Outdoors
Health & Beauty
Category


Product Sub-Category Distribution
Number of Products
0.00
0.25
0.50
0.75
1.00
1.25
1.50
1.75
2.00
Skincare
Activewear
Monitors
Audio
Accessories
Wearables
Casual Wear
Outerwear
Kitchen Appliances
Home Entertainment
Coffee Machines
Home Appliances
Bedding
Oral Care
Outdoor Gear
Fitness
Hydration
Action Sports
Sub-Category

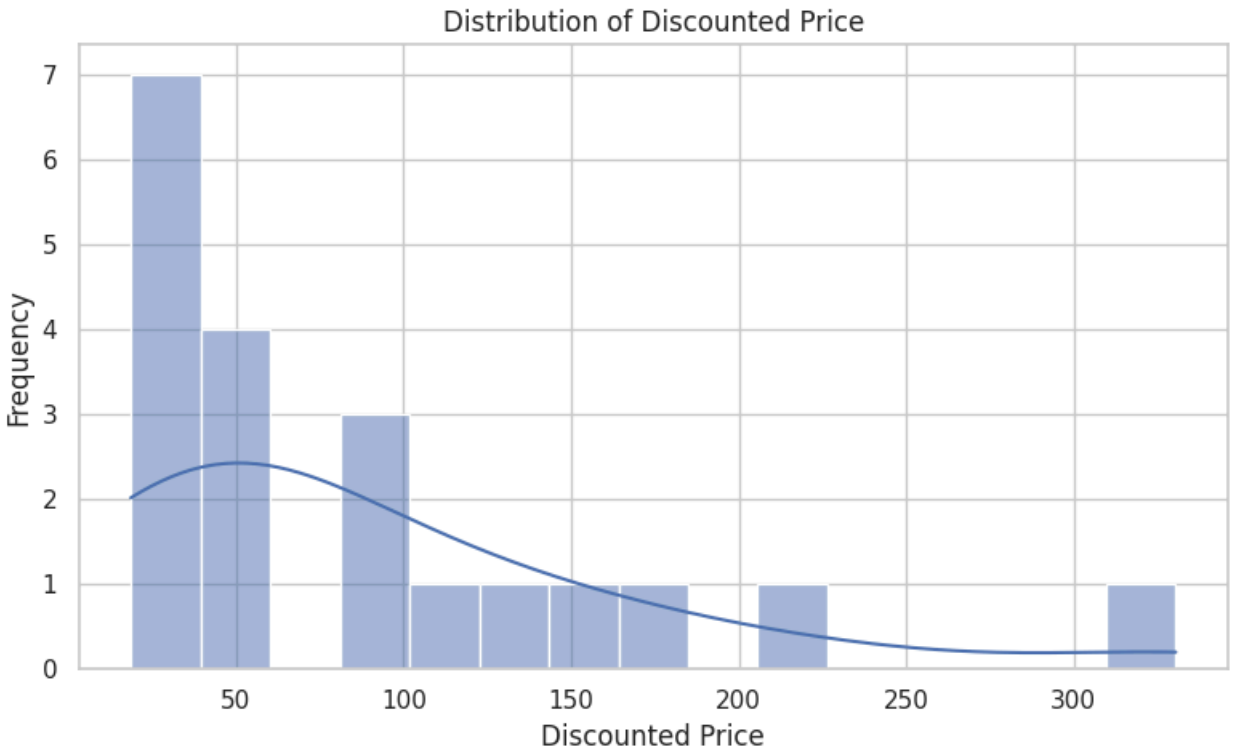

Actual Price vs Discounted Price
300
250

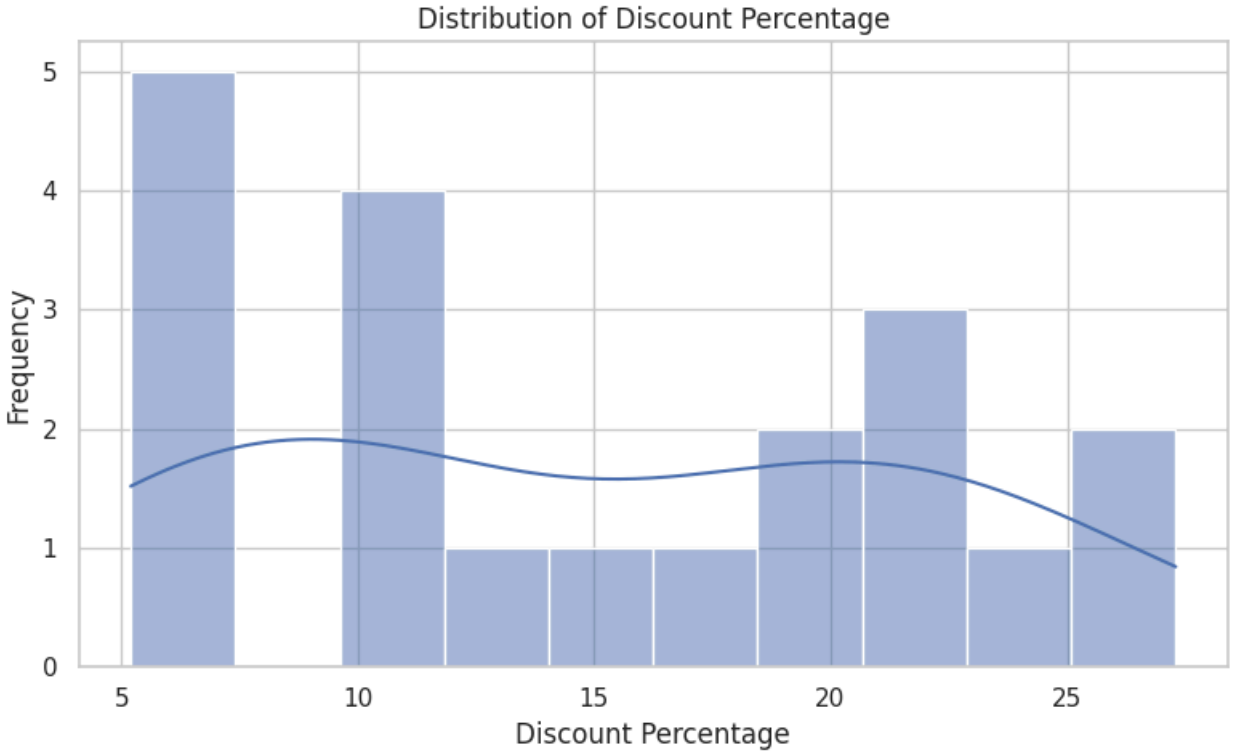

Discounted Pr
200
150
100
50
50
100
150
200
250
300
350
Actual Price / MSRP

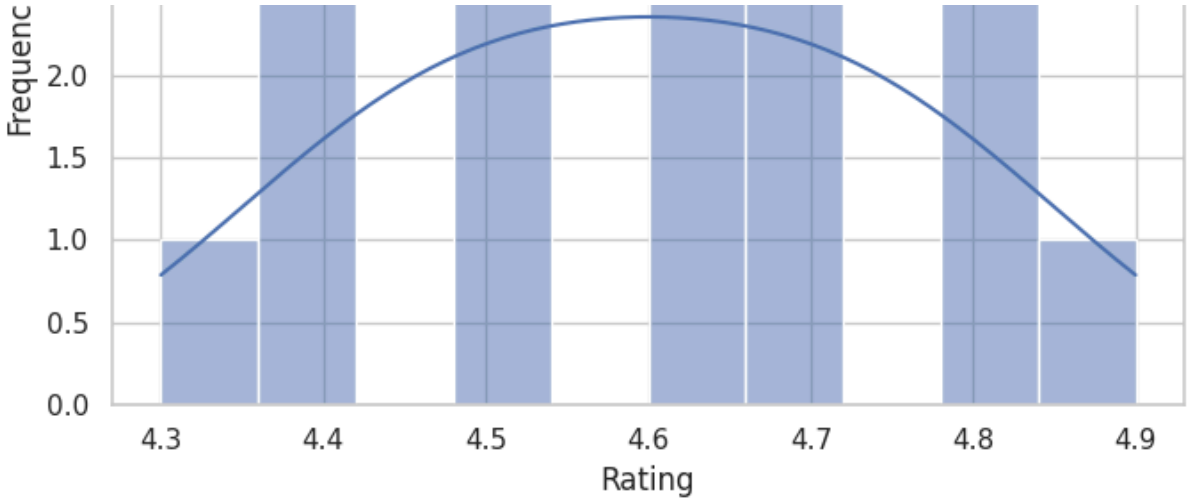

Cost Price vs Discounted Price
Discounted Price
300
250
200
150
100
50
0
25
50
75
100
125
150
175
200
Cost Price

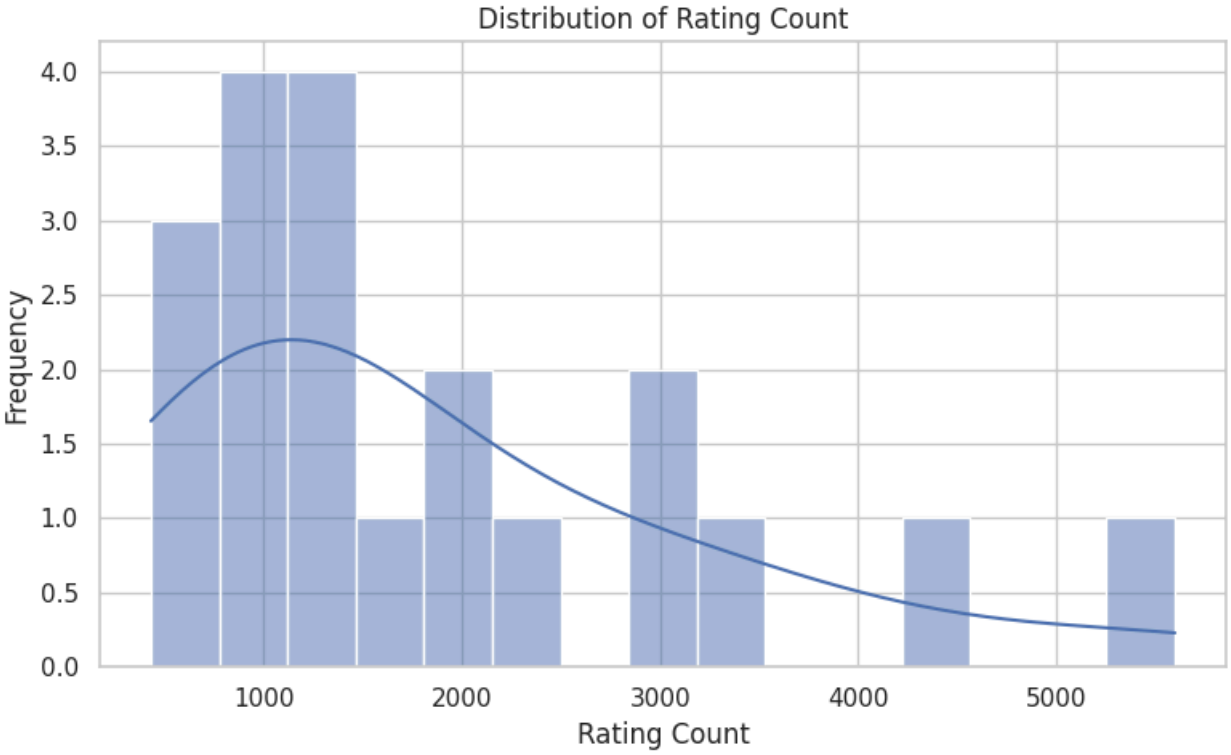

Distribution of Actual Price / MSRP
Frequency
6
5
4
3
2
1
0
50
100
150
200
250
300
350

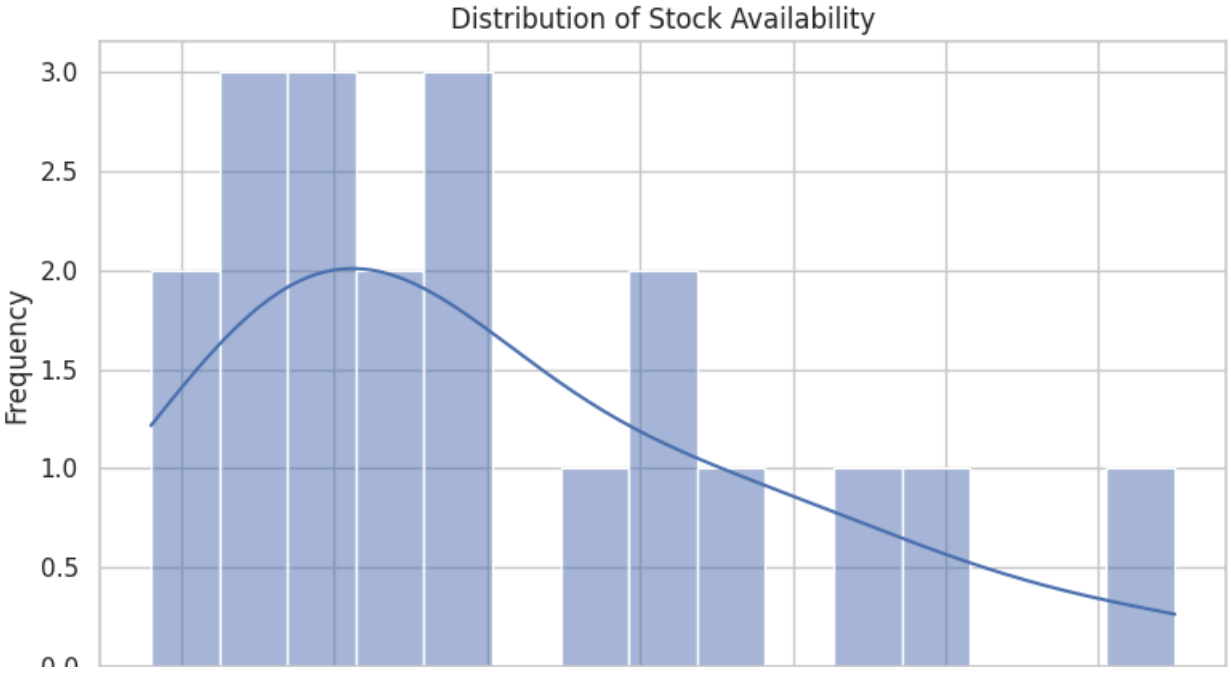
Actual Price

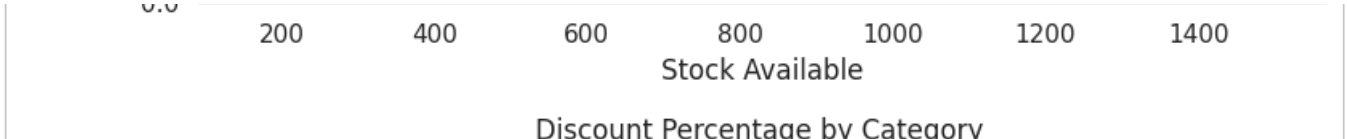


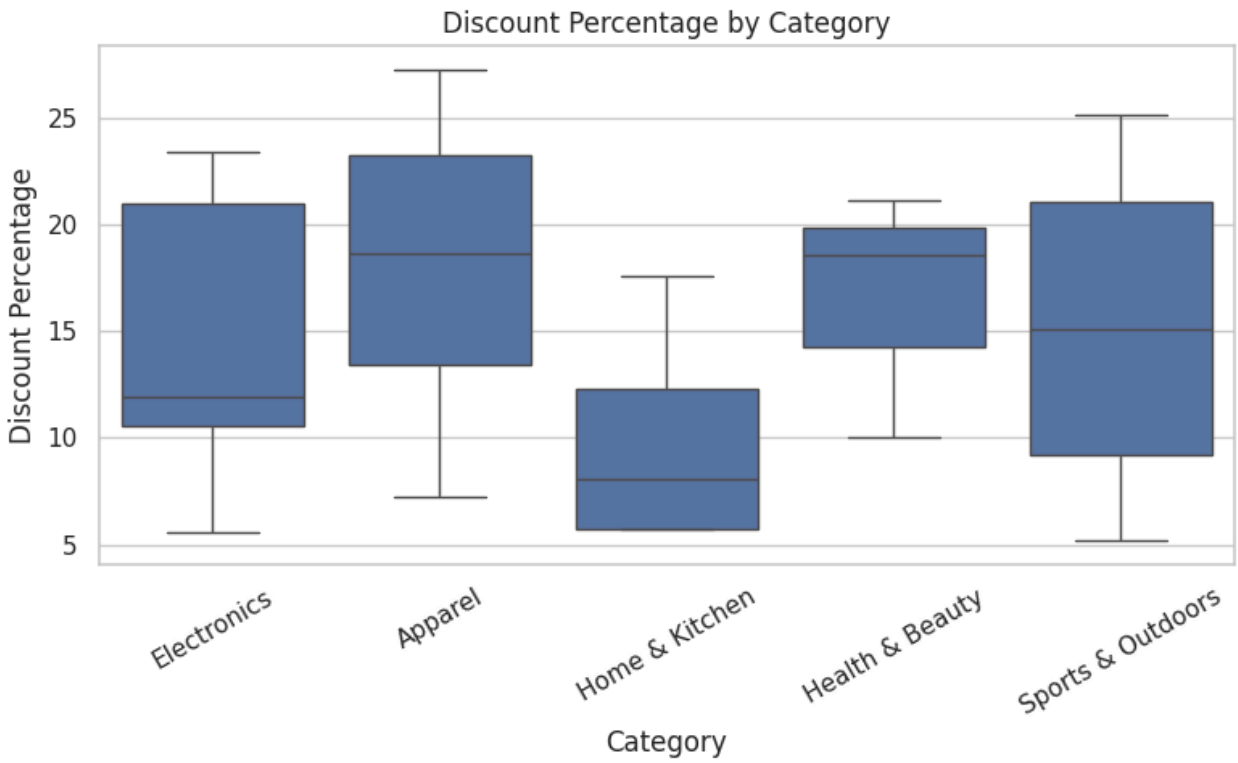


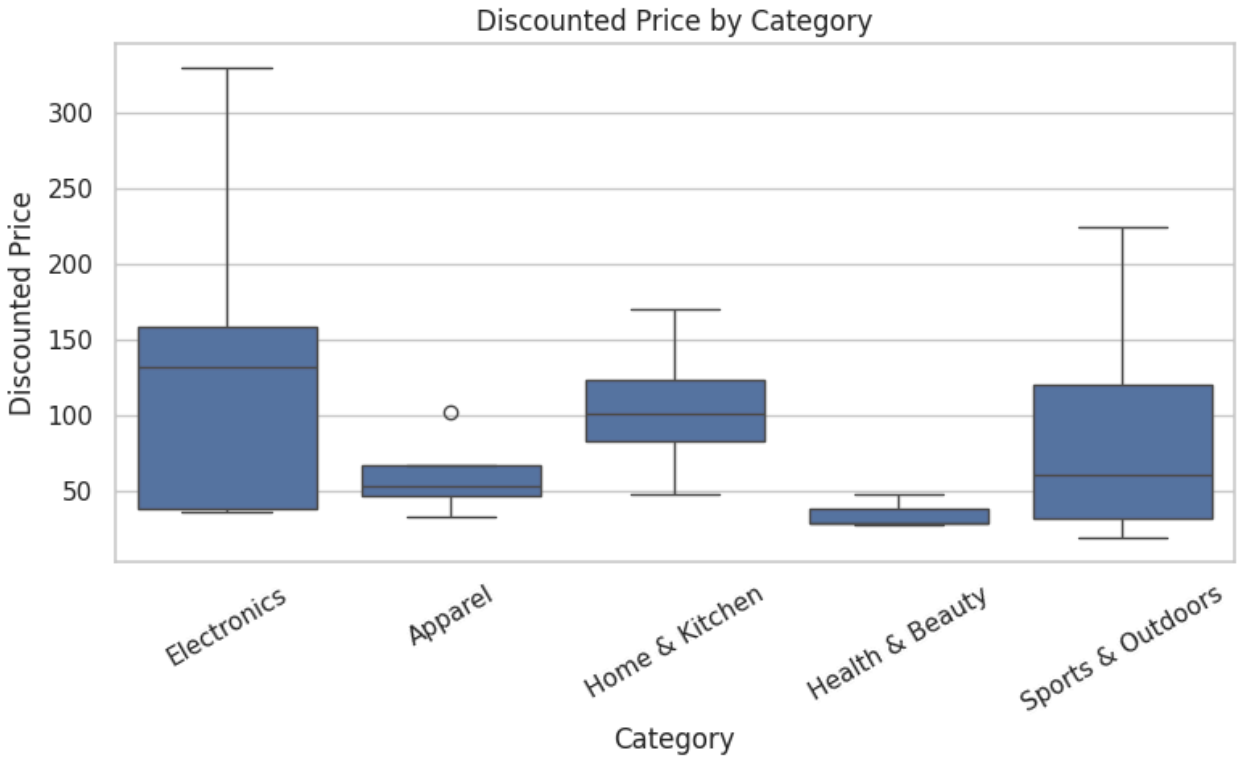

Frequenc
2.0
1.5
1.0
0.5
0.0
4.3
4.4
4.5
4.6
4.7
4.8
4.9
Rating

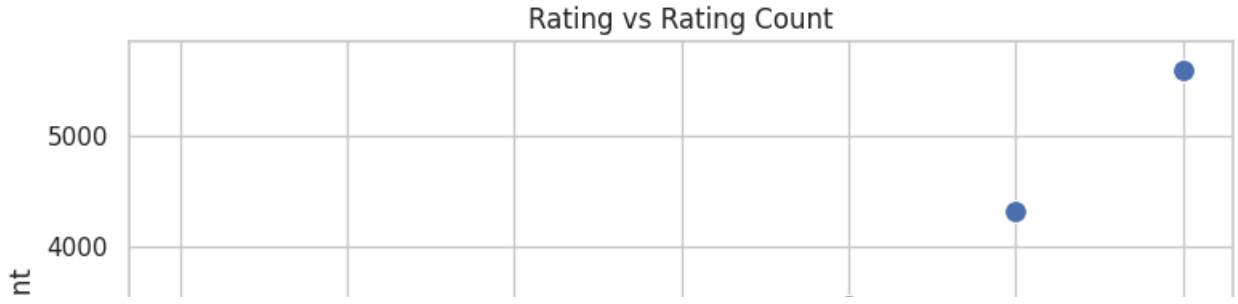

Distribution of Rating Count
Frequency
4.0
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
1000
2000
3000
4000
5000
Rating Count

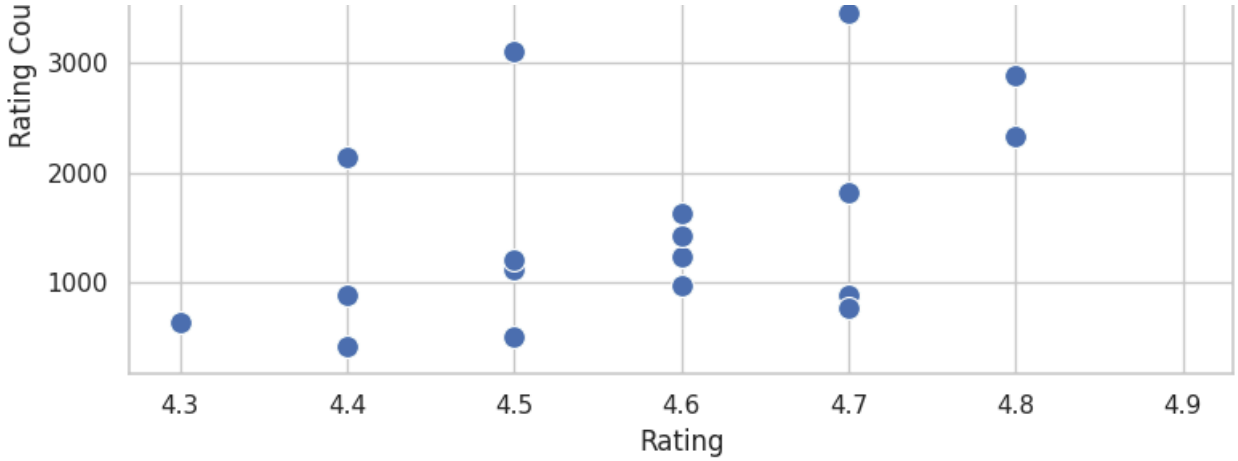

Distribution of Stock Availability
Frequency
3.0
2.5
2.0
1.5
1.0
0.5

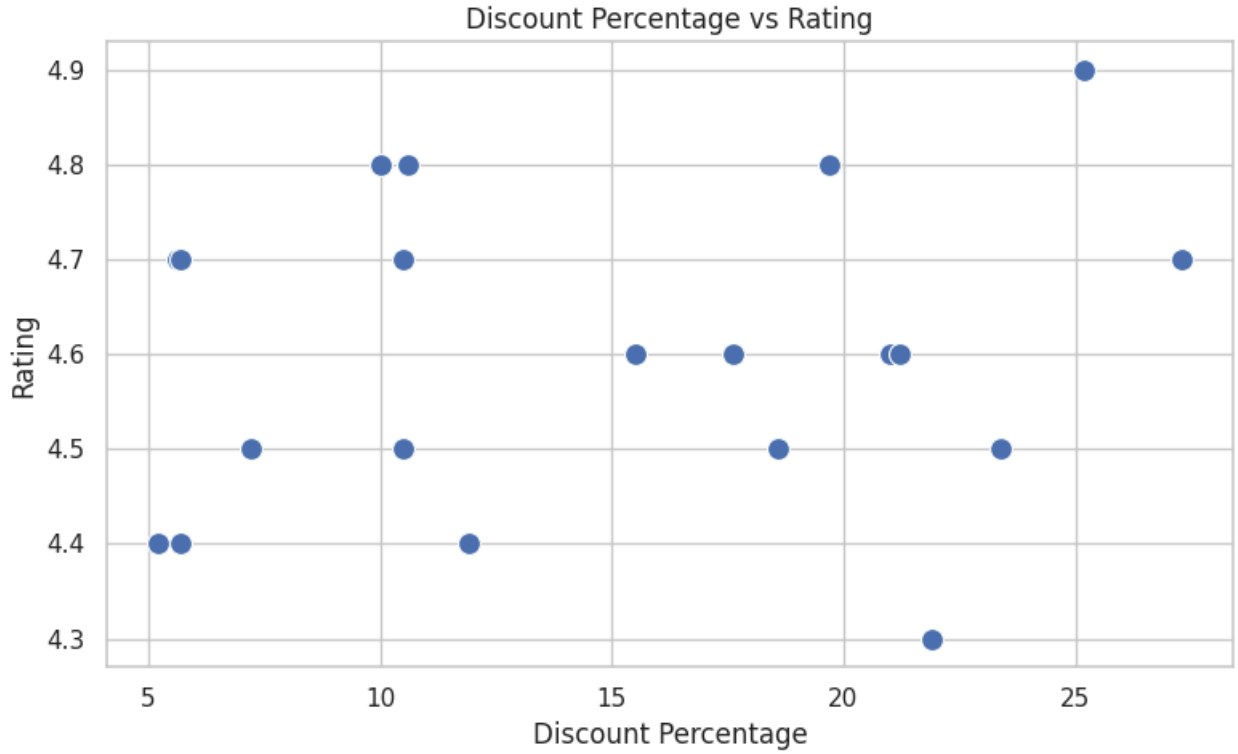

200
400
600
800
1000
1200
1400
Stock Available

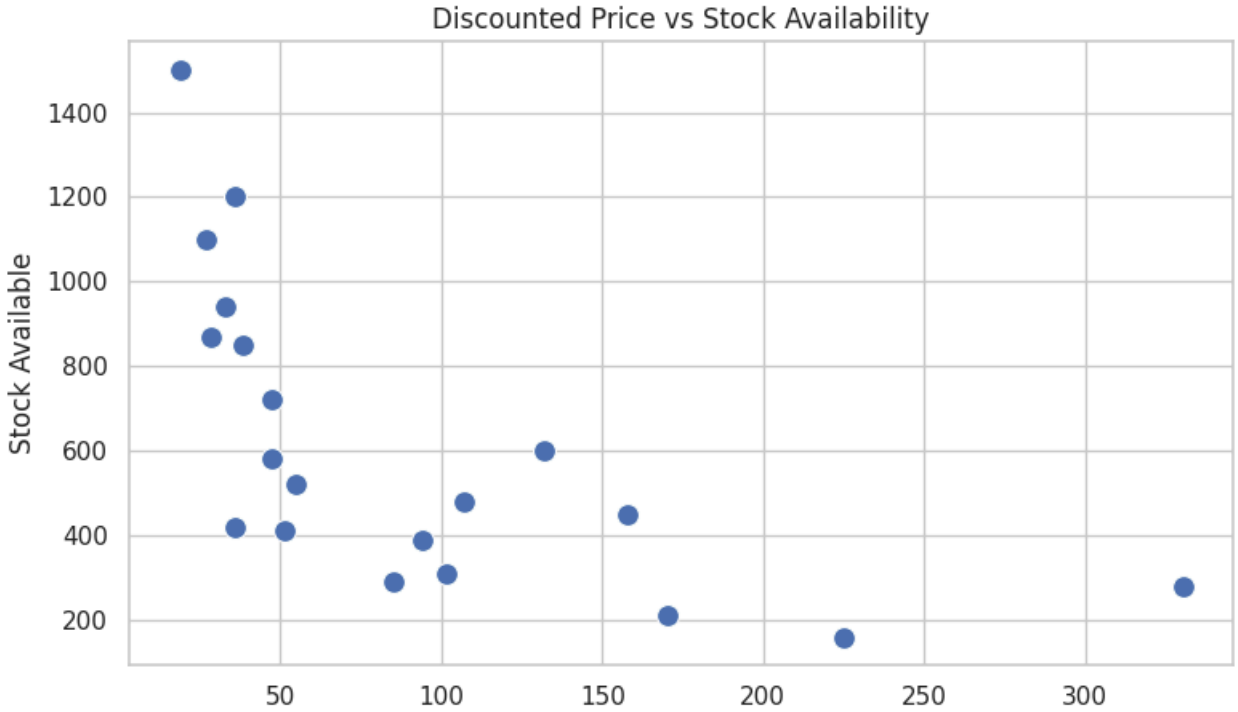

Discount Percentage by Category
Discount Percentage
25
20
15
10
5
Electronics
Apparel
Home & Kitchen
Health & Beauty
Sports & Outdoors
Category

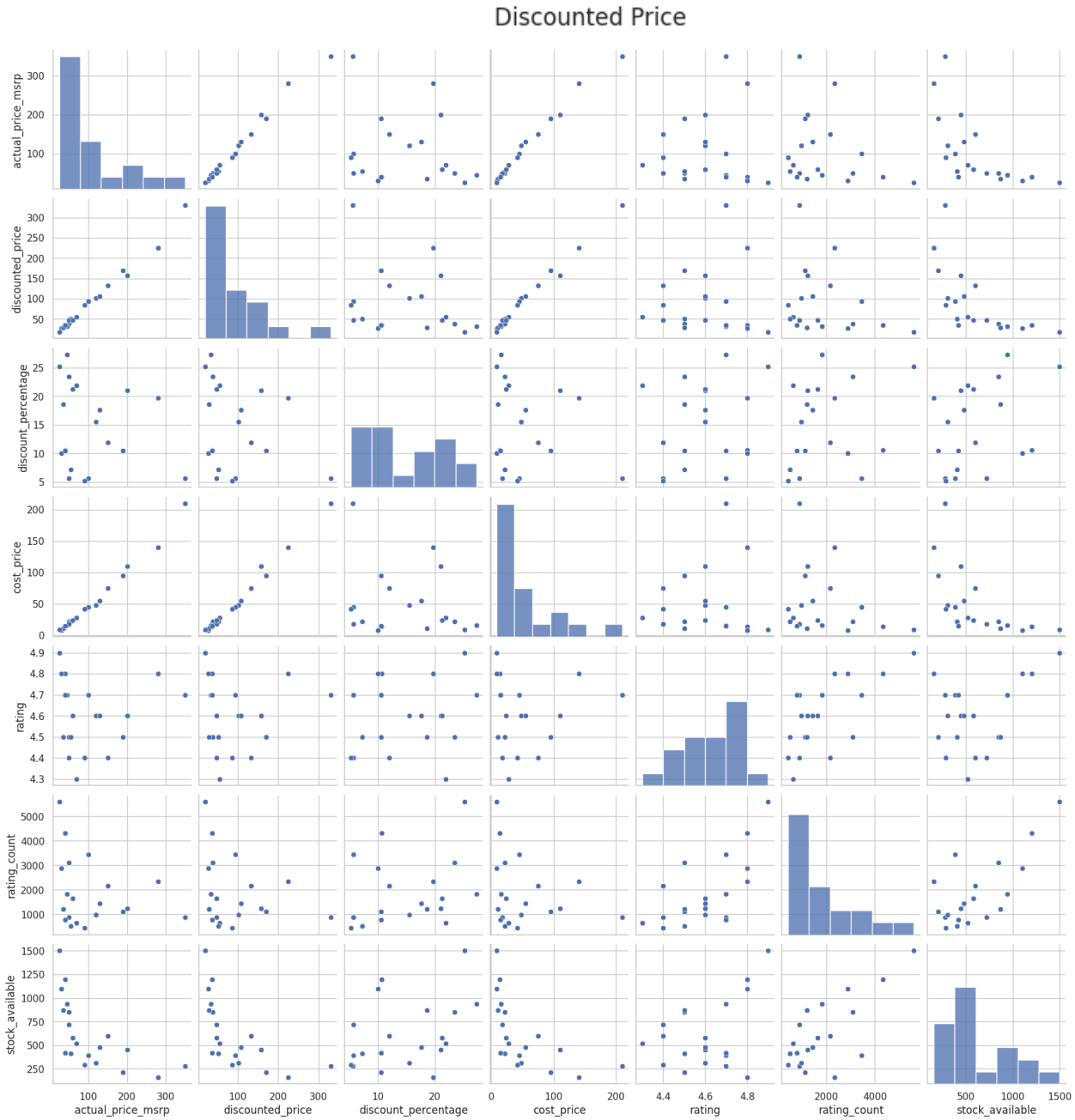

Discounted Price by Category
Discounted Price
300
250
200
150
100
50
Electronics
Apparel
Home & Kitchen
Health & Beauty
Sports & Outdoors
Category


Rating vs Rating Count
5000
4000


Rating Cou
3000
2000
1000
4.3
4.4
4.5
4.6
4.7
4.8
4.9
Rating


Discount Percentage vs Rating
Rating
4.9
4.8
4.7
4.6
4.5
4.4
4.3
5
10
15
20
25
Discount Percentage

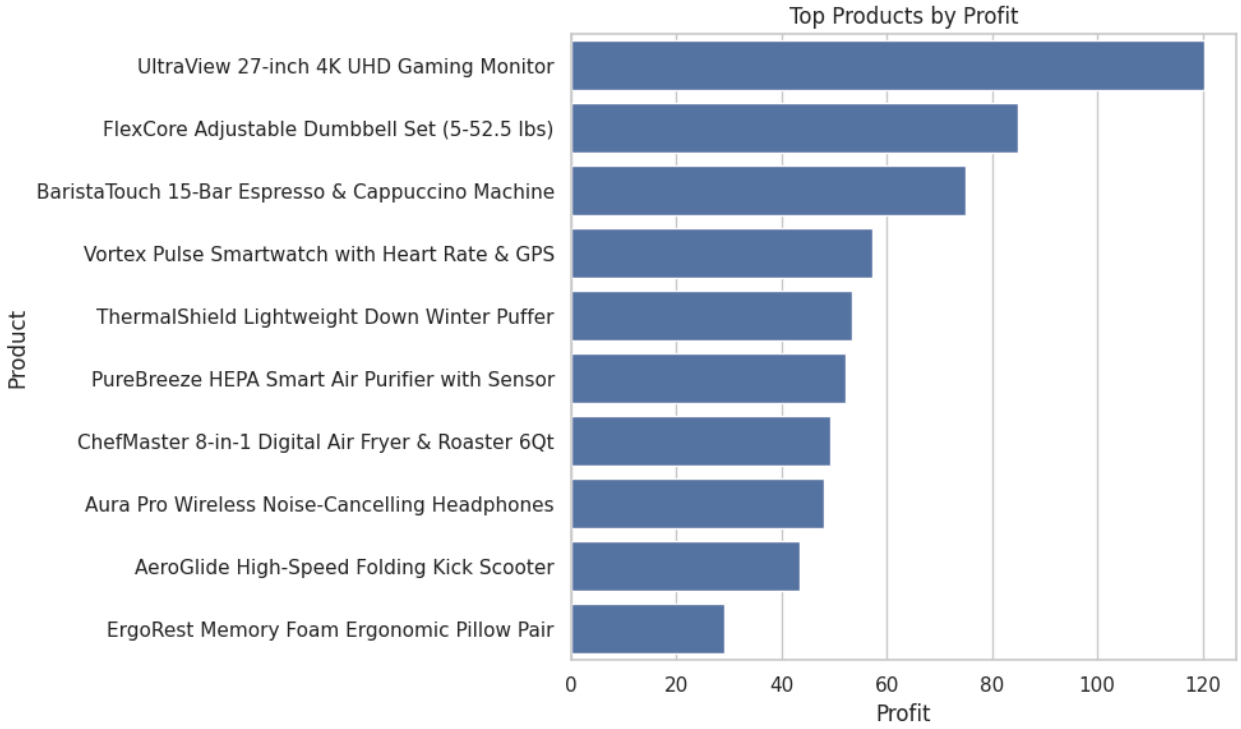

Discounted Price vs Stock Availability
Stock Available
1400
1200
1000
800
600
400
200
50
100
150
200
250
300

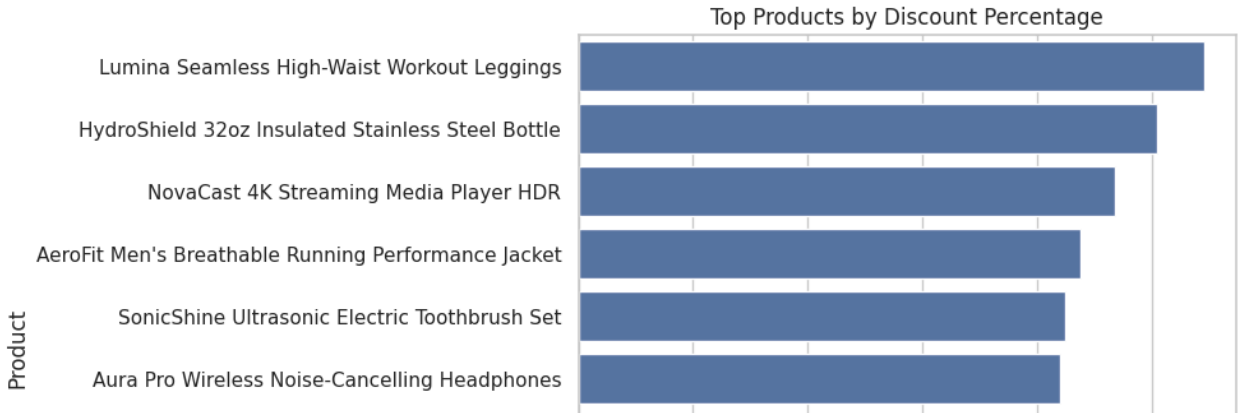


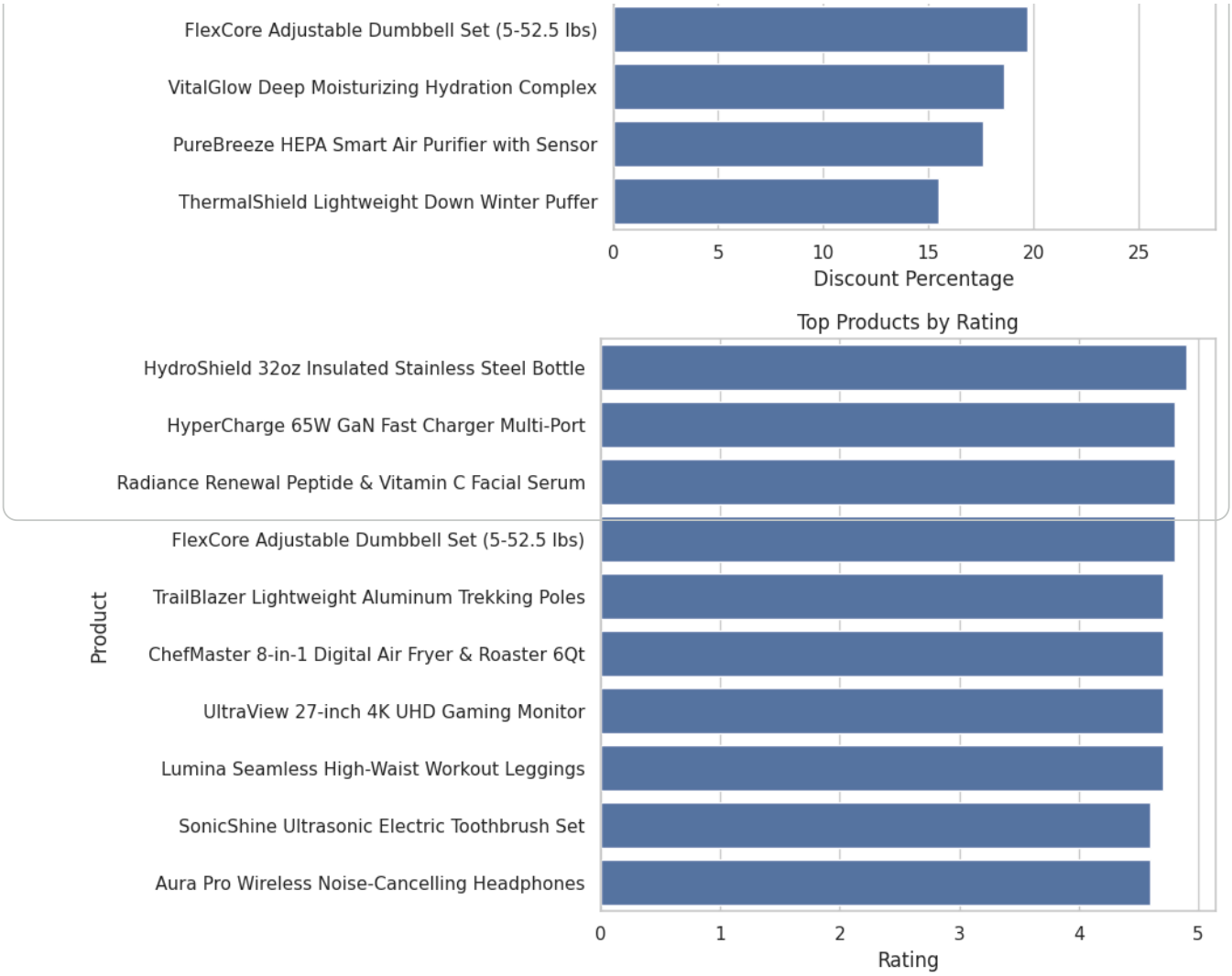

```
============================================================
PROFIT ANALYSIS
============================================================
                                        product_name  discounted_price  \
0       Aura Pro Wireless Noise-Cancelling Headphones            157.99
1             UltraView 27-inch 4K UHD Gaming Monitor            330.39
2       Vortex Pulse Smartwatch with Heart Rate & GPS            132.14
3         HyperCharge 65W GaN Fast Charger Multi-Port             35.75
4              NovaCast 4K Streaming Media Player HDR             38.29
5   AeroFit Men's Breathable Running Performance J...             54.66
6         Lumina Seamless High-Waist Workout Leggings             32.71
7       Oxford Heritage Slim-Fit Organic Cotton Shirt             51.03
8        ThermalShield Lightweight Down Winter Puffer            101.39
9   ChefMaster 8-in-1 Digital Air Fryer & Roaster 6Qt             94.29
10  BaristaTouch 15-Bar Espresso & Cappuccino Machine            170.04
11     PureBreeze HEPA Smart Air Purifier with Sensor            107.11
12         ErgoRest Memory Foam Ergonomic Pillow Pair             47.14
13  Radiance Renewal Peptide & Vitamin C Facial Serum             26.99
14      SonicShine Ultrasonic Electric Toothbrush Set             47.27
15      VitalGlow Deep Moisturizing Hydration Complex             28.48
```

```
16    TrailBlazer Lightweight Aluminum Trekking Poles            35.79
17      FlexCore Adjustable Dumbbell Set (5-52.5 lbs)           224.83
18  HydroShield 32oz Insulated Stainless Steel Bottle            18.69
19          AeroGlide High-Speed Folding Kick Scooter            85.31
```

	cost_price	profit	profit_margin_percentage
0	110.0	47.99	30.375340
1	210.0	120.39	36.438754
2	75.0	57.14	43.242016
3	14.0	21.75	60.839161
4	22.0	16.29	42.543745
5	28.0	26.66	48.774241
6	16.0	16.71	51.085295
7	22.0	29.03	56.888105
8	48.0	53.39	52.658053
9	45.0	49.29	52.274897
10	95.0	75.04	44.130793
11	55.0	52.11	48.650920
12	18.0	29.14	61.815868
13	8.5	18.49	68.506854
14	24.0	23.27	49.227840
15	11.0	17.48	61.376404
16	15.0	20.79	58.088852
17	140.0	84.83	37.730730
18	9.0	9.69	51.845907
19	42.0	43.31	50.767788








```
============================================================
EDA SUMMARY
============================================================
Dataset Shape: (20, 13)
Total Products: 20
Total Categories: 5
Total Sub-Categories: 18
Average Actual Price: 105.49
Average Discounted Price: 91.01
Average Discount: 14.72 %
Average Rating: 4.6
Average Stock: 614.0
Average Profit: 40.64

EDA Completed Successfully!
```

```
Start coding or generate with AI.
```

```
# ============================================================
# 2. Load Dataset
# ============================================================

from google.colab import files

uploaded = files.upload()

filename = next(iter(uploaded))
df = pd.read_csv(filename)

print("Dataset loaded successfully!")
print("File:", filename)
print("Shape:", df.shape)
```

```
Choose Files No file chosen  Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
Saving retail_price_optimization.csv to retail_price_optimization (1).csv
Dataset loaded successfully!
File: retail_price_optimization (1).csv
Shape: (1040, 13)
```

```
# ============================================================
# RETAIL PRICE OPTIMIZATION DATASET
# COMPLETE EXPLORATORY DATA ANALYSIS (EDA)
# ============================================================



# ============================================================
# 1. IMPORT LIBRARIES
# ============================================================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

import os
import glob

# Display settings
pd.set_option("display.max_columns", None)
pd.set_option("display.max_rows", 100)

sns.set_theme(style="whitegrid")

print("Libraries imported successfully!")



# ============================================================
# 2. LOAD DATASET
# ============================================================

# Automatically find CSV files in the current notebook folder
csv_files = glob.glob("*.csv")
```

```
csv_files = glob.glob("*.csv")

print("CSV files found:")
for file in csv_files:
    print(file)

if len(csv_files) == 0:
    raise FileNotFoundError(
        "No CSV file found. Please upload 'retail_price_optimization.csv' "
        "to the notebook folder."
    )

# If the required file exists, use it
required_file = "retail_price_optimization.csv"

if required_file in csv_files:
    file_path = required_file
else:
    # Use first CSV if exact filename is different
    file_path = csv_files[0]

df = pd.read_csv(file_path)

print("\nDataset loaded successfully!")
print("File:", file_path)
print("Shape:", df.shape)

display(df.head())


# ============================================================
# 3. BASIC DATASET INFORMATION
# ============================================================

print("\n========== FIRST 5 ROWS ==========")
display(df.head())

print("\n========== LAST 5 ROWS ==========")
display(df.tail())

print("\n========== DATASET SHAPE ==========")
print(df.shape)

print("\n========== COLUMN NAMES ==========")
print(df.columns.tolist())

print("\n========== DATASET INFO ==========")
df.info()

print("\n========== DATA TYPES ==========")
print(df.dtypes)


# ============================================================
# 4. MISSING VALUES ANALYSIS
# ============================================================

print("\n========== MISSING VALUES ==========")
```

```
missing_values = df.isnull().sum()

missing_percentage = (
    df.isnull().sum() / len(df) * 100
).round(2)

missing_df = pd.DataFrame({
    "Missing Values": missing_values,
    "Missing Percentage": missing_percentage
})

display(missing_df)

print("\nTotal missing values:", df.isnull().sum().sum())


# ============================================================
# 5. DUPLICATE VALUES
# ============================================================

print("\n========== DUPLICATE ROWS ==========")

duplicate_count = df.duplicated().sum()

print("Number of duplicate rows:", duplicate_count)

if duplicate_count > 0:
    df = df.drop_duplicates().reset_index(drop=True)
    print("Duplicates removed.")
else:
    print("No duplicate rows found.")


# ============================================================
# 6. STATISTICAL SUMMARY
# ============================================================

print("\n========== STATISTICAL SUMMARY ==========")

display(df.describe())

print("\n========== TRANSPOSED STATISTICAL SUMMARY ==========")

display(df.describe().T)


# ============================================================
# 7. UNIQUE VALUES
# ============================================================

print("\n========== UNIQUE VALUES ==========")

unique_values = df.nunique().sort_values()

display(unique_values.to_frame("Number of Unique Values"))


# ============================================================
```

```
# 8. NUMERICAL AND CATEGORICAL COLUMNS
# ============================================================

numeric_columns = df.select_dtypes(
    include=np.number
).columns.tolist()

categorical_columns = df.select_dtypes(
    exclude=np.number
).columns.tolist()

print("\n========== NUMERICAL COLUMNS ==========")
print(numeric_columns)

print("\n========== CATEGORICAL COLUMNS ==========")
print(categorical_columns)


# ============================================================
# 9. CORRELATION HEATMAP
# ============================================================

print("\n========== CORRELATION MATRIX ==========")

correlation_matrix = df[numeric_columns].corr()

display(correlation_matrix)

plt.figure(figsize=(12, 8))

sns.heatmap(
    correlation_matrix,
    annot=True,
    fmt=".2f",
    cmap="coolwarm",
    linewidths=0.5
)

plt.title("Correlation Heatmap")
plt.tight_layout()
plt.show()


# ============================================================
# 10. HISTOGRAMS OF NUMERICAL VARIABLES
# ============================================================

print("\n========== NUMERICAL DISTRIBUTIONS ==========")

for column in numeric_columns:

    plt.figure(figsize=(8, 5))

    sns.histplot(
        data=df,
        x=column,
        kde=True
    )
```

```
    plt.title(f"Distribution of {column}")
    plt.xlabel(column)
    plt.ylabel("Frequency")

    plt.tight_layout()
    plt.show()


# ============================================================
# 11. BOXPLOTS
# ============================================================

print("\n========== BOXPLOTS ==========")

for column in numeric_columns:

    plt.figure(figsize=(8, 4))

    sns.boxplot(
        x=df[column]
    )

    plt.title(f"Boxplot of {column}")
    plt.xlabel(column)

    plt.tight_layout()
    plt.show()


# ============================================================
# 12. PRODUCT CATEGORY DISTRIBUTION
# ============================================================

print("\n========== PRODUCT CATEGORY DISTRIBUTION ==========")

category_counts = df["product_category_name"].value_counts()

display(category_counts.to_frame("Count"))

plt.figure(figsize=(10, 6))

sns.countplot(
    data=df,
    y="product_category_name",
    order=category_counts.index
)

plt.title("Product Category Distribution")
plt.xlabel("Number of Records")
plt
```

```
Libraries imported successfully!
CSV files found:
amazon_product_pricing.csv
retail_price_optimization.csv
brazilian_ecommerce_olist (1).csv
retail_price_optimization (1).csv
brazilian_ecommerce_olist.csv

Dataset loaded successfully!
File: retail_price_optimization.csv
Shape: (1040, 13)
```

	product_id	product_category_name	week_num	unit_price	comp_1_price	comp_2_price
0	PROD_ELEC_001	Electronics	1	204.96	207.93	209.55
1	PROD_ELEC_001	Electronics	2	197.65	197.01	190.48
2	PROD_ELEC_001	Electronics	3	189.86	196.04	179.34
3	PROD_ELEC_001	Electronics	4	197.73	202.65	190.66
4	PROD_ELEC_001	Electronics	5	199.86	195.40	206.08

```
========== FIRST 5 ROWS ==========
```

	product_id	product_category_name	week_num	unit_price	comp_1_price	comp_2_price
0	PROD_ELEC_001	Electronics	1	204.96	207.93	209.55
1	PROD_ELEC_001	Electronics	2	197.65	197.01	190.48
2	PROD_ELEC_001	Electronics	3	189.86	196.04	179.34
3	PROD_ELEC_001	Electronics	4	197.73	202.65	190.66
4	PROD_ELEC_001	Electronics	5	199.86	195.40	206.08

```
========== LAST 5 ROWS ==========
```

	product_id	product_category_name	week_num	unit_price	comp_1_price	comp_2_pri
1035	PROD_SPRT_004	Sports & Outdoors	48	86.65	85.71	83.
1036	PROD_SPRT_004	Sports & Outdoors	49	87.74	91.40	85.
1037	PROD_SPRT_004	Sports & Outdoors	50	87.30	94.66	90.
1038	PROD_SPRT_004	Sports & Outdoors	51	86.56	87.25	92.
1039	PROD_SPRT_004	Sports & Outdoors	52	86.04	90.35	85.

```
========== DATASET SHAPE ==========
(1040, 13)

========== COLUMN NAMES ==========
['product_id', 'product_category_name', 'week_num', 'unit_price', 'comp_1_price', 'comp_2_pr

========== DATASET INFO ==========
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1040 entries, 0 to 1039
Data columns (total 13 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   product_id            1040 non-null   object
 1   product_category_name  1040 non-null   object
 2   week_num              1040 non-null   int64
```

```
 2   week_num              1040 non-null   int64
 3   unit_price            1040 non-null   float64
 4   comp_1_price          1040 non-null   float64
 5   comp_2_price          1040 non-null   float64
 6   comp_3_price          1040 non-null   float64
 7   freight_price         1040 non-null   float64
 8   product_score         1040 non-null   float64
 9   ps1                   1040 non-null   float64
 10  ps2                   1040 non-null   float64
 11  ps3                   1040 non-null   float64
 12  weekly_sales_volume   1040 non-null   int64
dtypes: float64(9), int64(2), object(2)
memory usage: 105.8+ KB

========== DATA TYPES ==========
product_id               object
product_category_name    object
week_num                  int64
unit_price              float64
comp_1_price            float64
comp_2_price            float64
comp_3_price            float64
freight_price           float64
product_score           float64
ps1                     float64
ps2                     float64
ps3                     float64
weekly_sales_volume       int64
dtype: object

========== MISSING VALUES ==========
```

	Missing Values	Missing Percentage
product_id	0	0.0
product_category_name	0	0.0
week_num	0	0.0
unit_price	0	0.0
comp_1_price	0	0.0
comp_2_price	0	0.0
comp_3_price	0	0.0
freight_price	0	0.0
product_score	0	0.0
ps1	0	0.0
ps2	0	0.0
ps3	0	0.0
weekly_sales_volume	0	0.0

```
Total missing values: 0

========== DUPLICATE ROWS ==========
Number of duplicate rows: 0
No duplicate rows found.

========== STATISTICAL SUMMARY ==========
```

	week_num	unit_price	comp_1_price	comp_2_price	comp_3_price	freight_price	pro
count	1040.000000	1040.000000	1040.000000	1040.000000	1040.000000	1040.000000	
mean	26.500000	105.284375	107.496635	103.964077	101.888231	8.333163	
std	15.015552	86.699783	88.904864	85.724865	84.219200	2.162901	
min	1.000000	21.190000	21.450000	20.880000	19.630000	4.500000	
25%	13.750000	42.637500	44.015000	42.605000	41.937500	6.437500	
50%	26.500000	64.340000	65.795000	64.170000	63.040000	8.410000	
75%	39.250000	142.470000	143.320000	139.565000	135.627500	10.175000	
max	52.000000	395.020000	403.860000	405.110000	413.340000	12.000000	

========== TRANSPOSED STATISTICAL SUMMARY ==========

	count	mean	std	min	25%	50%	75%	max
week_num	1040.0	26.500000	15.015552	1.00	13.7500	26.500	39.2500	52.00
unit_price	1040.0	105.284375	86.699783	21.19	42.6375	64.340	142.4700	395.02
comp_1_price	1040.0	107.496635	88.904864	21.45	44.0150	65.795	143.3200	403.86
comp_2_price	1040.0	103.964077	85.724865	20.88	42.6050	64.170	139.5650	405.11
comp_3_price	1040.0	101.888231	84.219200	19.63	41.9375	63.040	135.6275	413.34
freight_price	1040.0	8.333163	2.162901	4.50	6.4375	8.410	10.1750	12.00
product_score	1040.0	4.600000	0.158190	4.30	4.5000	4.600	4.7000	4.90
ps1	1040.0	4.553269	0.219170	4.00	4.4000	4.600	4.7000	5.10
ps2	1040.0	4.553173	0.259580	3.90	4.4000	4.600	4.7000	5.20
ps3	1040.0	4.688654	0.234564	4.10	4.5000	4.700	4.9000	5.30
weekly_sales_volume	1040.0	93.484615	12.648645	56.00	85.0000	93.000	102.0000	140.00

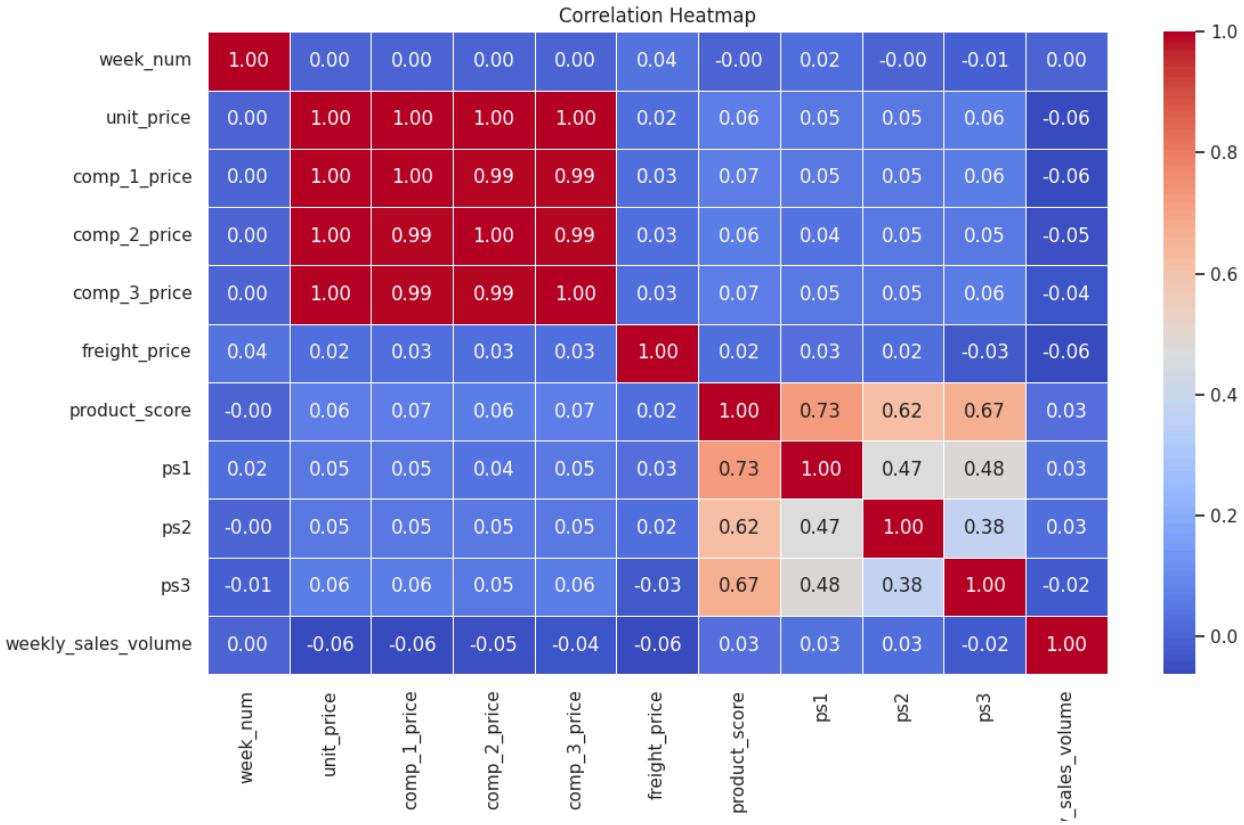
========== UNIQUE VALUES ==========

	Number of Unique Values
product_category_name	5
product_score	7
ps1	12
ps3	13
ps2	14
product_id	20
week_num	52
weekly_sales_volume	73
freight_price	575
comp_1_price	992
unit_price	994

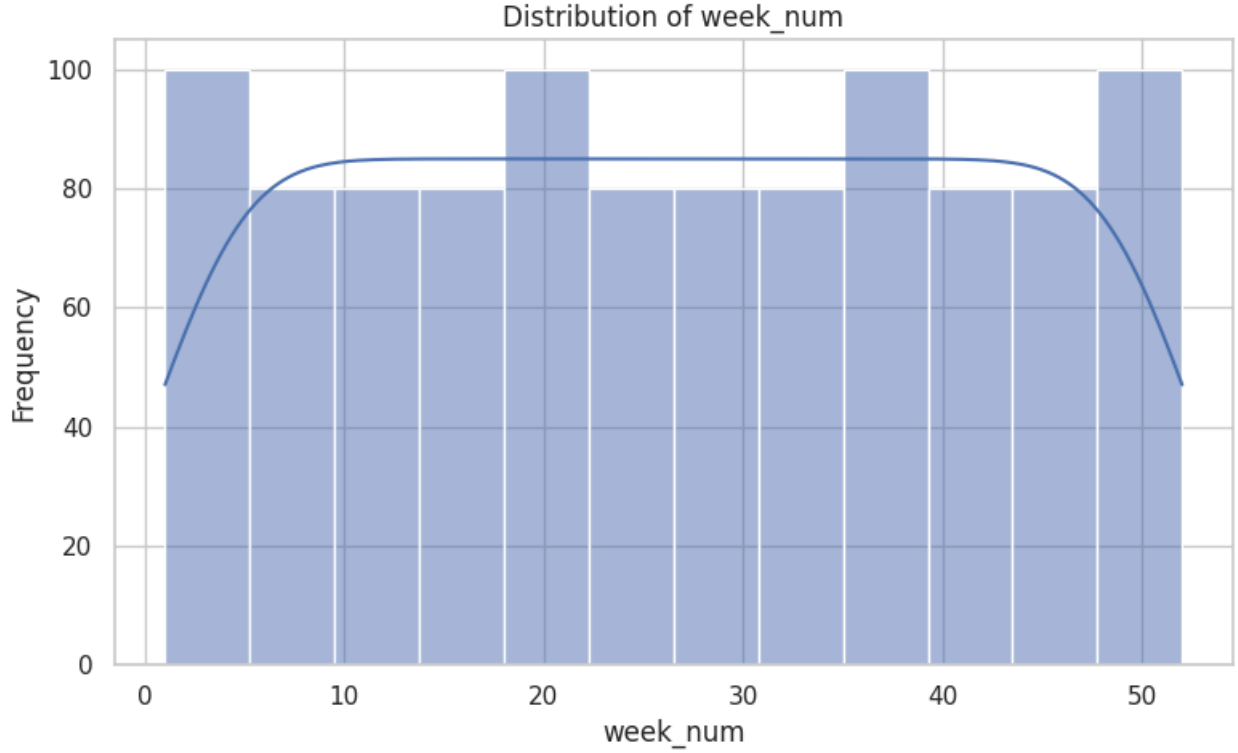
comp_2_price	998
comp_3_price	1000

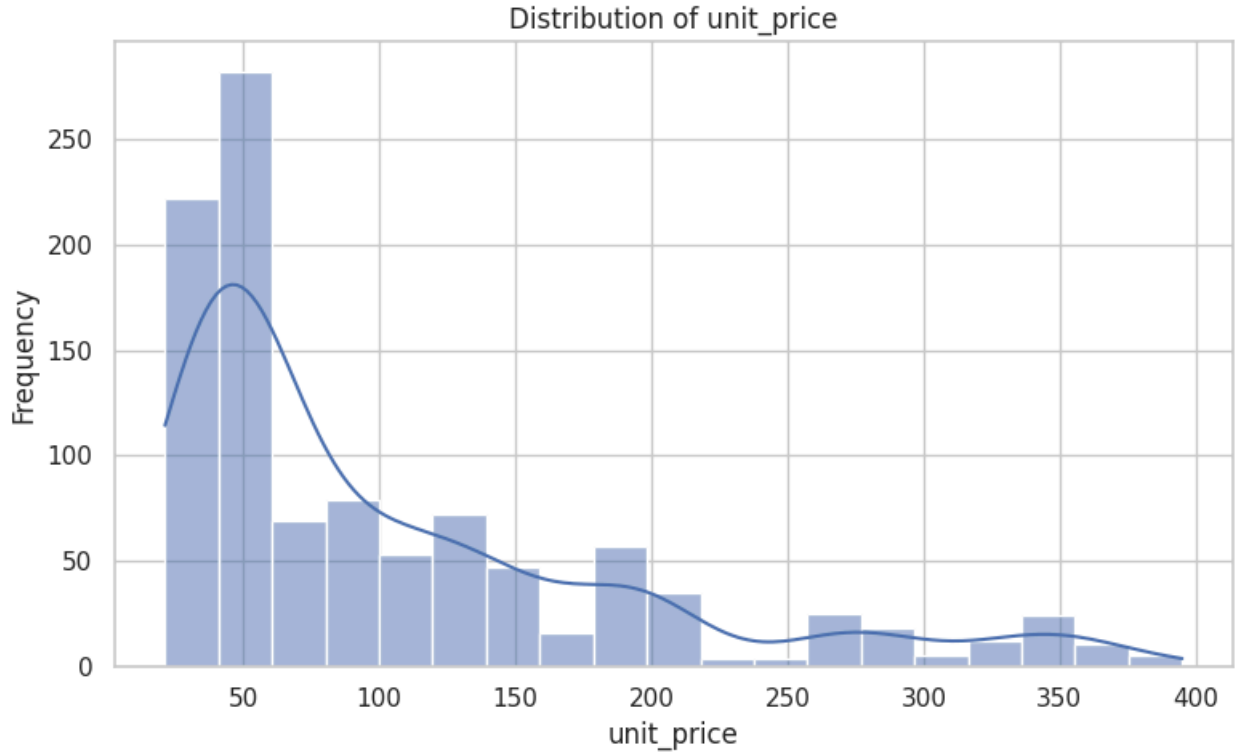
```
========== NUMERICAL COLUMNS ==========
['week_num', 'unit_price', 'comp_1_price', 'comp_2_price', 'comp_3_price', 'freight_price',

========== CATEGORICAL COLUMNS ==========
['product_id', 'product_category_name']

========== CORRELATION MATRIX ==========
```

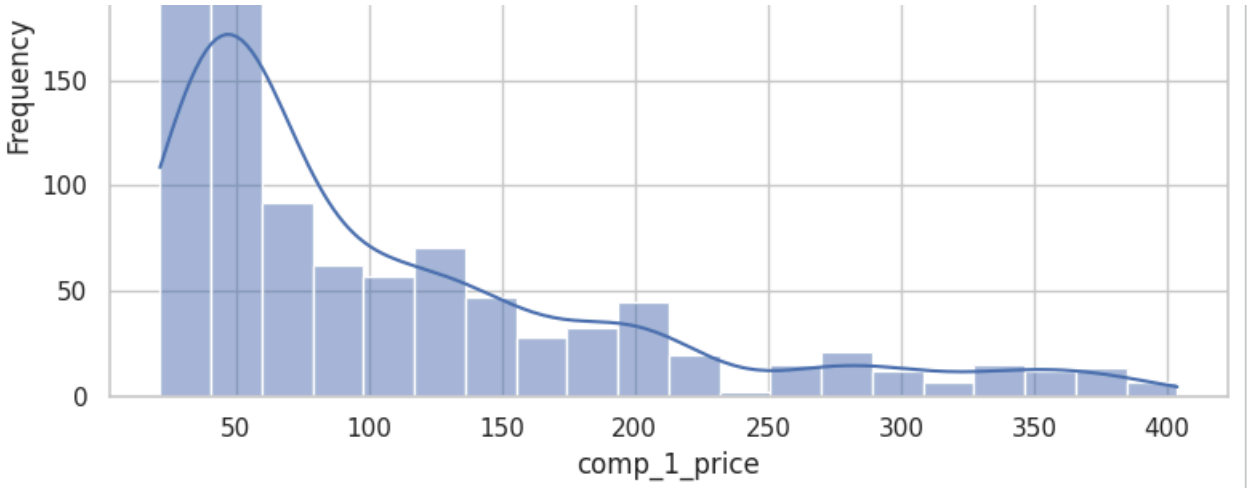
	week_num	unit_price	comp_1_price	comp_2_price	comp_3_price	fre
week_num	1.000000e+00	0.001514	0.002294	0.001514	0.000153	
unit_price	1.513854e-03	1.000000	0.997815	0.996676	0.995851	
comp_1_price	2.293538e-03	0.997815	1.000000	0.994475	0.993359	
comp_2_price	1.513750e-03	0.996676	0.994475	1.000000	0.992539	
comp_3_price	1.532745e-04	0.995851	0.993359	0.992539	1.000000	
freight_price	3.575342e-02	0.023664	0.025487	0.027066	0.025535	
product_score	-2.713537e-16	0.064809	0.065735	0.061803	0.065352	
ps1	2.178805e-02	0.047326	0.049360	0.044440	0.049054	
ps2	-4.506457e-03	0.048644	0.048031	0.050775	0.046381	
ps3	-1.207825e-02	0.058162	0.058627	0.053221	0.059902	
weekly_sales_volume	1.114864e-03	-0.060403	-0.059714	-0.050875	-0.040980	

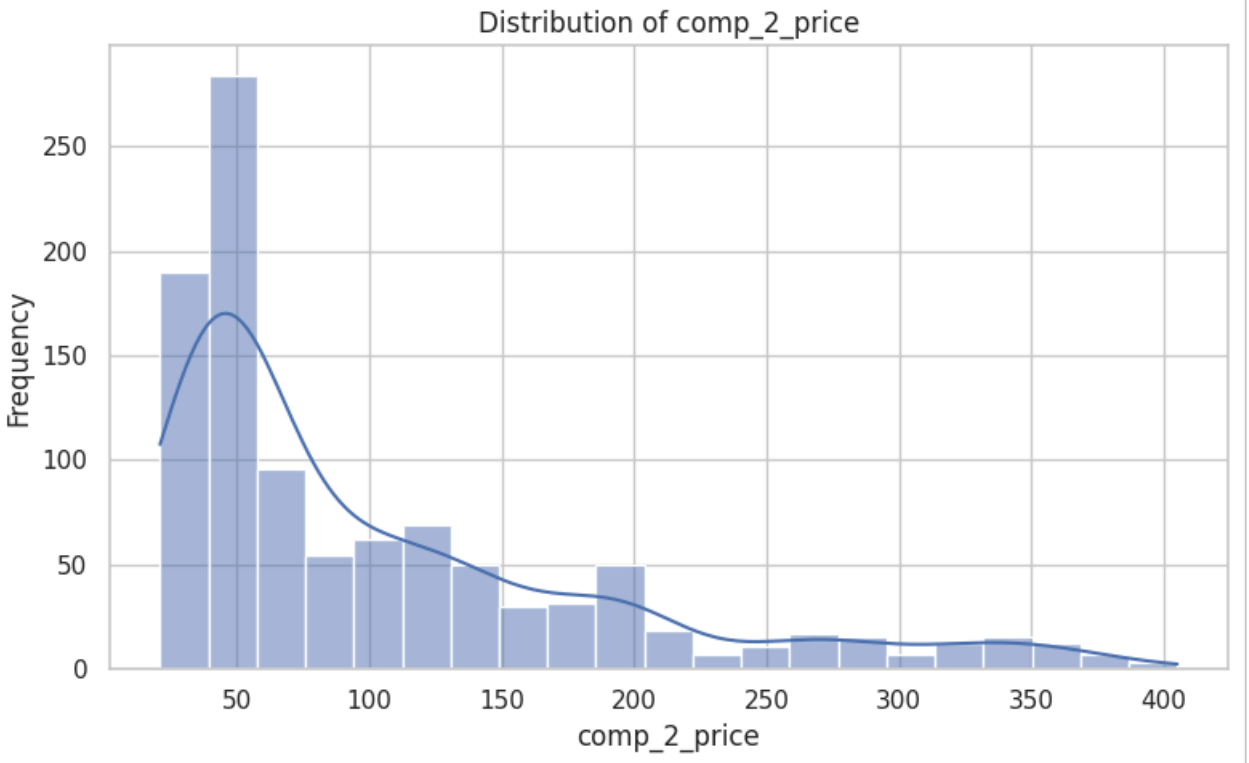


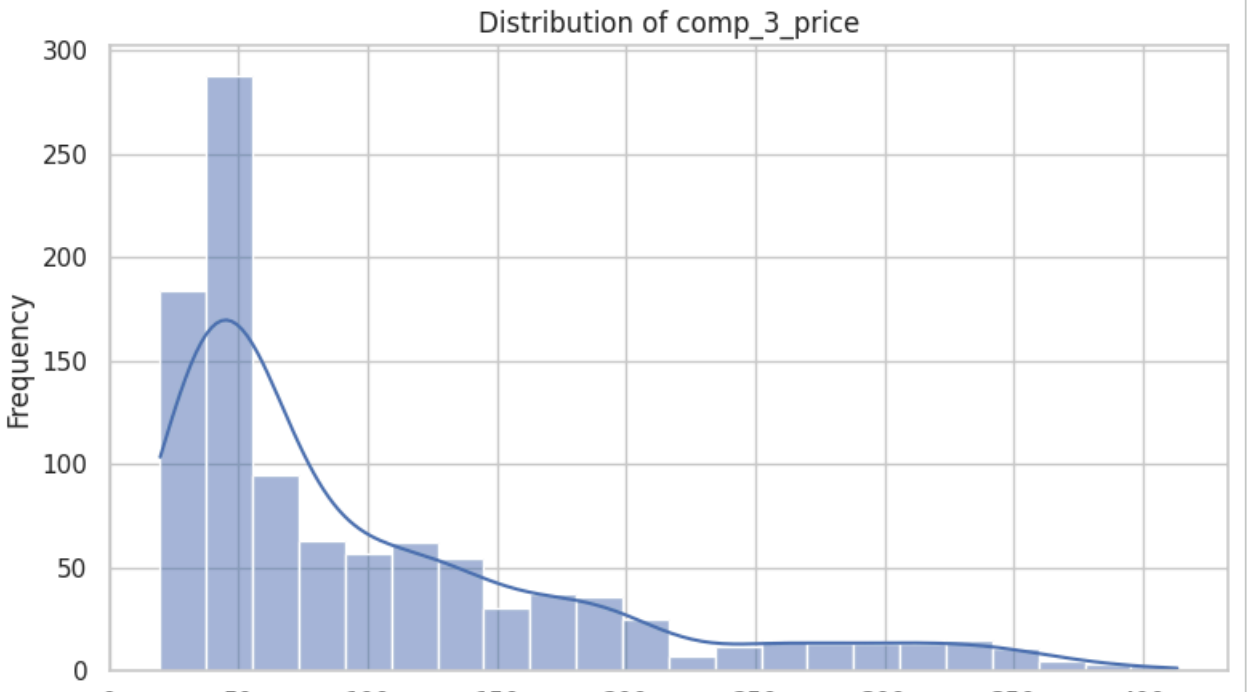
```
========== NUMERICAL DISTRIBUTIONS ==========
```

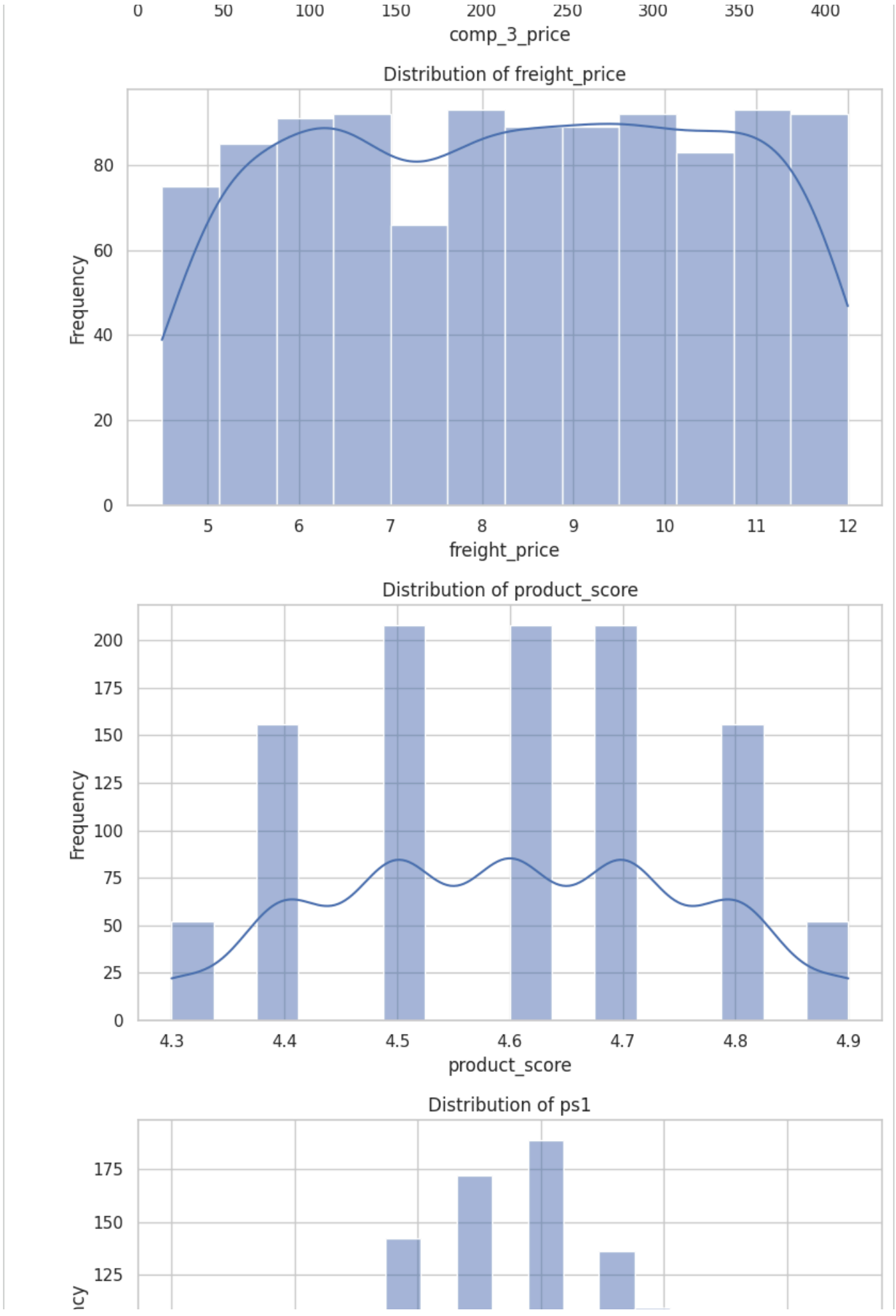


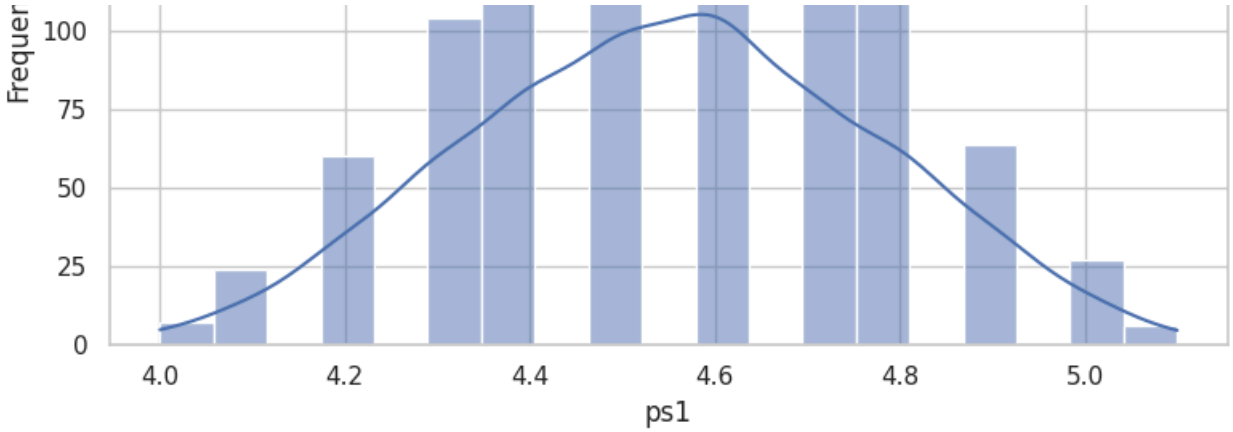


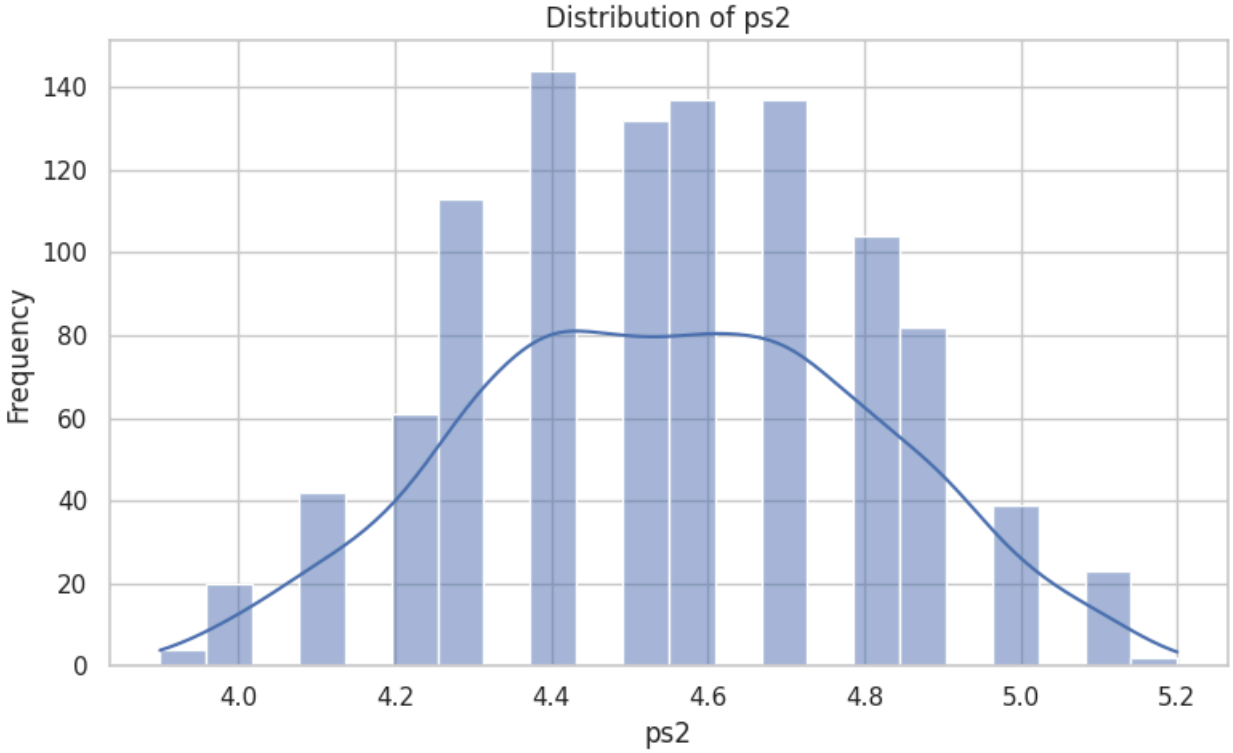

Frequency
0
50
100
150
50
100
150
200
250
300
350
400
comp_1_price

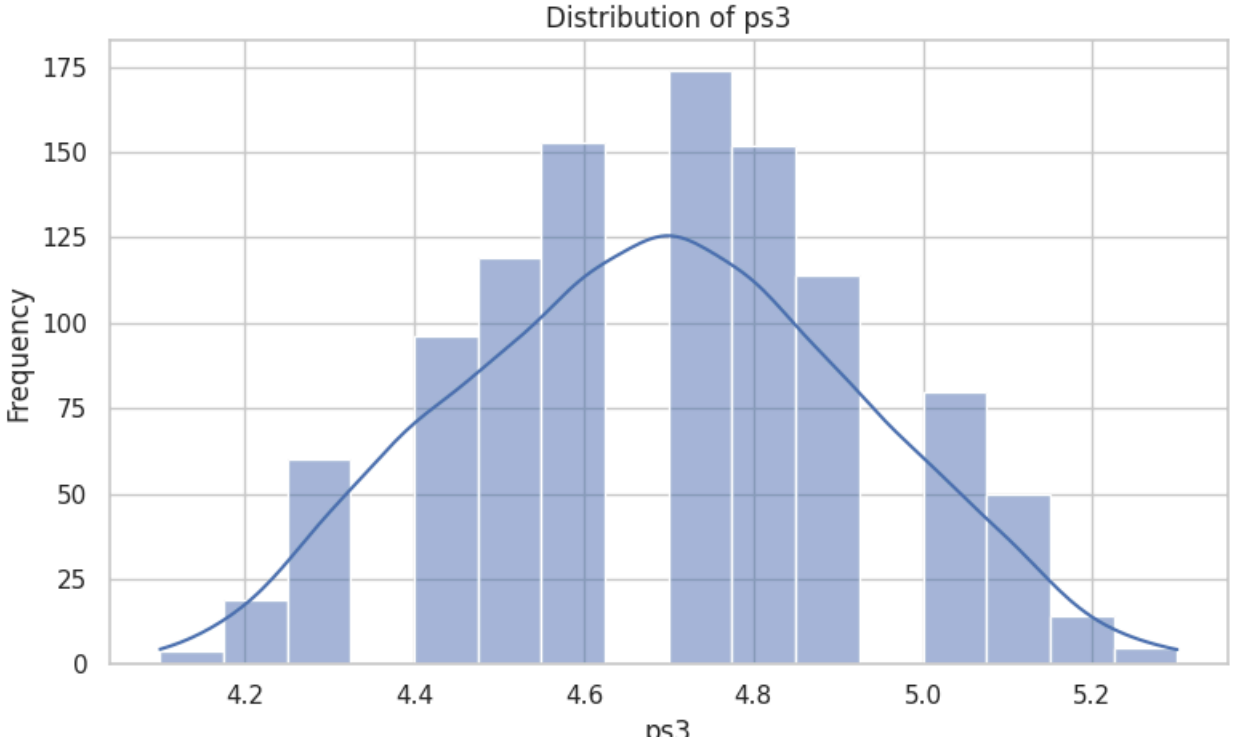

Distribution of comp_2_price
Frequency
0
50
100
150
200
250
50
100
150
200
250
300
350
400
comp_2_price

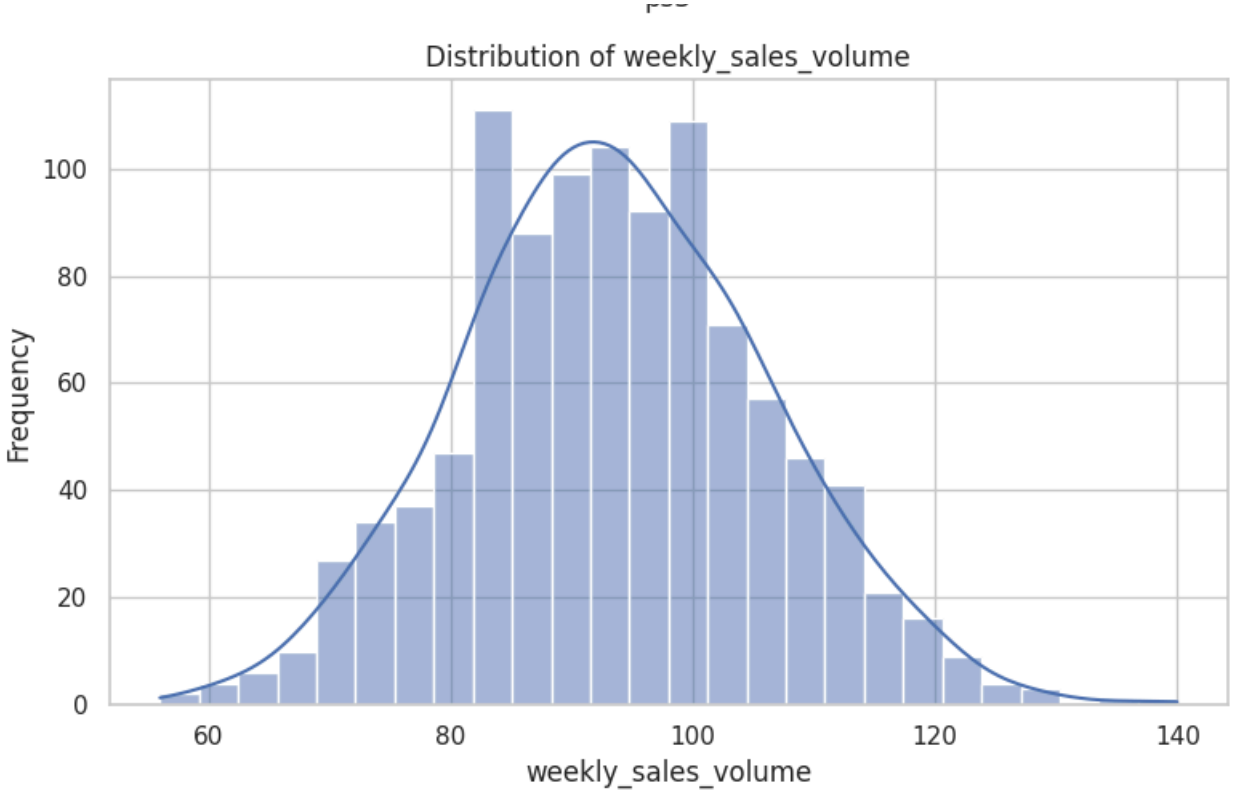

Distribution of comp_3_price
Frequency
0
50
100
150
200
250
300


0
50
100
150
200
250
300
350
400
comp_3_price
Distribution of freight_price
Frequency
0
20
40
60
80
5
6
7
8
9
10
11
12
freight_price
Distribution of product_score
Frequency
0
25
50
75
100
125
150
175
200
4.3
4.4
4.5
4.6
4.7
4.8
4.9
product_score
Distribution of ps1
175
150
125

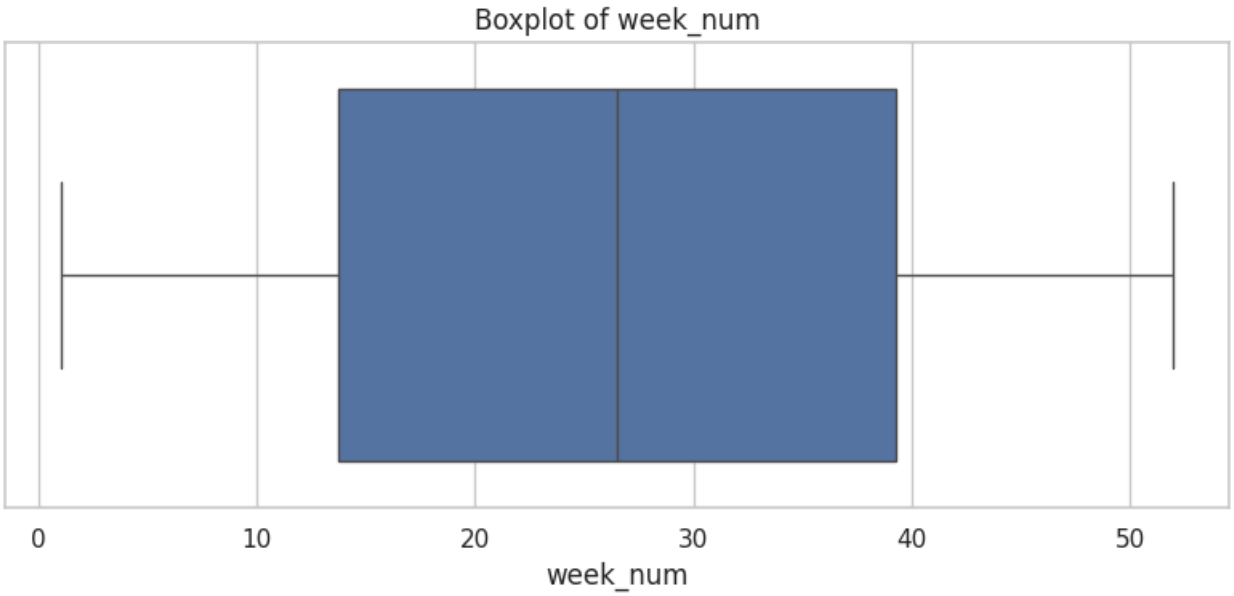

Frequer
100
75
50
25
0
4.0
4.2
4.4
4.6
4.8
5.0
ps1

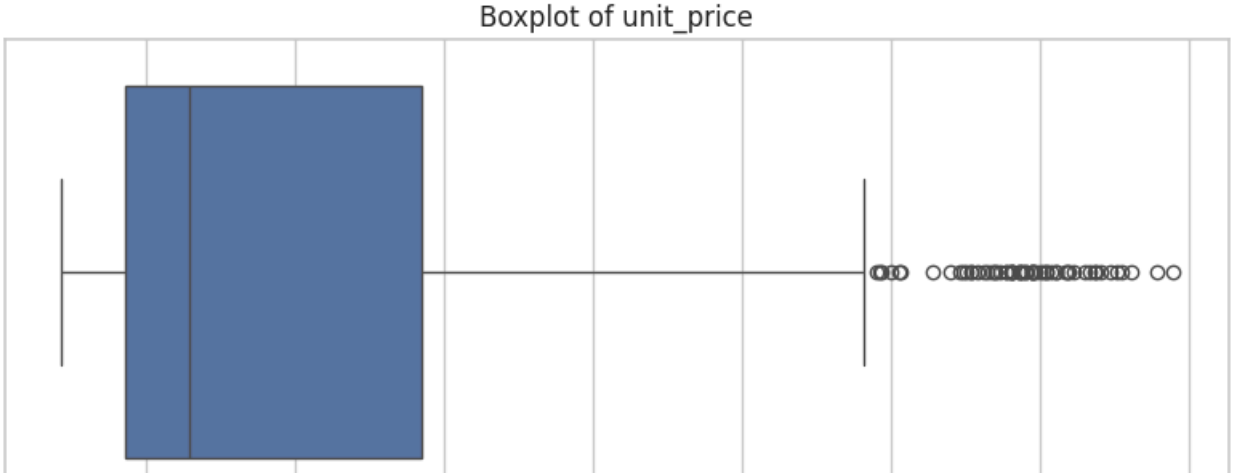

Distribution of ps2
Frequency
140
120
100
80
60
40
20
0
4.0
4.2
4.4
4.6
4.8
5.0
5.2
ps2


Distribution of ps3
Frequency
175
150
125
100
75
50
25
0
4.2
4.4
4.6
4.8
5.0
5.2
ps3

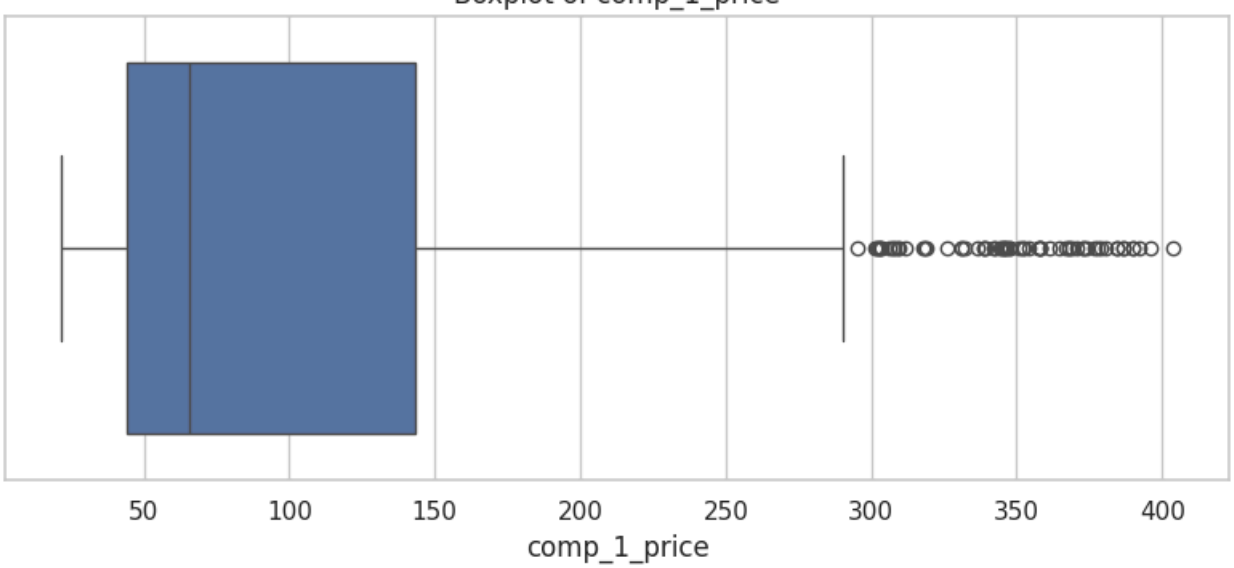

Distribution of weekly_sales_volume
Frequency
0
20
40
60
80
100
60
80
100
120
140
weekly_sales_volume

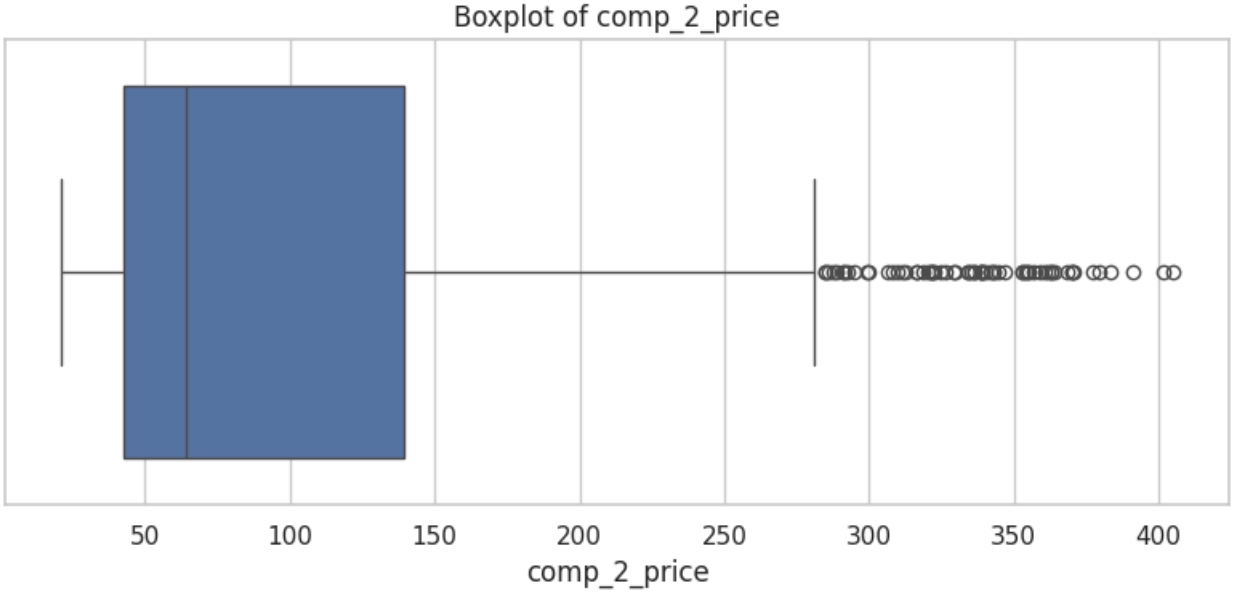
========== BOXPLOTS ==========

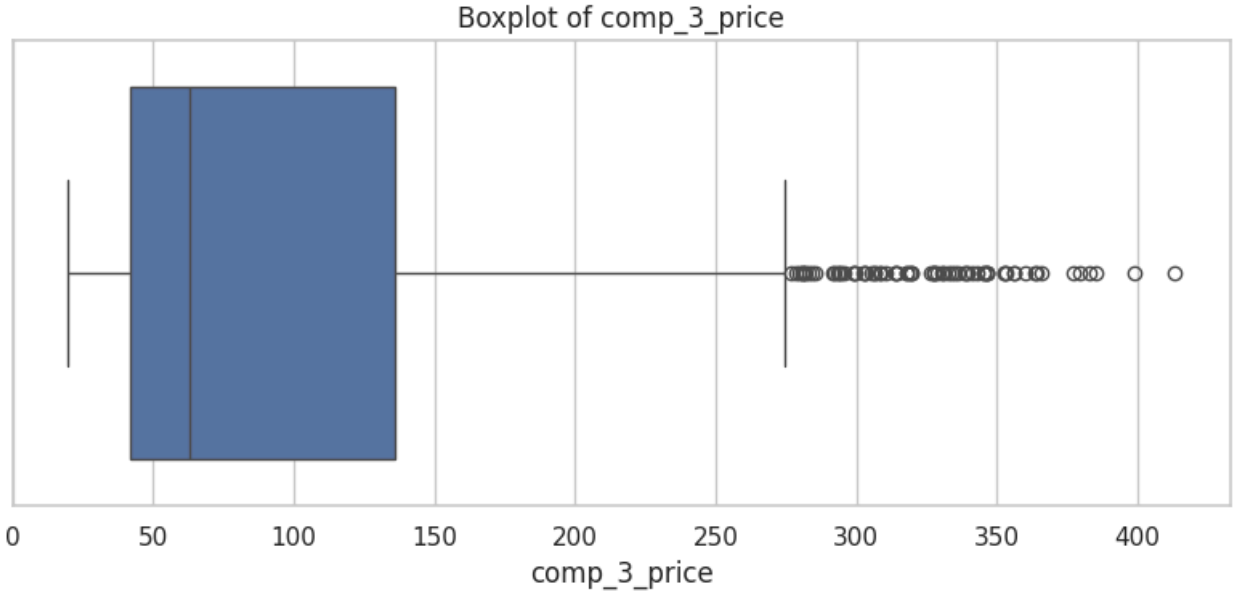

Boxplot of week_num
0
10
20
30
40
50
week_num

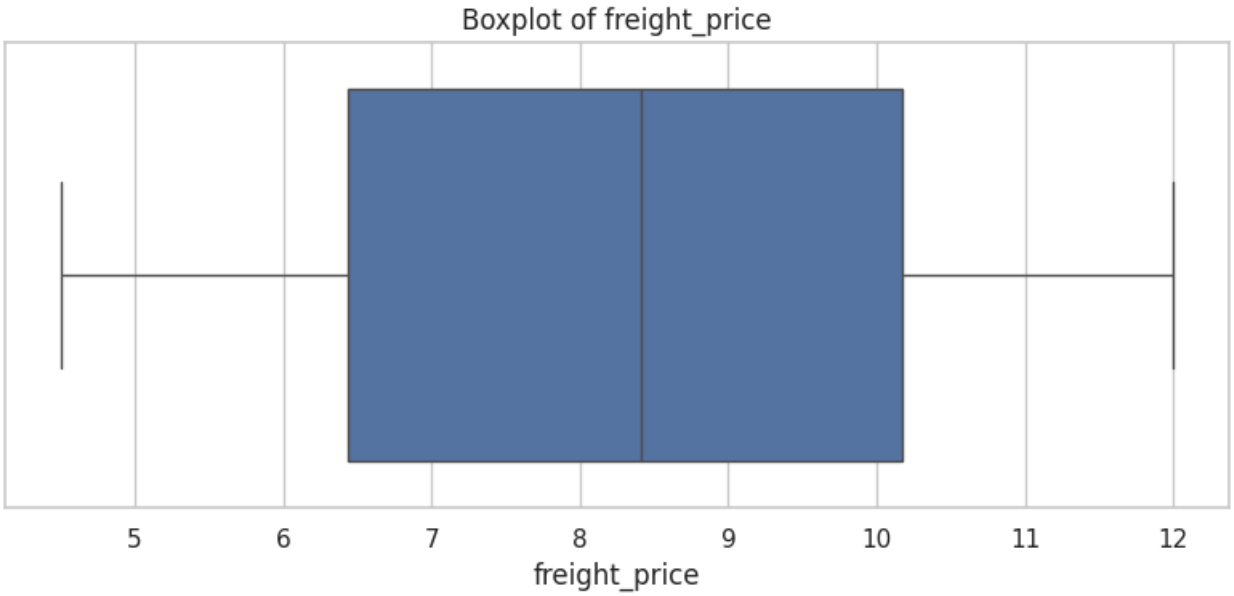

Boxplot of unit_price

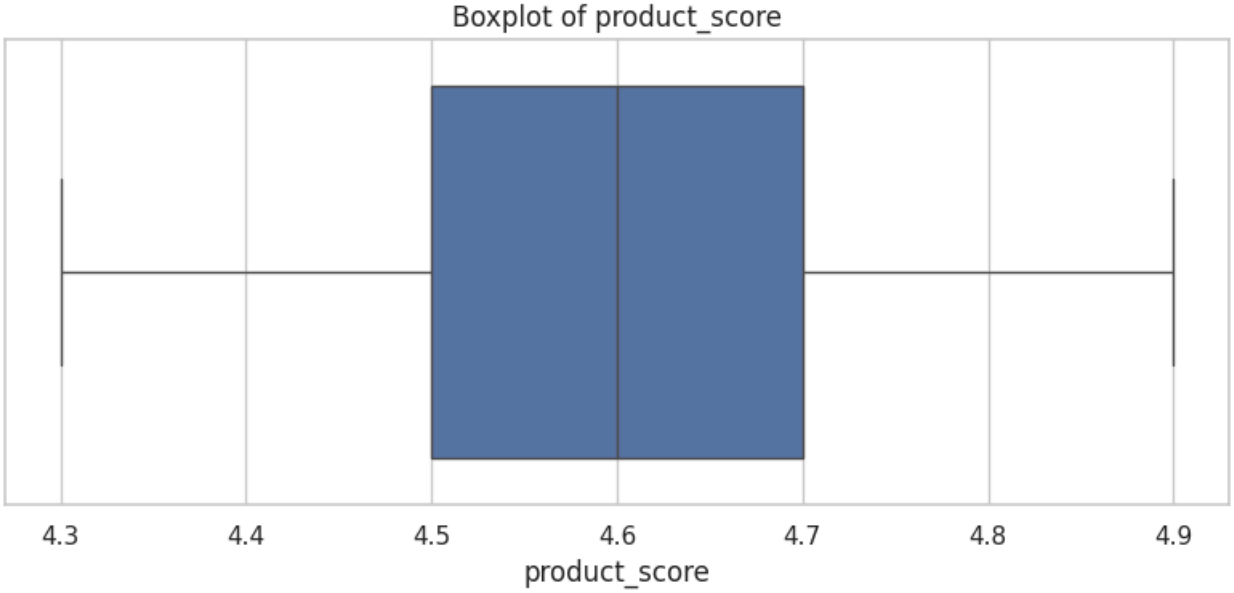

50
100
150
200
250
300
350
400
unit_price

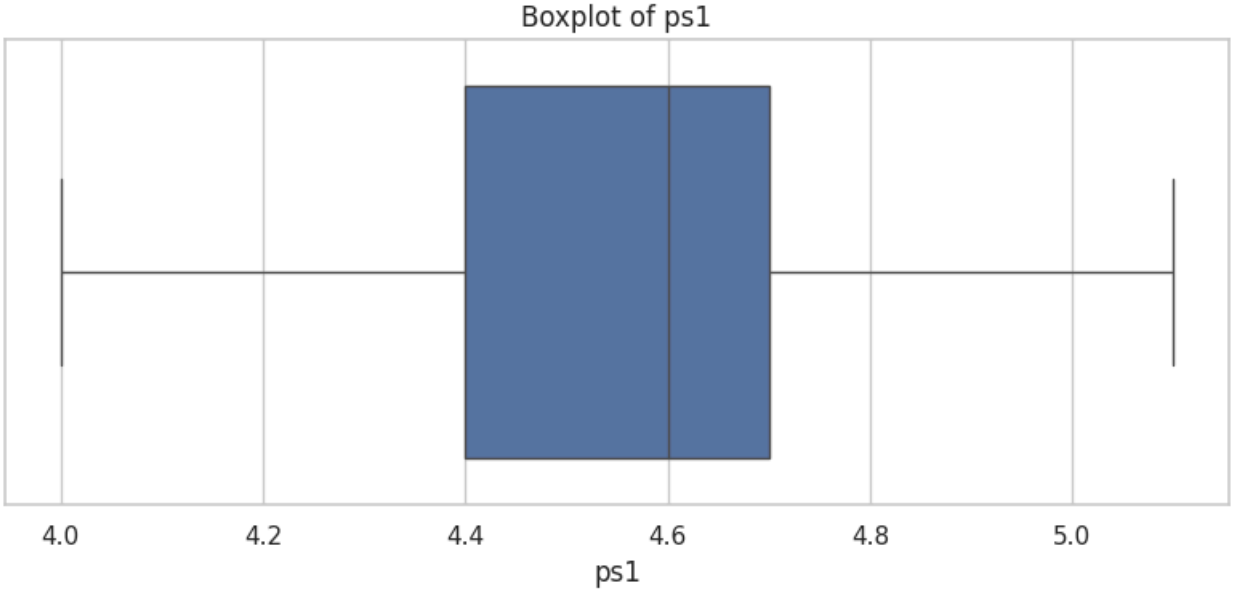

Boxplot of comp_1_price
50
100
150
200
250
300
350
400
comp_1_price

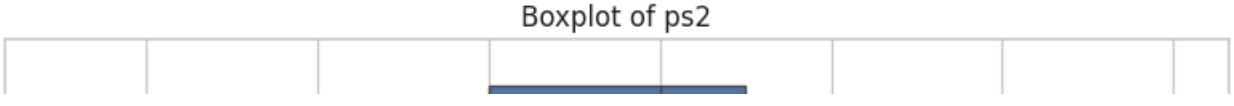

Boxplot of comp_2_price
50
100
150
200
250
300
350
400
comp_2_price

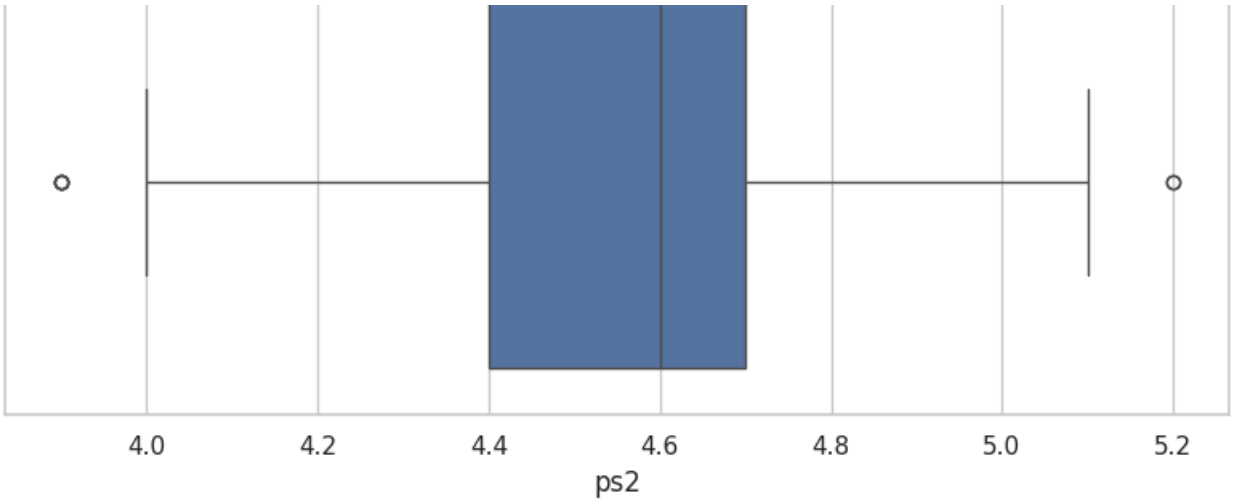

Boxplot of comp_3_price
0
50
100
150
200
250
300
350
400
comp_3_price

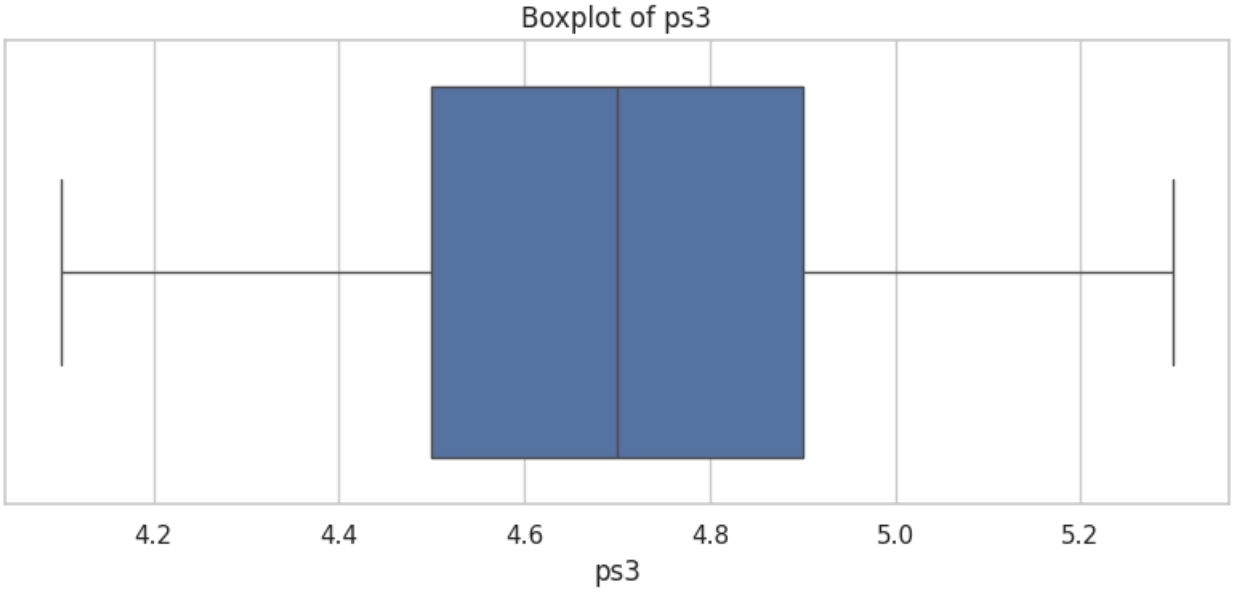

Boxplot of freight_price
5
6
7
8
9
10
11
12
freight_price

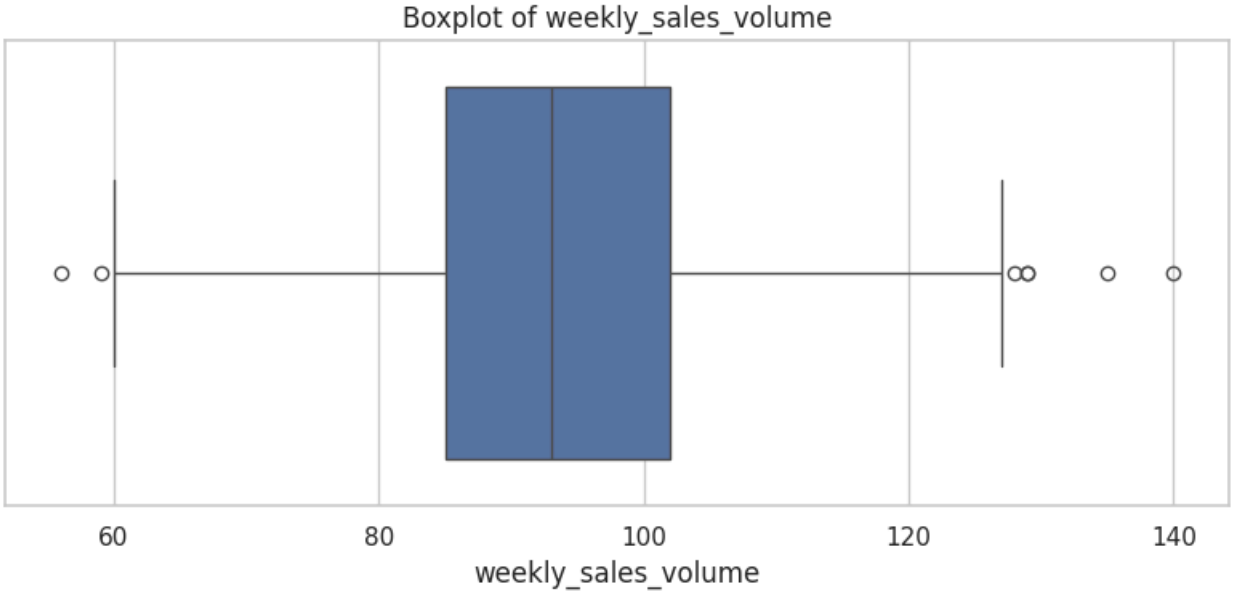

Boxplot of product_score
4.3
4.4
4.5
4.6
4.7
4.8
4.9
product_score


Boxplot of ps1
4.0
4.2
4.4
4.6
4.8
5.0
ps1


Boxplot of ps2







========== PRODUCT CATEGORY DISTRIBUTION ==========

product_category_name	Count
Electronics	260

```
  from google.colab import files

uploaded = files.upload()

filename = next(iter(uploaded))
df = pd.read_csv(filename)

print("Dataset loaded successfully!")
print("File:", filename)
print("Shape:", df.shape)
# RETAIL PRICE OPTIMIZATION DATASET
# COMPLETE EXPLORATORY DATA ANALYSIS (EDA)
# ============================================================


# ============================================================
# 1. IMPORT LIBRARIES
# ============================================================

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

import os
import glob

# Display settings
pd.set_option("display.max_columns", None)
pd.set_option("display.max_rows", 100)

sns.set_theme(style="whitegrid")

print("Libraries imported successfully!")


# ============================================================
# 2. LOAD DATASET
# ============================================================

# Automatically find CSV files in the current notebook folder
csv_files = glob.glob("*.csv")

print("CSV files found:")
for file in csv_files:
    print(file)

if len(csv_files) == 0:
    raise FileNotFoundError(
        "No CSV file found. Please upload 'retail_price_optimization.csv' "
        "to the notebook folder."
    )

# If the required file exists, use it
required_file = "retail_price_optimization.csv"

if required_file in csv_files:
```